PRACTICAL BIOINFORMATICS AND GENOMICS

A Comprehensive Textbook for Master Students
Health Biotechnologies and Medicine

Dr. Yassine Kasmi | Practical Course Companion

Preface: How to Use This Textbook

This textbook is your pre-course companion and in-course reference for the Practical Bioinformatics and Genomics course. It is written for Master students in Health Biotechnologies and Medicine who have foundational knowledge in molecular biology and elementary statistics. No prior bioinformatics experience is required — but curiosity is mandatory.

The book follows a problem-based learning philosophy. Every chapter opens with a realistic clinical or research scenario you might encounter as a future bioinformatician, translational researcher, or regulatory specialist. Before reading, try to formulate your own hypothesis. After reading, revisit the scenario and articulate your answer scientifically.

Structure of the Book

The textbook is organized into seven thematic chapters, each aligned with one session of the practical course:

- Chapter 1 — The NGS Revolution: Why Bioinformatics Matters in Medicine
- Chapter 2 — NGS Technologies: Illumina, Oxford Nanopore, and PacBio
- Chapter 3 — Library Preparation: From Sample to Sequencer
- Chapter 4 — Raw Data Formats and Public Genomic Databases
- Chapter 5 — Quality Control and Data Cleaning
- Chapter 6 — Variant Calling and Precision Oncology
- Chapter 7 — Single-Cell RNA Sequencing: Resolving Cellular Heterogeneity
- Chapter 8 — Metataxonomics: Sequencing the Human Microbiome

Each chapter includes: Learning Objectives, Key Concepts boxes, Clinical Case Studies, detailed explanations, comparative tables, important notes, common pitfalls, self-check exercises, a chapter summary, and a glossary. This structure is designed to prepare you before the practical sessions and serve as a reference during and after the course.

A Note on Language

Scientific terminology is presented in English throughout. Where relevant, German equivalents or context specific to German clinical practice and regulatory frameworks (IVDR, EU AI Act) are noted. The international scientific literature in bioinformatics is predominantly English; fluency in the English vocabulary of this field is itself a professional skill you will develop through this course.

Box Types and What They Mean

KEY CONCEPT

A fundamental principle or concept that underpins the analytical step being described.

Understanding this concept is required before executing any related commands.

PRACTICAL EXERCISE — Example Exercise Title

Step-by-step commands and tasks to execute on the HPC or your local environment.

Includes exact commands, parameter explanations, and expected outputs.

Expected Output / What to Look For:

What a successful result looks like. What numbers or patterns indicate correct execution.

INFORMATION

Background information that adds context without being directly required for the exercise.

Relevant biological, technical, or historical context.

WARNING

A critical mistake to avoid. These boxes flag decisions that can silently corrupt results.

In a clinical context, some of these errors could affect patient outcomes.

TIP

A practical shortcut, time-saving command option, or best-practice recommendation.

These are the lessons learned from running these pipelines in real projects.

```
# This is a code block. Commands are shown in monospace font on a dark background.
# Lines starting with '#' are comments — explanations, not commands.
$ fastqc sample_R1.fastq.gz sample_R2.fastq.gz --outdir qc_results/ --threads 4

# Parameters are explained inline as comments above or below the command.
# Copy commands exactly as shown — spacing and case matter in the Linux shell.
```

REVIEW QUESTIONS

Review Questions appear at the end of each chapter.

They test understanding at multiple cognitive levels (recall, application, analysis).

Attempting them before the practical session prepares you for deeper engagement.

Prerequisites

- Foundations of molecular biology (PCR, DNA replication, transcription, translation)
- Elementary statistics (mean, median, distributions, p-values, multiple testing correction)
- Basic familiarity with R (variable assignment, data frames, installing packages)
- Basic computer literacy (file management, browser, spreadsheets)

What This Textbook Does NOT Cover

By design, this book does not cover Linux/Shell scripting, HPC infrastructure, SLURM job scheduling, or Git version control. Those topics are addressed separately in the practical sessions. Here, we focus exclusively on the biological, technical, and bioinformatic concepts.

Software and Data

All software used in this course is pre-installed on the course server in Conda environments. The Git repository URL will be provided during the course. All exercise data, scripts, and solution notebooks are available in the repository under the exercises/ directory, organised by chapter.

A Note on Version Numbers

Bioinformatics software updates frequently. This textbook was written for the software versions current in 2024–2025. Command syntax and output formats may differ slightly in newer versions.

Always check the documentation for the version installed on the course server: tool_name --version or tool_name --help.

Other information.

https://classroom.google.com/c/ODU4MTcxMzA3NDAw?cjc=kly4hrhl

Create an Github account

Chapter 1: The NGS Revolution — Why Bioinformatics Matters in Medicine

CLINICAL CASE — Problem-Based Learning

Scenario: A 47-year-old woman presents with a newly diagnosed non-small-cell lung carcinoma (NSCLC). Standard histology confirms adenocarcinoma. Her oncologist orders a comprehensive genomic profiling panel. Two weeks later, the report identifies an EGFR exon 19 deletion with a variant allele frequency (VAF) of 34%. This single molecular finding changes the entire treatment strategy: she receives a tyrosine kinase inhibitor (TKI) instead of standard chemotherapy, achieving remission.

? What technology and analytical pipeline produced this report? Who interprets it? What happens if the bioinformatics pipeline misclassifies this variant?

Learning Objectives: After reading this chapter, you will be able to (1) explain the historical transition from Sanger to Next-Generation Sequencing; (2) describe at least six clinical applications of NGS and bioinformatics; (3) define the role of bioinformatics within precision medicine pipelines; (4) discuss ethical considerations of genomic medicine.

1.1 From Sanger to the Genomic Era: A Brief History

The sequencing of DNA has been central to biology since Frederick Sanger developed chain-termination sequencing in 1977. The Sanger method could sequence approximately 500 to 1,000 base pairs per reaction and was the workhorse of molecular biology for three decades. The Human Genome Project (HGP), completed in 2003, demonstrated both the power and the enormous cost of large-scale Sanger sequencing: it required 13 years of international collaboration and approximately 3 billion US dollars to sequence one human genome.

The sequencing landscape was fundamentally transformed with the introduction of second-generation, or next-generation sequencing (NGS) platforms in the mid-2000s. The Illumina Genome Analyzer (2006), building on the Solexa technology, enabled massively parallel sequencing: instead of sequencing one fragment at a time, millions of DNA fragments are sequenced simultaneously. This 'sequencing by synthesis' approach dramatically reduced cost, reduced time, and increased throughput. By 2023, the cost of sequencing a human genome had fallen below 1,000 USD and continues to decline.

DID YOU KNOW?

Moore's Law in genomics: The cost of sequencing a human genome has fallen faster than Moore's Law (which describes the doubling of transistor density every two years). From 2001 to 2023, the cost dropped from ~100 million USD to under 600 USD — a reduction of more than 5 orders of magnitude in two decades.

This cost reduction has enabled genomic sequencing to transition from purely research settings into routine clinical diagnostics — a transition that created an urgent need for bioinformaticians capable of processing, analyzing, and interpreting the vast quantities of data these technologies produce.

1.2 Clinical Applications of NGS and Bioinformatics

Today, NGS is integrated into clinical practice across multiple medical specialties. Understanding these applications provides the motivation and context for every analytical step you will learn in this course.

1.2.1 Oncology: Precision Cancer Medicine

Cancer is fundamentally a disease of the genome. Tumors accumulate somatic mutations — genetic alterations that arise in non-germline cells — that drive uncontrolled proliferation. NGS enables comprehensive molecular profiling of tumors, which informs treatment selection, predicts prognosis, and monitors treatment response.

Targeted gene panels: Panels of 50 to 500 cancer-relevant genes (e.g., TSO500, FoundationOne CDx, Oncomine Precision Assay) are sequenced at high depth (500x to 10,000x coverage) to detect single nucleotide variants (SNVs), insertions/deletions (indels), copy number variations (CNVs), and structural variants (SVs). Examples of actionable findings include EGFR mutations (TKI therapy in NSCLC), BRCA1/2 mutations (PARP inhibitor therapy in breast and ovarian cancer), and KRAS G12C (sotorasib in colorectal cancer).

Whole Exome Sequencing (WES) and Whole Genome Sequencing (WGS): These approaches sequence the entire protein-coding regions or the entire genome, respectively, enabling discovery of novel mutations beyond established panels. WGS is increasingly used in research and in rare cancer presentations.

Liquid Biopsy: Cell-free DNA (cfDNA) circulating in blood plasma contains tumor-derived circulating tumor DNA (ctDNA). NGS of liquid biopsies enables non-invasive early detection, treatment monitoring, and minimal residual disease (MRD) assessment. Ultra-deep sequencing (10,000x to 100,000x) and unique molecular identifiers (UMIs) are required to detect rare variants at allele frequencies below 1%.

IMPORTANT NOTE

The clinical interpretation of somatic variants requires classification according to established guidelines.

The Association for Molecular Pathology (AMP), the American Society of Clinical Oncology (ASCO),

and the College of American Pathologists (CAP) jointly published tiered classification criteria (2017):

Tier I: variants with strong clinical significance; Tier II: variants with potential significance;

Tier III: variants of unknown significance (VUS); Tier IV: benign or likely benign variants.

1.2.2 Rare Disease and Constitutional Genetics

Approximately 80% of rare diseases have a genetic basis. For millions of patients with rare, often undiagnosed conditions, NGS offers the possibility of a molecular diagnosis — sometimes after years of 'diagnostic odyssey'. Clinical exome sequencing (CES) and clinical genome sequencing (CGS) are increasingly reimbursed in European healthcare systems for patients with undiagnosed rare diseases, developmental disorders, or severe early-onset conditions.

Variant interpretation in constitutional genetics follows the American College of Medical Genetics (ACMG) five-tier classification system: Pathogenic, Likely Pathogenic, Variant of Uncertain

Significance (VUS), Likely Benign, and Benign. Databases such as ClinVar, DECIPHER, and OMIM are essential resources in this workflow.

1.2.3 Pharmacogenomics

Pharmacogenomics studies how an individual's genome influences their response to drugs. Variants in genes encoding drug-metabolizing enzymes (e.g., CYP2D6, CYP2C19, TPMT) or drug targets can predict efficacy, adverse drug reactions, and optimal dosing. The Clinical Pharmacogenomics Implementation Consortium (CPIC) provides evidence-based guidelines linking genotype to drug prescribing decisions. NGS panels targeting pharmacogenes are increasingly used in hospital pharmacy settings.

1.2.4 Infectious Disease and Pathogen Genomics

The COVID-19 pandemic demonstrated the transformative power of genomic epidemiology. Within weeks of the first SARS-CoV-2 cases, the virus genome was sequenced, published, and used to design diagnostic tests and vaccines. NGS-based whole-genome sequencing of pathogens enables outbreak tracking (phylogenetic analysis), antimicrobial resistance (AMR) profiling, and characterization of novel pathogens. Metagenomic sequencing — untargeted sequencing of all nucleic acids in a clinical sample — is emerging as a powerful diagnostic tool for culture-negative infections.

1.2.5 Prenatal and Reproductive Genetics

Non-invasive prenatal testing (NIPT) uses low-coverage whole-genome sequencing of maternal plasma cfDNA to screen for fetal chromosomal aneuploidies (trisomy 21, 18, 13) with high sensitivity and specificity. Preimplantation genetic testing (PGT) applies NGS to embryos during IVF to select chromosomally normal embryos or screen for known familial genetic variants.

1.2.6 Microbiome and Infectious Metagenomics

The human body harbors trillions of microorganisms — collectively the microbiome — that profoundly influence health and disease. 16S rRNA amplicon sequencing and shotgun metagenomics enable culture-independent characterization of microbial communities in the gut, lungs, skin, and other body sites. Dysbiosis of the gut microbiome has been associated with inflammatory bowel disease, colorectal cancer, metabolic syndrome, and the response to cancer immunotherapy. You will analyze microbiome data using QIIME2 and DADA2 in this course.

1.2.7 Epigenomics and Transcriptomics

Beyond DNA sequence, NGS platforms can interrogate the epigenome (ChIP-Seq for histone modifications, ATAC-Seq for chromatin accessibility, bisulfite sequencing for DNA methylation) and the transcriptome (RNA-Seq, single-cell RNA-Seq). These approaches are critical in understanding gene regulation, identifying biomarkers, and dissecting disease mechanisms at the molecular level.

Clinical Application	Key NGS Approach / Tool Used
Oncology — Targeted therapy selection	Gene panel sequencing, VCF annotation, ClinVar, OncoKB
Oncology — Liquid biopsy	Ultra-deep WGS/panel, UMI-corrected calling, Mutect2
Rare disease diagnosis	WES/WGS, ACMG classification, ClinVar, OMIM
Pharmacogenomics	Targeted pharmacogene panels, CPIC guidelines

Clinical Application	Key NGS Approach / Tool Used
Pathogen genomics / AMR	WGS, phylogenetics, ResFinder, MLST
NIPT	Low-coverage WGS, aneuploidy detection algorithms
Microbiome analysis	16S amplicon sequencing, QIIME2, DADA2
Transcriptomics	RNA-Seq, DESeq2/edgeR, pathway enrichment
Single-cell profiling	scRNA-Seq (10x Chromium), Seurat, UMAP clustering

1.3 The Role of the Bioinformatician in Clinical Genomics

A modern genomics laboratory is not complete without bioinformatics expertise. The bioinformatician occupies a critical node between the wet lab (where samples are processed and sequenced) and clinical interpretation (where findings influence medical decisions). Depending on the institution, this role may sit within a clinical laboratory, a research department, or an industrial bioinformatics team.

Key responsibilities of a clinical bioinformatician include: validating and maintaining bioinformatics pipelines; performing and documenting primary data analysis (demultiplexing, quality control, alignment, variant calling); interpreting results in the context of clinical guidelines; contributing to laboratory accreditation documentation (ISO 15189, ISO/IEC 17020); and staying current with evolving variant databases and clinical literature.

In Germany, clinical bioinformatics activities in IVD (In Vitro Diagnostics) settings are regulated under the EU IVDR (Regulation 2017/746), which came into full effect in 2022. Bioinformatics pipelines used for clinical reporting must be validated and documented according to these regulatory requirements. This is a rapidly evolving area where computational and regulatory expertise intersect.

⚠ COMMON PITFALL

A bioinformatics pipeline is not a black box. Every parameter choice — from quality score thresholds to variant calling filters — has biological and clinical consequences. Misconfigured pipelines can generate false-positive variant reports that lead to inappropriate treatment decisions, or false-negative reports that miss clinically relevant mutations. Critical thinking about each analytical step is a professional responsibility.

1.4 Ethical Dimensions of Genomic Medicine

As genomics becomes integrated into clinical practice, bioinformaticians must be aware of the ethical dimensions of their work. Key considerations include:

- **Patient privacy and data security: Genomic data is uniquely identifying. Even anonymized genomic data can potentially be re-identified. Institutional data governance frameworks and GDPR compliance are essential.**
  - Genomic databases must be stored on secure infrastructure. Access must be role-based and audited.
- **Secondary findings: WES/WGS may reveal variants unrelated to the primary indication (e.g., a BRCA1 variant detected during oncology workup). Policies for reporting or withholding secondary findings must be established a priori.**

- **Variants of Uncertain Significance (VUS): VUS are variants whose clinical significance is currently unknown. Reporting VUS without appropriate communication can cause significant patient anxiety. The proper handling of VUS is an ethical and scientific challenge.**

- **Health equity: Advanced genomic technologies are not equally accessible globally. Reference genomes are still predominantly derived from individuals of European ancestry, introducing potential biases in variant interpretation for underrepresented populations.**

- **AI and algorithmic transparency: AI-based variant callers (e.g., DeepVariant, Clair3) are entering clinical workflows. Understanding their training data, performance characteristics, and failure modes is essential for responsible use.**

SELF-CHECK EXERCISES

1. Name three somatic mutations in specific genes that determine targeted therapy selection in oncology.

2. What is the difference between a germline mutation and a somatic mutation?

3. A clinician asks: 'Why do I need bioinformatics if the sequencer produces results automatically?' How do you respond?

4. What does VAF stand for and why is it clinically important?

5. Describe two scenarios where a bioinformatics error could directly harm a patient.

CHAPTER SUMMARY

NGS reduced the cost of human genome sequencing by over 5 orders of magnitude since 2001.

Clinical applications include oncology panels, liquid biopsy, rare disease diagnosis, NIPT, pharmacogenomics,

infectious disease genomics, and microbiome profiling.

Bioinformaticians are critical nodes linking wet lab data to clinical interpretation.

Clinical genomics pipelines in Europe are regulated under IVDR; ISO 15189 governs lab quality.

Ethical considerations include data privacy (GDPR), secondary findings, VUS handling, and AI transparency.

Glossary — Chapter 1

NGS (Next-Generation Sequencing): Massively parallel sequencing technologies capable of sequencing millions of DNA fragments simultaneously, enabling whole-genome, exome, or targeted region sequencing at reduced cost and time.

WGS (Whole Genome Sequencing): Sequencing of the entire ~3.2 billion base pair human genome, including both coding and non-coding regions.

WES (Whole Exome Sequencing): Targeted sequencing of the ~1% of the genome encoding proteins (the 'exome'), comprising approximately 22,000 protein-coding genes.

Somatic mutation: A genetic alteration acquired in a non-germline cell during an organism's lifetime, not heritable by offspring. Found in cancer cells.

Germline mutation: A genetic alteration present in egg or sperm cells, heritable by offspring and present in all somatic cells of the organism.

VAF (Variant Allele Frequency): The proportion of sequencing reads supporting a variant allele at a given genomic position. Low VAF (<5%) is characteristic of liquid biopsy ctDNA.

ctDNA (Circulating Tumor DNA): Fragments of tumor-derived DNA circulating in blood plasma, detectable by liquid biopsy NGS.

VUS (Variant of Uncertain Significance): A genetic variant whose association with disease is currently unknown due to insufficient evidence.

UMI (Unique Molecular Identifier): Short random DNA barcodes ligated to individual library molecules before amplification, enabling PCR duplicate correction and accurate allele counting at low VAF.

IVDR: EU Regulation 2017/746 on In Vitro Diagnostic Medical Devices, governing clinical diagnostics including NGS-based assays.

Chapter 2: NGS Technologies — Illumina, Oxford Nanopore, and PacBio

CLINICAL CASE — Problem-Based Learning

Scenario: You are a bioinformatician in a university hospital. You receive two requests on the same day. Request A: Identify BRCA1/2 germline mutations in a 35-year-old BRCA-positive family. Request B: Characterize a novel bacterial pathogen from a patient with a culture-negative severe pneumonia — full characterization including structural variants and methylation patterns needed. You have access to Illumina NovaSeq 6000, Oxford Nanopore GridION, and PacBio Sequel IIe.

? Which sequencing platform would you choose for each request? What are the trade-offs? Could one platform serve both needs?

Learning Objectives: After reading this chapter, you will be able to (1) explain the sequencing-by-synthesis (SBS) mechanism of Illumina; (2) describe the nanopore sensing principle of Oxford Nanopore Technology (ONT); (3) explain PacBio SMRT sequencing; (4) compare platforms across key parameters (read length, accuracy, throughput, cost); (5) select the appropriate platform for a given biological question; (6) interpret raw data formats specific to each platform.

2.1 Illumina Sequencing by Synthesis (SBS)

Illumina is the dominant NGS platform in clinical and research genomics, accounting for the majority of sequencing data generated worldwide. Its dominance rests on high accuracy, high throughput, and a mature ecosystem of validated clinical workflows.

2.1.1 The SBS Mechanism

Illumina sequencing uses a cluster-based approach on a flow cell. DNA library molecules — fragments of approximately 150 to 500 bp with ligated platform-specific adapters — are loaded onto a flow cell surface coated with oligonucleotides complementary to the adapters. Each fragment undergoes bridge amplification, producing a clonal cluster of approximately 1,000 identical copies. This amplification step is critical: it creates sufficient signal for optical detection.

Sequencing proceeds by cyclic reversible termination. In each cycle, all four nucleotides — each labeled with a distinct, cleavable fluorescent dye and carrying a reversible 3'-blocker — are incorporated simultaneously by a polymerase. The blocker ensures that only one nucleotide is incorporated per cycle. After incorporation, the flow cell is imaged: each cluster emits a fluorescence signal corresponding to the incorporated nucleotide. The dye and blocker are then chemically cleaved, and the cycle repeats. Modern Illumina instruments perform 50 to 300 cycles, generating 50 to 300 bp reads.

Paired-end sequencing: After completing the first read (Read 1), the sequencing chemistry is reversed, the complement strand is synthesized, and Read 2 is sequenced from the opposite end of the fragment. This produces paired reads from both ends of the same DNA fragment, improving alignment accuracy (especially at repetitive regions) and enabling insert size determination. Paired-end sequencing is standard for WGS, WES, and RNA-Seq.

KEY CONCEPT

Phred Quality Score: Q = -10 × log10(Pe), where Pe = probability of base call error.
Q10 → 90% accuracy (1 error per 10 bases)
Q20 → 99% accuracy (1 error per 100 bases)
Q30 → 99.9% accuracy (1 error per 1,000 bases) ← clinical minimum benchmark
Q40 → 99.99% accuracy (1 error per 10,000 bases)
A sequencing run is considered high-quality when >80% of bases achieve Q≥30.

Illumina Platforms Comparison

Platform	Typical Application	Output / Read Length
MiSeq	Amplicons, small panels, microbiology	Up to 15 Gb | 2×300 bp
NextSeq 550	WES, transcriptomics, mid-scale labs	Up to 120 Gb | 2×150 bp
NovaSeq 6000	WGS population scale, clinical WGS	Up to 6,000 Gb | 2×150 bp
NovaSeq X	Ultra-high-throughput WGS	Up to 16,000 Gb | 2×150 bp
iSeq 100	Point-of-care, amplicon, field use	Up to 1.2 Gb | 2×150 bp

IMPORTANT NOTE

Phred Quality Score (Q-score): Each base call is associated with a quality score (Q) defined as:

Q = -10 × log10(P), where P is the probability that the base call is incorrect.

Q10 = 90% accuracy (1 error per 10 bases); Q20 = 99%; Q30 = 99.9%; Q40 = 99.99%.

Clinical NGS pipelines typically require >80% of bases at Q30 or above.

Q30 is a standard benchmark for Illumina run quality — reported in the run QC metrics.

2.1.2 Illumina Platforms and Clinical Relevance

Illumina manufactures a range of instruments designed for different throughput scales. Key platforms include:

Platform	Application / Scale	Output / Read Length
MiSeq	Small gene panels, amplicon sequencing, microbiology	Up to 15 Gb; 2×300 bp
NextSeq 550/2000	WES, transcriptomics, mid-scale clinical labs	Up to 360 Gb; 2×150 bp
NovaSeq 6000/X	WGS at population scale, large-cohort studies	Up to 6,000 Gb; 2×150 bp
iSeq 100	Point-of-care / field sequencing (amplicons)	Up to 1.2 Gb; 2×150 bp

2.1.3 Illumina Error Profile

Illumina sequencing is characterized by high base-call accuracy (>99.9% at Q30) but has specific systematic error modes: substitution errors are most common, particularly at the end of reads where

signal-to-noise ratio decreases; 'phasing errors' arise when a fraction of molecules in a cluster are one cycle behind or ahead; GC-bias affects coverage uniformity in GC-rich regions; repetitive sequences and homopolymers (runs of the same nucleotide) are poorly resolved because SBS cannot distinguish single from multiple insertions in a single cycle.

2.2 Oxford Nanopore Technology (ONT)

Oxford Nanopore Technology represents a fundamentally different sequencing paradigm. Instead of reading the fluorescent signal from incorporated nucleotides, ONT reads the ionic current disruptions caused by DNA or RNA molecules passing through a biological nanopore embedded in a synthetic membrane.

2.2.1 The Nanopore Sensing Principle

A nanopore is a protein channel (typically derived from Mycobacterium smegmatis porin A, MspA, or CsgG) embedded in an artificial lipid bilayer membrane. An electrical potential is applied across the membrane, driving an ionic current through the pore. When a DNA or RNA molecule is threaded through the pore by a motor protein (helicase), each ~5-mer of nucleotides within the pore creates a characteristic disruption in the ionic current. A neural network (the 'basecaller', implemented in software such as Guppy or Dorado) translates the raw current signal (stored in FAST5 or POD5 format) into a DNA or RNA sequence.

Key advantages of ONT include: (1) ultra-long reads — typical reads are 10 to 50 kb, and ultra-long preparations can exceed 2 Mb; (2) direct RNA sequencing without reverse transcription; (3) native base modification detection (e.g., 5-methylcytosine) directly from the current signal without bisulfite treatment; (4) real-time streaming data — sequencing can be stopped when sufficient coverage is achieved; (5) portable and accessible platforms (MinION, ~1,000 EUR device).

Key limitations include: higher per-read error rates compared to Illumina — R9.4.1 chemistry achieved ~95-97% single-read accuracy; R10.4.1 chemistry and Q20+ kits achieve >99% median accuracy; homopolymer resolution remains challenging; basecalling requires significant compute; systematic current drift can affect accuracy over long runs.

2.2.2 ONT Applications in Medicine

Rapid pathogen identification: ONT's real-time streaming enables sequencing-to-report in hours rather than days, critical for clinical microbiology. Structural variant (SV) detection: Long reads span complex genomic rearrangements inaccessible to short reads. Methylation profiling: Direct detection of 5mC, 5hmC, and 6mA without chemical treatment. Full-length transcript sequencing: Isoform characterization without assembly artifacts.

KEY CONCEPT
ONT Key Advantages:
• Ultra-long reads: median 15–50 kb; maximum reads >2 Mb (ultra-long protocol)
• Real-time data streaming: analysis begins while sequencing is ongoing
• Native base modification detection: 5mC, 5hmC, 6mA without bisulfite treatment
• Portable hardware: MinION (USB device, ~1,000 EUR)

ONT Key Limitations:
• Higher per-read error rate vs. Illumina (R10.4.1: ~99% median accuracy)
• Homopolymer resolution remains challenging
• Lower throughput than NovaSeq for standard applications

ONT Data Formats

Format	Description
FAST5 (legacy)	HDF5 binary format storing raw ionic current traces (squiggles). Large files (~5 GB per run hour).
POD5 (current)	Optimized binary format replacing FAST5. Smaller, faster read/write. Required for Dorado basecalling.
BLOW5/SLOW5	Community format for archiving ONT raw data; better random access than FAST5.
FASTQ (post-basecalling)	Standard output after Dorado basecalling. Same format as Illumina — compatible with most downstream tools.

2.3 PacBio SMRT Sequencing

Pacific Biosciences (PacBio) developed Single Molecule Real-Time (SMRT) sequencing, which also produces long reads but uses a different optical detection strategy.

SMRT sequencing uses zero-mode waveguides (ZMWs) — nanoscale chambers that confine laser illumination to the attoliter volume around a single polymerase molecule. Fluorescently labeled nucleotides are incorporated by the polymerase, and the fluorescence pulse (distinct color and duration for each nucleotide) is detected in real time. The circular SMRT bell library design allows the polymerase to traverse the same molecule multiple times, generating 'subreads' that are computationally combined into highly accurate consensus reads.

PacBio HiFi (CCS) reads: By generating multiple passes over the same molecule (typically 10 to 25 passes), PacBio achieves >99.9% accuracy (Q>30) on reads of 15 to 25 kb — combining the accuracy of Illumina with read lengths approaching ONT. HiFi reads have revolutionized de novo genome assembly, phasing of diploid genomes, and detection of full-length repeat expansions associated with neurological disease (e.g., Fragile X syndrome, spinocerebellar ataxias).

KEY CONCEPT
PacBio HiFi (CCS = Circular Consensus Sequencing):
• 15–25 kb reads at >99.9% accuracy (Q>30)
• Gold standard for: repeat expansion detection, de novo assembly, haplotype phasing
• Clinical use: Fragile X (CGG repeat), spinocerebellar ataxias, genome assembly
• PacBio Revio (2023): ~90 Gb HiFi data per day; significantly reduced cost

2.4 Comparative Platform Analysis

Parameter	Illumina (Short Read)	ONT / PacBio HiFi (Long Read)
Typical read length	75–300 bp	ONT: 10–50 kb (up to Mb); PacBio HiFi: 15–25 kb
Single-read accuracy	>99.9% (Q30+)	ONT R10.4.1: >99%; PacBio HiFi: >99.9%
Throughput (max)	~6 Tb/run (NovaSeq X)	ONT: ~100 Gb/run; PacBio: ~160 Gb/run
Cost per Gb	Lowest (~$5–10)	Moderate to high ($50–150)
Best for	SNVs, indels, WES, panels, RNA-Seq	SVs, phasing, repeat expansion, assembly, methylation
Native methylation	No (bisulfite treatment needed)	ONT: Yes; PacBio: Yes (with additional kits)
Clinical validation	Extensive (>15 years)	Growing; ONT approved for some clinical uses
Data format	FASTQ (BCL primary)	ONT: FAST5/POD5→FASTQ; PacBio: BAM/FASTQ

2.5 Ion Torrent Technology

Ion Torrent (Thermo Fisher Scientific) is a third short-read platform based on semiconductor sequencing. Instead of fluorescence, Ion Torrent detects the release of protons (H+) when a nucleotide is incorporated — a pH change measured by an ion-sensitive field-effect transistor (ISFET) beneath each well. Advantages include fast run times and no expensive optics. Limitations include systematic homopolymer errors and lower throughput compared to Illumina. The Ion GeneStudio S5 and Ion Torrent Genexus platforms are used in targeted oncology panels (e.g., Oncomine Focus Assay, AmpliSeq panels) in some clinical laboratories.

2.6 Choosing the Right Platform: A Decision Framework

Platform selection is determined by the biological question, required accuracy, desired read length, throughput needs, turnaround time, and cost constraints. Use the following decision logic:

Biological Question	Recommended Platform	Rationale
Germline SNVs/indels (WES/WGS)	Illumina NovaSeq	Highest accuracy, validated pipelines, lowest $/base
Somatic tumor panel (<500 genes)	Illumina NextSeq/NovaSeq	High depth, short TAT, validated clinical kits
Liquid biopsy (ctDNA, VAF<1%)	Illumina NovaSeq + UMI	Ultra-deep (10kx), UMI for error correction
Structural variants (SVs)	ONT GridION or PacBio Revio	Long reads span complex rearrangements
Repeat expansion (FMR1, C9orf72)	PacBio HiFi	Long + accurate reads span full repeat
De novo genome assembly	PacBio HiFi + ONT ultra-long	HiFi accuracy + ONT ultra-length for contiguity
Rapid pathogen ID (<6h)	ONT MinION/GridION	Real-time streaming, portable
16S microbiome profiling	Illumina MiSeq 2×300 bp	V3-V4 amplicon, DADA2 ASV pipeline
Shotgun metagenomics	Illumina NovaSeq (150 bp PE)	Cost-effective at required depth (~10 Gb/sample)

Biological Question	Recommended Platform	Rationale
DNA methylation (whole-genome)	ONT (native) or Illumina (WGBS)	ONT: no bisulfite; Illumina: validated WGBS
ChIP-Seq / ATAC-Seq	Illumina NextSeq	Short reads sufficient for peak calling

SELF-CHECK EXERCISES

1. Explain why SBS is unable to reliably sequence homopolymer runs longer than 6 nucleotides.

2. A researcher wants to identify Fragile X syndrome caused by CGG repeat expansion in the FMR1 gene.

   Standard Illumina WES fails to detect the expansion. Which platform would you recommend, and why?

3. What does Q30 mean in practical terms? If a base has Q20, what is the probability it is incorrect?

4. What is the role of the bridge amplification step in Illumina sequencing?

5. Compare the error profiles of Illumina and ONT R10.4.1. In which context could ONT errors be problematic?

CHAPTER SUMMARY

Illumina SBS: cyclic reversible termination, high accuracy (>Q30), short reads (75-300 bp), highest throughput, lowest cost per base. Gold standard for clinical SNV/indel detection.

Oxford Nanopore: ionic current sensing, ultra-long reads (10 kb to Mb), real-time, native methylation detection, improving accuracy (>Q20 with R10.4.1). Best for SVs, assembly, field sequencing.

PacBio HiFi: SMRT ZMW detection, 15-25 kb reads at >Q30 accuracy by circular consensus. Best for phasing, repeat expansions, and high-accuracy long-read applications.

Ion Torrent: semiconductor pH sensing, fast turnaround, targeted panels, prone to homopolymer errors.

Platform choice depends on: read length needed, accuracy requirements, throughput, cost, and application.

Glossary — Chapter 2

Flow cell: The consumable cartridge of an Illumina sequencer containing the glass slide or nanowell array on which library clusters are formed and sequenced.

Bridge amplification: The clonal amplification of a single library DNA molecule into a cluster of ~1,000 identical copies on the Illumina flow cell surface, generating sufficient signal for optical detection.

Paired-end sequencing: Sequencing both ends of a DNA library fragment, producing two reads (Read 1 and Read 2) from opposite ends of the same molecule.

Nanopore: A protein pore embedded in a membrane through which DNA/RNA is threaded; ionic current disruptions are measured to infer nucleotide sequence.

SMRT (Single Molecule Real-Time): PacBio's sequencing approach that monitors incorporation of fluorescent nucleotides by a single polymerase in real time within a zero-mode waveguide.

HiFi (CCS) reads: PacBio highly accurate long reads generated by circular consensus sequencing: multiple passes over the same circularized molecule correct errors.

FAST5 / POD5: ONT-specific raw signal data formats. FAST5 (legacy, HDF5-based); POD5 (newer, optimized format). Both store raw ionic current traces from the sequencer.

BCL: Binary Base Call format — the raw Illumina sequencer output, subsequently converted to FASTQ by bcl2fastq or DRAGEN.

Chapter 3: Library Preparation — From Biological Sample to Sequencer

CLINICAL CASE — Problem-Based Learning

Scenario: You receive a Bioanalyzer trace from a colleague. The electropherogram shows a broad peak ranging from 100 bp to over 2,000 bp with a prominent adapter dimer peak at 128 bp. The DIN (DNA Integrity Number) is 3.2. The sequencing run report shows 60% of reads are duplicates, 15% are adapter sequences, and only 50% pass quality filters. Your colleague insists the library was prepared correctly.

? What went wrong during library preparation? Which specific steps were likely suboptimal? What would a high-quality library trace look like?

Learning Objectives: After reading this chapter, you will be able to (1) describe each step of a standard NGS library preparation workflow; (2) explain the purpose of each enzymatic and quality-control step; (3) interpret Bioanalyzer/TapeStation traces to assess library quality; (4) explain the differences between WGS, WES, and targeted panel library preparation; (5) describe indexing and demultiplexing; (6) understand how library preparation artifacts appear in bioinformatics QC.

3.1 Overview of the Library Preparation Workflow

NGS library preparation is the set of biochemical steps that transform a biological sample (DNA, RNA, or other nucleic acids) into a sequencer-compatible library: a collection of DNA fragments with platform-specific adapter sequences that allow cluster formation (Illumina) or nanopore threading (ONT). Every artifact introduced in library preparation will propagate through sequencing and appear as a QC problem in bioinformatics analysis. Understanding library preparation is therefore essential for interpreting bioinformatics QC reports.

The standard Illumina library preparation workflow consists of the following major steps:

1. Sample collection and nucleic acid extraction
2. DNA/RNA quality assessment
3. DNA fragmentation (shearing)
4. End repair and A-tailing
5. Adapter ligation
6. Library amplification (PCR)
7. Size selection
8. Library quantification
9. Library quality assessment (Bioanalyzer/TapeStation)
10. Flow cell loading and sequencing

3.2 Step-by-Step Library Preparation

3.2.1 Sample Collection and Nucleic Acid Extraction

The starting material determines everything downstream. For genomic DNA sequencing, standard extraction uses silica column-based methods (e.g., Qiagen QIAamp), magnetic bead-based methods, or phenol-chloroform extraction. DNA quality is assessed by absorbance spectrophotometry (A260/A280 ratio: ideally ~1.8 for pure DNA; values <1.6 indicate protein contamination; A260/A230

ratio: ideally 2.0–2.2; low values indicate chaotropic salt or organic solvent contamination) and fluorometric quantification (Qubit, which measures double-stranded DNA specifically, unlike spectrophotometers that also detect RNA and single-stranded DNA).

For FFPE (formalin-fixed, paraffin-embedded) samples — a common source in clinical oncology — DNA quality is severely compromised by formalin-induced DNA-protein crosslinks, fragmentation, and cytosine deamination artifacts (C→T/G→A substitutions). FFPE-specific library preparation kits (e.g., KAPA HyperPlus FFPE) include additional repair steps to address these artifacts. The DIN (DNA Integrity Number, measured by the Agilent 2100 Bioanalyzer or 4200 TapeStation) quantifies DNA integrity on a scale from 1 (highly fragmented) to 10 (completely intact). WGS typically requires DIN >7; FFPE samples often have DIN 2–4.

IMPORTANT NOTE

FFPE Artifact Alert: Cytosine deamination in FFPE-derived DNA generates C→T artifacts that can be mistaken for real somatic mutations during variant calling.

FFPE-specific bioinformatics filters (e.g., GATK FilterMutectCalls with FFPE-specific parameters) must be applied to avoid reporting deamination artifacts as clinically relevant variants.

3.2.2 DNA Fragmentation

Most Illumina protocols require DNA fragments of 150–500 bp. High-molecular-weight genomic DNA (typically >10 kb) must be fragmented. The primary method is acoustic shearing (ultrasonication): the Covaris S/E/M-series instruments apply focused acoustic energy to shear DNA into fragments with defined size distributions. Parameters (duty cycle, peak incident power, duration) are optimized for each target insert size. Enzymatic fragmentation kits (e.g., KAPA HyperPlus, NEBNext Ultra II FS) use non-specific endonucleases and are increasingly used for their compatibility with automation and lower DNA input requirements.

For RNA-Seq, the RNA is first fragmented (by heat or RNase III digestion) before or after reverse transcription into cDNA. For targeted panels, PCR amplicon-based approaches bypass the fragmentation step entirely, instead using amplification of target regions directly from genomic DNA.

3.2.3 End Repair and A-tailing

Shearing produces DNA fragments with overhanging ends (5' or 3' overhangs) and potentially damaged 3' ends. End repair uses a combination of enzymatic activities: a 5'→3' polymerase fills in 5' overhangs; a 3'→5' exonuclease removes 3' overhangs; a polynucleotide kinase (PNK) phosphorylates 5' ends. This produces blunt-ended, 5'-phosphorylated fragments. A-tailing then adds a single adenine nucleotide to the 3' end of each fragment (using a non-templated 3' A-adding activity of Taq polymerase or a specialized enzyme). This 3' A-overhang prevents fragment self-ligation and provides a complementary base for the T-overhang on adapter molecules.

3.2.4 Adapter Ligation

Adapters are short double-stranded DNA oligonucleotides with a T-overhang (complementary to the A-tail) that are ligated to both ends of each fragment by T4 DNA ligase. Adapters contain: (1) the P5 and P7 sequences required for binding to the Illumina flow cell; (2) the sequencing primer binding sites; (3) sample index sequences (barcodes) for multiplexing; and (4) optionally, UMI sequences for duplicate correction.

Adapter dimer formation: If adapters ligate to each other instead of to library fragments — a common problem with low-input samples or suboptimal adapter:insert molar ratios — they produce adapter dimers of ~120–130 bp. Adapter dimers consume sequencing capacity, generate uninformative reads, and are detectable as a prominent peak at 128 bp on the Bioanalyzer trace. They should be removed by AMPure XP bead-based size selection or gel-based size selection.

⚠ COMMON PITFALL
Adapter dimer contamination: If not removed before sequencing, adapter dimer reads
will appear in FASTQ files as ~120 bp reads consisting entirely of adapter sequence.
FastQC will flag these as 'Overrepresented sequences' and 'Adapter content'.
Trimmomatic, Cutadapt, or fastp must be used to remove these reads computationally.
Prevention is better than cure: optimize adapter concentration (typically 10-fold molar excess
over fragment ends) and use bead purification after ligation.

3.2.5 Library Amplification (PCR)

After adapter ligation, the library is amplified by PCR using primers complementary to the adapter sequences. PCR serves two purposes: (1) it enriches for adapter-ligated fragments (removing unligated DNA) and (2) it increases library quantity to the amount required for loading on the flow cell. PCR amplification introduces duplicate reads: multiple copies of the same original library molecule produce identical reads that inflate apparent coverage and introduce bias. The PCR duplicate rate should be minimized by: (1) maximizing library input (use high-input protocols); (2) minimizing PCR cycles (4–8 cycles is typically sufficient for good-quality samples); (3) using UMIs to computationally identify true duplicates even when starting from the same molecule.

PCR-free libraries: For WGS of high-quality, high-quantity DNA, PCR-free library preparation can be used, eliminating PCR amplification bias and duplicate reads entirely. This requires higher DNA input (~1–3 µg) and a very efficient adapter ligation step.

3.2.6 Size Selection

Size selection removes fragments outside the desired insert size range (typically 300–400 bp for 150 bp paired-end sequencing, giving ~150 bp inserts). Methods include: AMPure XP magnetic beads (binding kinetics depend on PEG/NaCl concentration; different bead:sample ratios select different size ranges); agarose gel excision (lower throughput, useful for preparative sizing); BluePippin electrophoresis (automated gel-based size selection). Size selection removes adapter dimers, very small fragments, and oversized fragments that reduce sequencing efficiency.

3.2.7 Library Quantification and Quality Assessment

Before loading on the flow cell, the final library must be quantified and quality-assessed. Quantification is performed by qPCR using adapter-specific primers (e.g., KAPA Library Quantification Kit) — the most accurate method for quantifying sequenceable library molecules. Fluorometric quantification (Qubit) measures total dsDNA, including non-sequenceable fragments. Quality assessment uses capillary electrophoresis: the Agilent 2100 Bioanalyzer (chip-based) or Agilent TapeStation (tape-based) generates an electropherogram showing the size distribution and concentration of library fragments. Key features of a high-quality library trace include: a sharp, symmetric peak at the target insert size; no adapter dimer peak at 128 bp; no high-molecular-weight shoulder; average fragment size consistent with the fragmentation protocol.

3.3 Interpreting Bioanalyzer/TapeStation Traces

The electropherogram provides a visual fingerprint of library quality. Learning to interpret these traces is a critical wet-lab competency that directly informs bioinformatics QC expectations.

Bioanalyzer Feature	Interpretation and Action
Sharp peak at 350–400 bp	Good library. Proceed to sequencing.
Peak at 128 bp	Adapter dimers present. Perform additional bead purification.
Broad peak (100–2000+ bp)	Over-fragmentation or heterogeneous fragmentation. Optimize shearing conditions.
High-MW shoulder (>1000 bp)	Under-fragmentation. Increase shearing intensity or enzymatic digestion time.
DIN <5 (DNA)	Degraded input. Consider FFPE-specific protocol or obtain higher-quality sample.
No signal / very low signal	Failed library. Check ligation efficiency; verify adapter concentration.
RIN <7 (RNA for RNA-Seq)	Degraded RNA. Results will show 3' bias in transcript coverage.

3.4 Library Types: WGS, WES, and Targeted Panels

The library preparation approach is tailored to the sequencing application. Understanding these differences is essential for interpreting bioinformatics QC metrics and for understanding the limitations of each approach.

3.4.1 Whole Genome Sequencing (WGS) Libraries

WGS libraries are prepared without any enrichment step — all genomic regions are sequenced proportionally to their representation in the genome. Advantages: comprehensive coverage of coding and non-coding regions; detection of all variant types (SNVs, indels, CNVs, SVs, repeat expansions); uniform coverage across the genome; no capture bias. Disadvantages: high cost (30x WGS requires >100 Gb of sequencing data per sample); complex bioinformatics; regulatory validation is more extensive for clinical use. WGS is standard in rare disease genomics, population studies, cancer research, and increasingly in comprehensive clinical cancer profiling.

3.4.2 Whole Exome Sequencing (WES) Libraries

WES uses a capture hybridization step to enrich for the ~22,000 protein-coding genes (~50 Mb of the 3.2 Gb genome). After standard library preparation through adapter ligation and PCR, the library is hybridized to biotinylated RNA or DNA capture oligonucleotides (baits) targeting the exome. Non-target fragments are washed away; captured fragments are eluted, re-amplified, and sequenced. Commercial capture kits include the Agilent SureSelect Human All Exon, Roche SeqCap EZ, and IDT xGen Exome kits. WES achieves ~100x mean coverage of targeted regions at ~10-fold lower cost than WGS. Limitations: non-coding variants are missed; coverage at exon-intron boundaries is variable; CNV detection is less accurate than WGS; structural variant detection is limited.

3.4.3 Targeted Gene Panels

Targeted panels sequence a defined set of clinically relevant genes (50 to ~500 genes) at very high depth (500x to 10,000x+). The high depth enables detection of low-frequency somatic variants (VAF <5%) in tumor samples and liquid biopsies. Two approaches are used: hybridization capture (similar to WES, using baits targeting panel genes) and amplicon-based panels (multiplex PCR amplification of target regions — faster, lower input requirement, but introduces primer bias and is limited to detecting variants not overlapping primer binding sites). Clinical examples: TSO500 (Illumina, 523-gene oncology panel), Oncomine Precision Assay (Thermo Fisher), BRCA MASTR Dx (Multiplicom).

3.5 Multiplexing: Indexing and Demultiplexing

Running one sample per sequencing flow cell is expensive and wasteful. Multiplexing allows multiple samples to be combined in a single sequencing run. Each sample's library is tagged with a unique combination of short oligonucleotide index sequences (also called barcodes or i5/i7 indexes) integrated into the adapter sequences during ligation. The combination of i5 (read from the P5 end) and i7 (read from the P7 end) indexes uniquely identifies each sample.

Index hopping: On Illumina patterned flow cells (HiSeq 3000/4000, NovaSeq, NextSeq 2000), library molecules are seeded directly into wells (ExAmp clustering chemistry) without bridge amplification. Index hopping — the misassignment of an index sequence from one molecule to another — occurs at a rate of 0.1 to 1% and is caused by free index oligos in solution hybridizing to adjacent molecules. Solutions include: using unique dual indexes (UDI) instead of combinatorial dual indexes; removing free indexes by bead purification before pooling; and bioinformatic filtering of index-hopped reads.

Demultiplexing: After sequencing, the bcl2fastq software (Illumina) or DRAGEN reads the index sequences from each cluster and assigns reads to their originating sample FASTQ files. This requires the sample sheet — a comma-separated file specifying sample IDs, index sequences, and flow cell lane assignments. Mismatches between declared and actual index sequences (due to sequencing errors or sample sheet errors) result in 'undetermined reads' — reads that cannot be assigned to any sample. An undetermined read rate >5% should trigger investigation.

IMPORTANT NOTE

Sample Sheet Accuracy: The sample sheet is a critical document. Errors in index sequences

will result in reads being assigned to the wrong sample — a catastrophic bioinformatics problem

that may not be immediately obvious in downstream analysis.

Always double-check sample sheets against the adapter plate documentation before sequencing.

Index sequences must be entered in the correct orientation (some instruments require reverse complement of i5).

SELF-CHECK EXERCISES

1. A library shows 45% duplicate reads in its bioinformatics QC report. List three biological and three technical causes of high duplication rates.

2. Why does a PCR-free WGS library preparation tend to produce more uniform genomic coverage than a PCR-amplified library?

3. You observe an A260/A280 ratio of 1.4 for your DNA extract. What does this suggest, and how might it affect library preparation?

4. Explain the purpose of A-tailing in Illumina library preparation. What would happen if this step were skipped?

5. What is the difference between hybridization capture and amplicon-based targeted panel sequencing? In which clinical scenario would you prefer each?

CHAPTER SUMMARY

Library preparation converts biological samples into sequencer-ready tagged DNA fragment collections.

Key steps: extraction → fragmentation → end repair + A-tailing → adapter ligation → PCR → size selection → QC.

Bioanalyzer/TapeStation traces reveal adapter dimers (128 bp), fragmentation quality, and insert size.

WGS: no enrichment, comprehensive but expensive. WES: exome capture, ~100x coverage, cost-effective.

Targeted panels: 50–500 genes, ultra-deep sequencing (500x–10,000x), clinical cancer and BRCA panels.

Multiplexing uses i5/i7 dual indexes; demultiplexing assigns reads to samples via bcl2fastq or DRAGEN.

Library prep artifacts (adapter dimers, duplicates, FFPE deamination) propagate into bioinformatics QC.

Glossary — Chapter 3

DIN (DNA Integrity Number): A metric (1–10) measuring the integrity of genomic DNA by gel electrophoresis (Bioanalyzer). DIN 10 = fully intact; DIN 1 = completely degraded. WGS requires DIN >7.

Adapter dimer: A library artifact produced by ligation of two adapter molecules to each other instead of to a genomic fragment. Appears at ~128 bp on Bioanalyzer and generates uninformative sequencing reads.

AMPure XP: Ampure eXtra Pure magnetic beads used for size-selective purification of DNA fragments in library preparation, based on differential binding affinity in PEG/salt conditions.

End repair: Enzymatic processing of sheared DNA fragments to produce blunt, 5'-phosphorylated ends by filling overhangs (polymerase) and removing 3' overhangs (exonuclease).

A-tailing: Addition of a single adenine nucleotide to the 3' end of end-repaired fragments, complementary to the T-overhang on adapter oligonucleotides.

PCR duplicate: A sequencing read generated by PCR amplification of the same library molecule, resulting in identical reads that inflate apparent coverage. Marked by tools such as Picard MarkDuplicates or samtools markdup.

Index (barcode): A short oligonucleotide sequence incorporated in adapter molecules that uniquely identifies a sample in a multiplexed sequencing run.

Demultiplexing: The computational process of assigning sequencing reads to their originating samples based on their index sequences. Performed by bcl2fastq, DRAGEN, or Dorado (ONT).

Capture hybridization: An enrichment strategy using biotinylated bait oligonucleotides that hybridize to target genomic regions; used in WES and targeted panel library preparation.

FFPE: Formalin-Fixed, Paraffin-Embedded — the standard tissue preservation method for surgical specimens. Causes DNA degradation and C→T deamination artifacts requiring special library preparation and bioinformatics handling.

Chapter 4: Raw Data Formats and Genomic Databases

Before analyzing sequencing data, you must understand the formats in which it is stored and the repositories where it is deposited. This chapter introduces the essential file formats of NGS data and the major public databases that store and share genomic data.

4.1 The FASTQ Format

FASTQ is the universal format for storing raw sequencing reads. It is a text-based format in which each read is represented by four consecutive lines:

- **Line 1: Sequence identifier (header), beginning with '@'. Contains the instrument ID, run number, flow cell coordinates, and sample index.**
- **Line 2: The nucleotide sequence (A, T, G, C, and N for ambiguous bases).**
- **Line 3: A '+' separator (optionally followed by the sequence identifier again).**
- **Line 4: The quality string — one ASCII character per base, encoding the Phred quality score as Q + 33 (Phred+33 encoding, the current standard).**

FASTQ files are large: a 30x WGS run for a human genome (~100 Gb of sequence data) produces FASTQ files of approximately 150–200 Gb (before compression). FASTQ files are always compressed with gzip (.fastq.gz or .fq.gz) to reduce storage requirements by ~5-fold. Paired-end sequencing produces two FASTQ files per sample: *_R1_001.fastq.gz (Read 1) and *_R2_001.fastq.gz (Read 2). The order of reads must be synchronized between R1 and R2 files.

KEY CONCEPT
FASTQ = FASTA + Quality scores. The universal format for storing raw NGS reads.
Each read is represented by exactly 4 lines:
Line 1: '@' + read identifier (instrument, run, flow cell, coordinates, index)
Line 2: Nucleotide sequence (A, T, G, C, N for ambiguous)
Line 3: '+' separator (optionally followed by the identifier again)
Line 4: Quality string — one ASCII character per base (Phred+33 encoding)

A real FASTQ read looks like this:

```
@NB551234:45:H7YVKBGXF:1:11101:1234:1045 1:N:0:ATCACG
ACGTACGTACGTACGTACGTACGTACGTACGTACGTACGTACGT
+
IIIIIIIIIIIIIIIIIIIIHHHHHGGGGGGFFFFFEEEEEDDDDDCCCCC

# Line 1 breakdown:
# NB551234   = instrument serial number
# 45         = run number
# H7YVKBGXF  = flow cell ID
# 1:11101:1234:1045 = lane:tile:x:y coordinates
# 1:N:0:ATCACG = read number : filtered : control : sample index

# Line 4 quality encoding (Phred+33):
```

```
# 'I' = ASCII 73 → Q = 73-33 = 40  → 99.99% accuracy
# 'H' = ASCII 72 → Q = 72-33 = 39
# 'F' = ASCII 70 → Q = 70-33 = 37
# 'E' = ASCII 69 → Q = 69-33 = 36
# 'C' = ASCII 67 → Q = 67-33 = 34
```

TIP

Decoding quality scores manually: Q = ASCII(character) - 33

Quick reference: '!' = Q0 (worst) | '5' = Q20 | '?' = Q30 | 'I' = Q40 (best)

In Python: ord('I') - 33 = 40

In bash: echo 'I' | awk '{printf "%d\n", ord($0)-33}' (requires awk with ord)

PRACTICAL EXERCISE — Inspecting a FASTQ File

Navigate to your data directory and examine a FASTQ file without decompressing it.

```
# Count the number of reads in a gzipped FASTQ file:
$ zcat sample_R1.fastq.gz | wc -l | awk '{print $1/4}'

# Display the first 8 lines (= 2 complete reads):
$ zcat sample_R1.fastq.gz | head -8

# Check file size:
$ ls -lh sample_R1.fastq.gz

# View read length distribution (first 1000 reads):
$ zcat sample_R1.fastq.gz | awk 'NR%4==2 {print length($0)}' | head -1000 | sort | uniq -c

# Check if paired FASTQ files have equal read counts:
$ echo "R1: $(zcat sample_R1.fastq.gz | wc -l | awk '{print $1/4}')"
$ echo "R2: $(zcat sample_R2.fastq.gz | wc -l | awk '{print $1/4}')"
```

Expected Output / What to Look For:

Read count should be identical in R1 and R2 (paired-end requirement).

Read length should be uniform (e.g., all 150 bp for NextSeq 150 bp runs).

File sizes of R1 and R2 should be approximately equal.

4.2 SAM/BAM/CRAM Formats

After alignment to a reference genome, reads are stored in SAM (Sequence Alignment/Map) format or its binary compressed equivalent, BAM. CRAM is a more recent, highly compressed format that stores only the differences from the reference sequence.

A SAM file consists of a header section (lines beginning with '@') and an alignment section. Each alignment line contains 11 mandatory fields, including: the read name (QNAME), the bitwise FLAG (encoding mapping status, strand, paired-end status), the reference chromosome (RNAME), the mapping position (POS), the mapping quality (MAPQ), the CIGAR string (encoding alignment operations), and the sequence and quality of the read. The FLAG field is a 12-bit integer whose individual bits encode properties of the read — understanding FLAG values (e.g., 0x4 = read

unmapped; 0x10 = reverse strand; 0x400 = PCR duplicate) is essential for filtering in bioinformatics pipelines.

The CIGAR string (Compact Idiosyncratic Gapped Alignment Report) encodes how the read aligns to the reference: M (alignment match or mismatch), I (insertion in read relative to reference), D (deletion from read), S (soft clip — bases present in read but not in alignment), and N (skipped region — used in RNA-Seq for intron spans). For example, CIGAR '50M2I48M' means 50 matched bases, 2 inserted bases, then 48 matched bases.

A SAM file has two sections: the header (lines starting with '@') and the alignment records.

```
 # SAM HEADER LINES:
@HD  VN:1.6  SO:coordinate
# @HD = file-level metadata; VN = SAM version; SO = sort order

@SQ  SN:chr1  LN:248956422
# @SQ = sequence dictionary; SN = chromosome name; LN = chromosome length
# Every chromosome in the reference must have a @SQ line

@RG  ID:sample01  SM:patient_A  PL:ILLUMINA  LB:WES_lib1  PU:H7YVKBGXF.1
# @RG = read group; SM = sample name; PL = platform; LB = library; PU = platform
unit
# Read groups are CRITICAL for multi-sample GATK analysis — always add them

@PG  ID:bwa-mem2  PN:bwa-mem2  VN:2.2.1  CL:bwa-mem2 mem -t 8 ref.fa R1.fq R2.fq
# @PG = program record; records the exact command used to generate the file

# SAM ALIGNMENT RECORD (11 mandatory tab-separated fields):
# QNAME   FLAG   RNAME   POS    MAPQ  CIGAR     RNEXT   PNEXT   TLEN   SEQ          QUAL
read1    99     chr1    10001  60    100M      =       10201   200    ACGT...      IIII...
```

The FLAG Field — Decoding Read Properties

KEY CONCEPT
The FLAG field is a 12-bit integer. Each bit encodes one property:
0x1 (1) = read is paired
0x2 (2) = read mapped in proper pair
0x4 (4) = read unmapped
0x8 (8) = mate unmapped
0x10 (16) = read on reverse strand
0x20 (32) = mate on reverse strand
0x40 (64) = read 1 (first in pair)
0x80 (128) = read 2 (second in pair)
0x100 (256) = secondary alignment
0x200 (512) = fails quality filters
0x400 (1024) = PCR or optical duplicate
0x800 (2048) = supplementary alignment

PRACTICAL EXERCISE — Decoding SAM FLAGS

```
# Use samtools view with FLAG filters:


# Count properly paired reads:
$ samtools view -c -f 0x2 aligned.bam

# Count unmapped reads:
$ samtools view -c -f 0x4 aligned.bam

# Count PCR duplicate reads:
$ samtools view -c -f 0x400 aligned.bam

# Extract only properly paired, non-duplicate, non-secondary reads (MAPQ >= 20):
$ samtools view -b -f 0x2 -F 0x400 -F 0x100 -q 20 aligned.bam > filtered.bam

# Get alignment summary statistics:
$ samtools flagstat aligned.bam

# Online tool to decode any FLAG value:
# https://broadinstitute.github.io/picard/explain-flags.html
```

Expected Output / What to Look For:

samtools flagstat outputs: total reads, duplicates, mapped%, properly paired%, singletons%.

Acceptable WGS/WES: mapped >95%, properly paired >90%, duplicates <30%.

FLAG 99 = 0x63 = 0x1+0x2+0x20+0x40 = paired + proper pair + mate reverse + read1.

The CIGAR String — Encoding the Alignment

KEY CONCEPT

CIGAR (Compact Idiosyncratic Gapped Alignment Report) encodes per-base alignment operations:

M = alignment match (can be match OR mismatch — just aligned positions)

= = sequence match (exact match; less common)

X = sequence mismatch

I = insertion in read relative to reference

D = deletion from read (relative to reference)

N = skipped region from reference (intron span in RNA-Seq)

S = soft clip (bases in read but NOT in alignment, e.g. adapter remnants)

H = hard clip (bases removed from read entirely)

Example: 50M2I98M = 50 aligned + 2 inserted + 98 aligned = 150 bp read

Example: 10S90M50N50M = 10 bp soft-clipped + 90 aligned + 50 bp intron + 50 aligned

PRACTICAL EXERCISE — Inspecting Alignments with samtools and IGV

```
# Sort and index BAM (required for IGV and random-access operations):

$ samtools sort -@ 4 -o sorted.bam input.bam
$ samtools index sorted.bam

# View alignments at a specific locus (e.g., EGFR exon 19):
$ samtools view sorted.bam chr7:55174772-55174820 | head -20

# Calculate coverage depth at each position in a region:
$ samtools depth -r chr7:55174772-55174820 sorted.bam

# Get coverage statistics for the whole BAM:
$ samtools coverage sorted.bam | head -25

# In IGV (GUI): File → Load from File → sorted.bam
# Navigate to chr7:55,174,772-55,174,820 to see EGFR exon 19
# Color: mismatches shown in red; insertions as purple I marks
```

Expected Output / What to Look For:

- samtools depth output: chromosome, position, depth (one line per base).
- Coverage at EGFR exon 19 should be uniform for a WES sample (typically 80-200x).
- In IGV: reads should be uniformly distributed, no systematic artifacts at variant position.

4.3 VCF Format

The Variant Call Format (VCF) is the standard format for representing genetic variants. A VCF file has a header section (lines beginning with '##' for metadata and '#' for the column header line) followed by variant records, one per line. Each record contains: CHROM (chromosome), POS (1-based position), ID (variant identifier, e.g., rsID), REF (reference allele), ALT (alternative allele), QUAL (variant quality score), FILTER (PASS or failure reason), INFO (semicolon-separated key=value annotations), FORMAT (field definitions for sample columns), and one or more sample genotype columns.

gVCF (Genomic VCF): A variant of VCF that represents ALL genomic positions — both variant sites and reference sites (represented as non-variant blocks). gVCF format is used in GATK's joint genotyping workflow (GVCF mode of HaplotypeCaller) and is required for joint calling across cohorts. The gVCF allows sites with insufficient coverage to be explicitly represented as 'no call', avoiding the ambiguity of whether a reference genotype represents true homozygous reference or simply missing data.

KEY CONCEPT

VCF (Variant Call Format) is the universal standard for representing genetic variants.

File structure:

- ## lines: metadata (reference genome, tool version, filter definitions, INFO/FORMAT fields)
- # line: column headers (CHROM POS ID REF ALT QUAL FILTER INFO FORMAT SAMPLE1...)
- data lines: one variant per line

A real VCF record:

```
##fileformat=VCFv4.2
##reference=GRCh38
##FILTER=<ID=PASS,Description="All filters passed">
##FILTER=<ID=LowQual,Description="Low quality variant">
##INFO=<ID=AF,Number=A,Type=Float,Description="Allele Frequency">
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=AD,Number=R,Type=Integer,Description="Allelic depths">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read depth">
#CHROM  POS      ID          REF  ALT  QUAL  FILTER  INFO           FORMAT   TUMOR
NORMAL
chr7    55174775 rs121434568 T    G    .     PASS    AF=0.34;DP=287 GT:AD:DP
0/1:189:97:286 0/0:0:142:142


# Interpretation:
# chr7:55174775 = EGFR exon 19 position
# REF=T, ALT=G = T→G substitution (EGFR L858R is actually chr7:55,259,515 T>G in
GRCh38)
# QUAL=. = quality not reported (common for Mutect2 output)
# FILTER=PASS = variant passed all filters → use this variant
# AF=0.34 = 34% of reads support the ALT allele (tumor VAF)
# GT:AD:DP → Genotype: Allelic Depth (ref:alt): Total Depth
# TUMOR: 0/1 = heterozygous; 97 ref reads : 189 alt reads; 286 total depth
# NORMAL: 0/0 = homozygous reference; 0 alt reads; 142 total depth
```

PRACTICAL EXERCISE — Inspecting and Filtering VCF Files

Install bcftools if not available:

$ module load bcftools # on HPC

# View VCF header only:
$ bcftools view -h variants.vcf.gz | less

# Count total variants:
$ bcftools stats variants.vcf.gz | grep 'number of records'

# Filter: keep only PASS variants:
$ bcftools view -f PASS variants.vcf.gz -o pass_variants.vcf.gz -O z
$ bcftools index pass_variants.vcf.gz

# Filter by minimum VAF (AF >= 0.05 = 5%) and minimum depth (DP >= 30):
$ bcftools filter -i 'INFO/AF >= 0.05 && INFO/DP >= 30' pass_variants.vcf.gz

# Extract specific columns for quick review:
$ bcftools query -f '%CHROM\t%POS\t%REF\t%ALT\t%INFO/AF\n' pass_variants.vcf.gz |
head -20

# Convert VCF to tab-separated for Excel/R import:
$ bcftools query -H -f
'%CHROM\t%POS\t%ID\t%REF\t%ALT\t%QUAL\t%FILTER\t%INFO/AF\n' \
  pass_variants.vcf.gz > variants_table.tsv

✔ Expected Output / What to Look For:

PASS variant count should be a small fraction of raw calls (e.g., 500 PASS from 50,000 raw).

AF distribution: germline SNPs cluster at ~0.5 (het) and ~1.0 (hom); somatic mutations spread widely.

The tsv output can be directly imported into R: variants <- read.table('variants_table.tsv', header=T)

4.4 Reference Genomes

Alignment requires a reference genome — a consensus representation of the human genome sequence to which reads are mapped. The current standard human reference is GRCh38/hg38 (Genome Reference Consortium Human Build 38, released in 2013 and regularly updated through patches). Key features: chromosome sequences (chr1–chr22, chrX, chrY, chrM), unplaced scaffolds, alternate contigs representing population variation, and repeat-masked regions.

T2T-CHM13 (Telomere-to-Telomere): In 2022, the first truly complete human genome assembly was published by the Telomere-to-Telomere consortium, resolving all centromeres, satellite repeats, and previously unsequenced regions. T2T-CHM13 contains ~200 Mb of novel sequence not present in GRCh38. It is beginning to be adopted in research workflows and will likely become the next clinical reference standard.

⚠ COMMON PITFALL

Reference genome version consistency: ALWAYS use the same reference genome version throughout

a project. Mixing GRCh37 and GRCh38 coordinates is a common and serious error.

Chromosome naming conventions also vary: UCSC uses 'chr1', Ensembl uses '1' (without 'chr').

Mismatched chromosome naming causes silent alignment failures and incorrect variant coordinates.

PRACTICAL EXERCISE — Downloading and Indexing the Reference Genome

```
# Download GRCh38 reference from NCBI (use the analysis set — no ALT contigs):

$ wget https://ftp.ncbi.nlm.nih.gov/genomes/all/GCA/000/001/405/
      GCA_000001405.15_GRCh38/seqs_for_alignment_pipelines.ucsc_ids/
      GCA_000001405.15_GRCh38_no_alt_analysis_set.fna.gz

# Decompress:
$ gunzip GCA_000001405.15_GRCh38_no_alt_analysis_set.fna.gz
$ mv GCA_000001405.15_GRCh38_no_alt_analysis_set.fna GRCh38.fa

# Index for BWA-MEM2 (takes ~60 min, ~30 GB RAM, only done once):
$ bwa-mem2 index GRCh38.fa
# Creates: GRCh38.fa.0123  GRCh38.fa.amb  GRCh38.fa.ann
#          GRCh38.fa.bwt.2bit.64  GRCh38.fa.pac

# Create samtools index (for random access by chromosome):
$ samtools faidx GRCh38.fa
# Creates: GRCh38.fa.fai  (chromosome names and lengths)
```

```
# Create sequence dictionary (required by GATK):
$ gatk CreateSequenceDictionary -R GRCh38.fa
# Creates: GRCh38.dict

# Verify: check chromosome naming convention:
$ head -5 GRCh38.fa.fai
```

✔ **Expected Output / What to Look For:**

fai file first column: should show 'chr1', 'chr2', ..., 'chrX', 'chrY', 'chrM' (UCSC naming).

Total sequence length should be ~3.2 billion bp (sum of column 2 in fai).

These index files must be in the same directory as the reference FASTA.

4.5 Major Genomic Databases

The following databases are essential resources for clinical genomics. You will use several of them in this course for downloading reference datasets and annotating variants.

Database	Description and Clinical Use
SRA (Sequence Read Archive)	NCBI repository for raw sequencing data. Download with fasterq-dump (SRA Toolkit).
ENA (European Nucleotide Archive)	EMBL-EBI's raw sequencing data repository; mirrors SRA.
GEO (Gene Expression Omnibus)	NCBI repository for processed gene expression and functional genomic data.
ENCODE	Encyclopedia of DNA Elements: chromatin, transcription factor, and histone modification data.
Ensembl	Genome browser with gene annotation, comparative genomics, and variant data (Homo sapiens and 300+ species).
UCSC Genome Browser	Interactive genome browser with tracks for genes, conservation, regulatory elements, and variants.
ClinVar	NCBI's database of variant-disease relationships with clinical significance classifications.
gnomAD	Population allele frequencies for >800 million variants from >730,000 genomes/exomes. Essential for variant filtering.
dbSNP	NCBI's database of single nucleotide polymorphisms and short indels with rsID identifiers.
COSMIC	Catalogue of Somatic Mutations in Cancer: somatic variant database with tissue-specific mutation frequencies.
OncoKB	Memorial Sloan Kettering's database of oncogenic mutations with therapeutic implications (evidence-based).
TCGA (The Cancer Genome Atlas)	Multi-omics data from 33 cancer types (NCI). Access via GDC portal.
1000 Genomes Project	Whole-genome sequencing data from 2,504 individuals across 26 populations. Gold standard for population genetics.
NA12878 (HG001)	NIST Genome in a Bottle reference sample: the 'gold standard' for NGS pipeline validation.

SRA Toolkit: The fasterq-dump command-line tool (part of SRA Toolkit) enables efficient download of FASTQ files from SRA using accession numbers (SRR/ERR/DRR format for individual runs; SRP/ERP/DRP for projects). Accession numbers are reported in published methods sections and in GEO/ENA records. Metadata (sample information, library strategy, instrument) is retrievable via efetch or the SRA Explorer web tool.

SELF-CHECK EXERCISES

1. A FASTQ read has the quality string 'IIIII'. What Phred quality scores do these characters represent (Phred+33 encoding)? What is the accuracy at each position?

2. A SAM record has FLAG value 99. Convert this to binary and identify what properties this read has.

3. You receive a VCF file annotated with GRCh37 coordinates. Your pipeline uses GRCh38. What tool could you use to convert coordinates, and what are the risks of this conversion?

4. Why is gnomAD critical for somatic variant filtering in tumor samples?

5. What is the difference between ClinVar and COSMIC? Which would you primarily consult for germline variant interpretation? For somatic variant interpretation?

CHAPTER SUMMARY

FASTQ: four-line text format for raw reads. Quality encoded as ASCII (Phred+33). Always paired (R1/R2) for paired-end.

SAM/BAM/CRAM: alignment formats. FLAG encodes read properties; CIGAR encodes alignment operations.

VCF: variant format with CHROM, POS, REF, ALT, QUAL, FILTER, INFO, FORMAT, and sample columns.

GRCh38 is the current clinical standard reference. T2T-CHM13 is the complete reference.

Key databases: SRA/ENA (raw data), ClinVar (germline variants), COSMIC/OncoKB (somatic), gnomAD (population), TCGA (cancer multi-omics).

Chapter 5: Quality Control and Data Cleaning

CLINICAL CASE — Problem-Based Learning

Scenario: The sequencing facility sends you a run report for 24 clinical exome samples. Summary metrics: cluster density 1,450 K/mm², %PF 72%, Q30 score 78%, PhiX error rate 1.8%. The MultiQC report shows that three samples have mean read quality below 25, two samples have >40% adapter content, and one sample shows a strong GC-content bias with a GC peak at 85% (normal range: 38–42% for human exome).

? Which samples are usable? Which require re-sequencing? What are the likely causes of each QC failure? Should you report these samples to the clinical team before completing the analysis?

Learning Objectives: After reading this chapter, you will be able to (1) interpret Illumina run QC metrics (InterOp report); (2) explain every module of a FastQC report; (3) aggregate and compare QC reports using MultiQC; (4) perform adapter trimming and quality-based read filtering; (5) understand the specific QC challenges of Oxford Nanopore data and the tools used to address them.

KEY CONCEPT

Quality control in NGS has two distinct levels:
1. RUN-level QC: Did the sequencer perform correctly? (InterOp metrics)
2. SAMPLE-level QC: Did each individual sample meet quality thresholds? (FastQC/MultiQC)
Rule: Never proceed to alignment until BOTH levels pass.
A run-level failure means ALL samples may be affected — re-sequence all.
A sample-level failure means only that sample needs investigation or re-sequencing.

5.1 Illumina Run Quality Metrics (InterOp)

Before analyzing individual sample data, the run-level quality of an Illumina sequencing run must be assessed using the InterOp metrics generated by the sequencer and accessible through the Illumina Sequencing Analysis Viewer (SAV) or as machine-readable binary files.

5.1.1 Key Run Metrics

Metric	Interpretation and Typical Acceptable Range
Cluster Density (K/mm²)	Number of clusters per mm² of flow cell. Illumina NovaSeq: 1,200–1,400 K/mm². Too high → overcrowding, reduced quality. Too low → insufficient data yield.
%PF (Percent Passing Filter)	Percentage of clusters passing Illumina's chastity filter (signal purity at cycle 1–25). Target: >80%. Low %PF indicates high-density overcrowding or flow cell problems.
Q30 Score (%)	Percentage of bases with Phred quality ≥30. Clinical standard: >75–80%. <70% indicates run-level quality failure; investigate before proceeding with analysis.

Metric	Interpretation and Typical Acceptable Range
PhiX Error Rate (%)	Observed error rate in the PhiX spike-in control (known perfect sequence). Target: <0.5%. High error rate (>1%) indicates sequencing chemistry problems.
%Aligned (PhiX)	Percentage of clusters aligning to PhiX genome. Expected: 0.5–2% (matching spike-in level). Very low alignment may indicate sample sheet errors.
Intensity by Cycle	Signal intensity for each of the four channels across cycles. Should remain stable; rapid decline indicates reagent depletion or focus issues.

5.1.2 PhiX Spike-in

PhiX is a 5,386 bp bacteriophage genome with well-characterized sequence, used as an internal sequencing control. A small fraction (typically 1–5%) of PhiX library is spiked into every Illumina sequencing run. By aligning a subset of clusters to the PhiX reference in real time, the sequencer reports the error rate and alignment percentage as quality indicators. A high PhiX error rate (>1%) indicates sequencing chemistry problems independent of sample quality, helping distinguish machine failures from sample failures.

5.2 FastQC: Per-Sample Read Quality Assessment

FastQC is the standard tool for assessing the quality of FASTQ files at the sample level. It generates an HTML report with multiple modules, each reporting a different quality dimension. Understanding each module is essential for diagnosing library preparation and sequencing problems.

5.2.1 Basic Statistics

Reports: total sequences, sequence length, %GC, number of sequences flagged as poor quality. The %GC should match the expected GC content of the organism (human: ~41% for WGS, 45–50% for WES, depending on the capture kit). Significant deviation indicates contamination or strong GC bias.

5.2.2 Per-Base Sequence Quality

A boxplot showing the distribution of Phred quality scores for each position in the read. Quality typically declines toward the 3' end due to phasing errors and signal degradation. FastQC flags: green/pass if median Q>28 across all positions; orange/warn if Q20–28; red/fail if Q<20 in many positions. Illumina reads from a good run should show Q>30 for at least the first 80–90% of bases, with acceptable decline in the last 10–20% of the read.

5.2.3 Per-Sequence Quality Scores

A distribution of mean quality scores per read. Should show a single sharp peak at high quality (>30). A bimodal distribution (a peak at high quality and a second peak at low quality) indicates a contaminating library or a subset of reads from a failed sequencing cycle. A broad, low-quality distribution indicates a run-level problem.

5.2.4 Per-Base Sequence Content

Shows the percentage of each nucleotide (A, T, G, C) at each position. In random library sequencing, A≈T and G≈C at every position. Illumina reads often show a systematic bias in the first 10–15 bases

due to random hexamer priming in RNA-Seq or adapter-related biases. This is normal and is not removed — it is accommodated by the aligner. Extreme non-random composition at specific positions indicates adapter contamination or a fixed sequence artifact.

5.2.5 Per-Sequence GC Content

A distribution of GC content per read, compared to a theoretical normal distribution. The expected peak corresponds to the organism's average GC content. Deviations from the expected distribution include: a shifted peak (GC bias from enrichment or PCR); a narrow, high peak (over-represented sequences); a secondary peak (contamination with another species or rRNA reads in RNA-Seq). A broad, flat distribution may indicate high sequence diversity (metagenomics sample) or random fragmentation artifacts.

5.2.6 Sequence Duplication Levels

Estimates the percentage of reads that are exact duplicates (by sampling 200,000 reads). High duplication (>20–30%) is expected for: amplicon-based panels (inevitable because all reads come from PCR products); RNA-Seq with very high coverage of highly expressed genes; very low-input libraries with high PCR amplification. High duplication in WGS/WES (>30%) suggests low input DNA, excessive PCR cycles, or very low coverage. PCR duplicates are marked computationally (Picard MarkDuplicates, samtools markdup) and excluded from variant calling.

5.2.7 Overrepresented Sequences

Lists sequences present in more than 0.1% of reads. FastQC compares these to a database of known contaminants (adapter sequences, primer sequences, rRNA). Overrepresented sequences indicate: adapter dimer contamination; primer sequences; rRNA contamination in RNA-Seq; PhiX reads (if >5%); contamination from another library (index hopping or cross-contamination).

5.2.8 Adapter Content

Shows the cumulative percentage of reads containing adapter sequences at each position. The ideal library shows 0% adapter content. Adapter sequences in reads arise when the sequenced DNA insert is shorter than the read length — the sequencer reads through the insert and into the adapter on the other end. This is common with highly fragmented or low-quality samples (short inserts). FastQC flags adapter contamination at >5%. Adapter sequences must be removed computationally before alignment.

IMPORTANT NOTE

FastQC Traffic Light System: Green (PASS), orange (WARN), red (FAIL).

These colors are calibrated for generic Illumina data and do NOT automatically indicate a problem requiring action.

For example: RNA-Seq data legitimately FAILS Per-base sequence content (random hexamer bias);

Amplicon panels legitimately FAIL Duplication (all reads come from amplicons).

Always interpret FastQC results in the biological and library context — do not blindly filter based on colors.

FastQC (Babraham Institute) produces an HTML report with ~10 quality modules. Run FastQC BEFORE any trimming to assess raw data quality.

PRACTICAL EXERCISE — Running FastQC

```
# Run FastQC on paired-end FASTQ files (single sample):
$ fastqc sample_R1.fastq.gz sample_R2.fastq.gz \
    --outdir ./fastqc_results/ \
    --threads 4 \
    --extract

# Run FastQC on all samples in a directory (batch mode):
$ fastqc ./raw_data/*_R1.fastq.gz ./raw_data/*_R2.fastq.gz \
    --outdir ./fastqc_results/ \
    --threads 8

# Options explained:
# --outdir     : output directory for HTML and zip reports
# --threads    : number of files processed in parallel (not per-file speed)
# --extract    : unzip the results archive for direct file access

# View the HTML report:
$ firefox fastqc_results/sample_R1_fastqc.html &

# Check FastQC version:
$ fastqc --version
```

Expected Output / What to Look For:

Output: sample_R1_fastqc.html (visual report) + sample_R1_fastqc.zip (raw data).

Open the HTML in a browser. Navigate each module with the left sidebar.

Green = PASS | Orange = WARN | Red = FAIL — but see interpretation table below.

Interpreting Each FastQC Module

FastQC Module	What It Shows	When FAIL is OK / When It's a Real Problem
Per-base sequence quality	Q-score boxplots per position	FAIL: real problem — reads too short or 3' degradation. Trimming required.
Per-sequence quality scores	Distribution of mean Q per read	Bimodal distribution: two populations (failed reads). Single low peak = run problem.
Per-base sequence content	% A/T/G/C per position	FAIL normal for RNA-Seq (hexamer bias at positions 1-10). Problem: fixed sequence artifact.
Per-sequence GC content	GC distribution vs. expected normal	Secondary peak = contamination (another species). Very narrow peak = low-complexity library.
Per-base N content	% ambiguous bases per position	Any N% > 5% at any position = problem. Common at very end of reads.
Sequence duplication levels	% duplicate reads (sampled)	FAIL is OK for amplicon panels or highly expressed genes (RNA-Seq). Problem for WGS (>50%).

FastQC Module	What It Shows	When FAIL is OK / When It's a Real Problem
Overrepresented sequences	Sequences present in >0.1% of reads	Adapter dimers, rRNA, PhiX contamination. Always investigate these.
Adapter content	% reads containing adapter per position	Any value >5% = trimming required. Indicates insert shorter than read length.

⚠ WARNING

Do NOT treat FastQC FAIL as automatic rejection — context matters.

RNA-Seq data ALWAYS fails Per-base sequence content (hexamer priming bias) — this is normal.

Amplicon panels ALWAYS fail Sequence Duplication (all reads come from PCR amplification) — normal.

The decision to trim or re-sequence must be made based on the biological/library context,

not by blindly following the green/orange/red traffic light system.

5.3 MultiQC: Aggregating QC Reports

When analyzing tens or hundreds of samples simultaneously, reviewing individual FastQC reports is impractical. MultiQC aggregates the results from multiple FastQC reports (and outputs from >100 other bioinformatics tools including STAR, BWA, GATK, Trimmomatic, Cutadapt, Picard, featureCounts) into a single interactive HTML report.

MultiQC automatically detects and parses supported tool outputs in a directory and its subdirectories. Key visualizations include: a general statistics table (one row per sample, key metrics as columns); per-module comparative plots; and interactive heatmaps. MultiQC enables rapid identification of outlier samples — samples with anomalous quality metrics that stand out visually from the cohort — without manual inspection of individual reports.

In a clinical laboratory context, MultiQC reports are typically included as QC documentation in the sequencing run report, providing evidence of run-level and sample-level quality for accreditation purposes (ISO 15189).

MultiQC (Philip Ewels, Babraham Institute) aggregates QC reports from hundreds of samples and dozens of tools into one interactive HTML report.

🔬 PRACTICAL EXERCISE — Running MultiQC

```
# After running FastQC on all samples, run MultiQC from the parent directory:
$ multiqc ./fastqc_results/ \
    --outdir ./multiqc_report/ \
    --filename cohort_QC_report \
    --interactive \
    --verbose

# Run MultiQC across multiple tool outputs simultaneously:
# (FastQC + Trimmomatic + STAR alignment + featureCounts):
$ multiqc \
```

```
    ./fastqc_results/ \
    ./trimming_logs/ \
    ./alignment_logs/ \
    ./quant_results/ \
    --outdir ./multiqc_full/ \
    --filename full_pipeline_QC

# Generate a TSV table of all QC metrics (for automated quality checks):
$ multiqc ./fastqc_results/ --export --outdir ./multiqc_report/
# Creates: multiqc_data/multiqc_general_stats.txt  — one row per sample

# Options:
# --interactive : enables interactive Plotly charts
# --export      : also generates static PDF and TSV files
# --ignore      : exclude specific samples  e.g. --ignore '*undetermined*'
```

✔ **Expected Output / What to Look For:**

Open cohort_QC_report.html in browser.

General Statistics table: scan for outlier samples (anomalous values in any column).

FastQC: Per-sequence quality violin plot — any sample below Q30 median is flagged.

Export TSV: import multiqc_general_stats.txt into R or Excel for automated thresholding.

5.4 Adapter Trimming and Quality-Based Read Filtering

After QC assessment, reads require preprocessing before alignment. The two main operations are: (1) adapter trimming (removal of adapter sequences from read ends) and (2) quality-based trimming (removal of low-quality base calls from read ends or filtering of low-quality reads).

5.4.1 Trimmomatic

Trimmomatic is a widely used read preprocessing tool. It processes paired-end or single-end reads using a series of user-defined steps in sequential order. Key steps include:

- **ILLUMINACLIP: Removes adapter sequences by alignment to a provided adapter FASTA file. Parameters control seed mismatch tolerance, palindrome clip threshold (for adapter dimers), and simple clip threshold.**
- **SLIDINGWINDOW:4:15: Scans the read with a sliding window of size 4 bp. If the average quality within the window falls below 15, the read is clipped at the beginning of the window.**
- **LEADING:3 and TRAILING:3: Removes leading or trailing bases below quality threshold 3.**
- **MINLEN:36: Discards reads shorter than 36 bp after all trimming operations.**

Parameter choice involves a trade-off: aggressive trimming improves alignment quality but reduces total sequence data, potentially reducing coverage at targeted regions. In clinical exome/panel sequencing, conservative trimming (removing only adapter sequences, minimal quality trimming) is generally preferred to preserve data depth.

PRACTICAL EXERCISE — Trimming with Trimmomatic

```
# Download Trimmomatic adapter file if not available:
```

```
# Available at: https://github.com/usadellab/Trimmomatic/tree/main/adapters

# Standard paired-end trimming command:
$ trimmomatic PE \
    -threads 8 \
    -phred33 \
    sample_R1.fastq.gz sample_R2.fastq.gz \
    sample_R1_trimmed.fastq.gz sample_R1_unpaired.fastq.gz \
    sample_R2_trimmed.fastq.gz sample_R2_unpaired.fastq.gz \
    ILLUMINACLIP:TruSeq3-PE-2.fa:2:30:10:8:true \
    LEADING:3 \
    TRAILING:3 \
    SLIDINGWINDOW:4:15 \
    MINLEN:36

# Parameter explanation:
# PE            = paired-end mode
# -phred33      = quality encoding (standard for modern Illumina data)
# ILLUMINACLIP  =
adapter:seed_mismatches:palindrome_clip:simple_clip:min_adapter_length:keepBothReads
#   2           = max 2 mismatches in seed region when looking for adapter
#   30          = score threshold for palindromic adapter pairs
#   10          = score threshold for simple adapter matches
#   8           = minimum adapter length to clip
#   true        = keep both reads even if adapter found in only one
# LEADING:3     = remove leading bases with Q < 3
# TRAILING:3    = remove trailing bases with Q < 3
# SLIDINGWINDOW:4:15 = scan 4-base window; clip if mean Q < 15
# MINLEN:36     = discard reads shorter than 36 bp after trimming

# Compare read count before and after trimming:
$ echo 'Before:' && zcat sample_R1.fastq.gz | wc -l | awk '{print $1/4}'
$ echo 'After:'  && zcat sample_R1_trimmed.fastq.gz | wc -l | awk '{print $1/4}'
```

✔ **Expected Output / What to Look For:**

Log file shows: Input reads, surviving both pairs, surviving R1 only, surviving R2 only, dropped.

Good result: >85% of reads survive trimming with both pairs intact.

<70% survival rate: investigate — may indicate short inserts, adapter dimer overload, or low-quality run.

Run FastQC on trimmed files to confirm adapter content is now 0% and quality improves.

5.4.2 fastp

fastp is a modern, all-in-one FASTQ preprocessing tool that is significantly faster than Trimmomatic (leveraging SIMD acceleration) and generates its own JSON and HTML QC reports. It performs adapter auto-detection (no adapter file required in most cases), quality trimming, per-read quality filtering, base correction for paired-end overlap regions, and optional PolyG/PolyX trimming (especially useful for NextSeq 500/550 data, where high-quality G calls at read ends indicate failed synthesis cycles). fastp is increasingly favored in modern pipelines due to its speed and integrated QC output.

PRACTICAL EXERCISE — Trimming with fastp (Recommended for Speed)

```
# fastp performs QC + trimming in one step, auto-detects adapters:

$ fastp \
    --in1 sample_R1.fastq.gz --in2 sample_R2.fastq.gz \
    --out1 sample_R1_trimmed.fastq.gz --out2 sample_R2_trimmed.fastq.gz \
    --json sample_fastp.json \
    --html sample_fastp.html \
    --thread 8 \
    --qualified_quality_phred 15 \
    --unqualified_percent_limit 40 \
    --length_required 36 \
    --correction \
    --detect_adapter_for_pe

# Key parameters:
# --qualified_quality_phred 15  = base quality cutoff for window trimming
# --unqualified_percent_limit 40 = discard reads if >40% bases below cutoff
# --length_required 36          = discard reads shorter than 36 bp
# --correction                  = correct mismatches in overlapping PE reads
# --detect_adapter_for_pe       = auto-detect adapter sequences (no file needed)

# fastp JSON output can be parsed programmatically:
$ cat sample_fastp.json | python3 -c "import json,sys; d=json.load(sys.stdin); \
  print('Pass rate:', d['filtering_result']['passed_filter_reads'] /
d['summary']['before_filtering']['total_reads'])"
```

✔ Expected Output / What to Look For:

fastp HTML report integrates both QC and trimming results (similar to FastQC+Trimmomatic combined).

fastp is 2-4x faster than Trimmomatic due to SIMD acceleration.

The JSON output is parseable for automated pipeline QC thresholding.

5.4.3 Cutadapt

Cutadapt is a flexible adapter-trimming tool that supports complex adapter configurations including non-Illumina adapters, 5' adapters, linked adapters, and paired-end adapter trimming. It is the trimming backend for popular workflow tools including Trim Galore! (a wrapper combining Cutadapt with FastQC) and is commonly used in RNA-Seq and small RNA-Seq workflows.

5.5 Quality Control of Oxford Nanopore Data

ONT data presents distinct QC challenges compared to Illumina data. The error profile, read length distribution, and data volumes are fundamentally different, requiring specialized QC tools.

5.5.1 NanoPlot

NanoPlot generates quality statistics and visualizations for ONT data from FASTQ, BAM, or sequencing summary files. Key plots include: read length histogram (ONT reads range from hundreds of bases to hundreds of kilobases — the N50 read length is the primary quality metric); quality score

violin plots; read length vs. quality scatter plots; cumulative yield over time. A typical high-quality ONT run (R10.4.1 with Q20+ kit) should show N50 >15 kb and mean read quality >Q20.

5.5.2 NanoStat

NanoStat generates a concise text summary of ONT read statistics: total bases, number of reads, mean/median read length, N50, mean quality, and bases above Q5/Q7/Q10/Q12/Q15 thresholds. NanoStat is useful for rapid run QC before visualization.

PRACTICAL EXERCISE — QC of ONT Data with NanoPlot and NanoStat

```
# Generate statistics summary from a sequencing summary file (preferred) or FASTQ:

$ NanoStat --summary sequencing_summary.txt --outdir nanostat_results/ --name
sample01

# If only FASTQ available:
$ NanoStat --fastq sample.fastq.gz --outdir nanostat_results/ --name sample01

# Generate detailed plots:
$ NanoPlot \
    --summary sequencing_summary.txt \
    --outdir nanoplot_results/ \
    --plots kde hex dot \
    --N50 \
    --loglength \
    --dpi 300

# Adapter trimming for ONT data (if using older kits):
$ porechop \
    --input sample.fastq.gz \
    --output sample_trimmed.fastq.gz \
    --threads 8 \
    --discard_middle  # remove reads with internal adapters (chimeras)

# Quality filtering with NanoFilt (filter by length and quality):
$ gunzip -c sample.fastq.gz | NanoFilt --quality 8 --length 1000 | gzip >
sample_filtered.fastq.gz
# --quality 8 = minimum mean Q-score of 8 (Q8 = ~85% accuracy, minimum for alignment)
# --length 1000 = minimum read length 1,000 bp
```

✔ Expected Output / What to Look For:

NanoStat output: Number of reads, total Gbp, N50, mean read length, mean quality, % reads >Q10/Q15.

Good R10.4.1 run: N50 >15 kb, mean Q >15, total yield matching expected (e.g., >10 Gb for GridION).

NanoPlot generates: read length histogram, quality vs. length scatter plot, yield over time.

5.5.3 Porechop (Adapter Removal for ONT)

ONT libraries may contain adapter sequences from the ligation step (SQK-LSK109 or other kit adapters). Porechop detects and removes ONT-specific adapters from FASTQ reads. Note that for

newer ONT kits and updated Dorado basecalling, adapter sequences are often detected and trimmed during basecalling, making Porechop less necessary in modern workflows.

⚠ COMMON PITFALL

Trimmomatic FAILS on ONT data: Illumina-specific trimming tools (Trimmomatic, Cutadapt in Illumina mode)

are not designed for ONT reads. They may corrupt or discard valid ONT reads.

For ONT data, use: Porechop (adapter removal), NanoFilt (quality and length filtering),

or the filtering options built into Dorado (the current ONT basecaller).

Each sequencing platform requires platform-specific bioinformatics tools.

SELF-CHECK EXERCISES

1. A FastQC report shows Per-base sequence quality PASS, Per-sequence GC content FAIL (bimodal distribution with peaks at 42% and 65%), Overrepresented sequences: rRNA (human 18S). What is the likely problem and how would you address it computationally?

2. A WES run shows 55% of reads are duplicates. Is this acceptable? What parameters would you examine in the library preparation to understand the cause?

3. Describe the sliding window trimming algorithm in Trimmomatic (SLIDINGWINDOW:4:15). Draw a diagram showing how it works on a read with declining quality at the 3' end.

4. What is %PF and how is it calculated? What does a %PF of 68% suggest about a sequencing run?

5. You have 30 WES samples to QC. You run FastQC on all 30 and generate 30 HTML reports. What is the next step, and which tool do you use?

CHAPTER SUMMARY

Run QC (InterOp): %PF >80%, Q30 >75%, PhiX error <0.5%, cluster density within instrument-specific range.

FastQC: 8+ modules covering read quality, GC content, duplication, adapter content, overrepresented sequences.

Interpret FastQC in biological context — FAIL does not always mean action required.

MultiQC aggregates QC results from all samples and 100+ bioinformatics tools into one interactive report.

Trimmomatic, fastp, Cutadapt: adapter removal and quality trimming for Illumina data.

NanoPlot, NanoStat, Porechop: ONT-specific QC and adapter trimming tools.

Never use Illumina trimming tools on ONT data — use platform-specific tools.

Glossary — Chapter 5

InterOp: Illumina's binary run metric files generated during and after sequencing, containing cluster density, %PF, quality scores, and error rates per tile.

%PF: Percent Passing Filter — the fraction of clusters passing Illumina's chastity filter, an internal signal purity metric calculated from the first 25 cycles.

PhiX: A 5,386 bp bacteriophage genome used as an internal sequencing quality control spike. PhiX clusters report the real-time error rate.

FastQC: A quality control tool generating per-sample HTML reports with 10+ quality modules for FASTQ files. Written by Simon Andrews at Babraham Institute.

MultiQC: A tool that aggregates QC reports from 100+ bioinformatics tools into a single interactive HTML report for cohort-level QC review.

Trimmomatic: A flexible Java-based paired-end/single-end read trimming tool with adapter removal and quality-based trimming steps.

fastp: An ultra-fast all-in-one FASTQ preprocessor with integrated QC, adapter auto-detection, and quality filtering.

NanoPlot: ONT data visualization tool generating read length and quality distributions from FASTQ, BAM, or sequencing summary files.

Sliding window trimming: A quality trimming algorithm that scans reads with a window of N bases and clips the read where the mean quality in the window drops below a threshold.

Chapter 6: Variant Calling and Precision Oncology

CLINICAL CASE — Problem-Based Learning

Scenario: You receive FASTQ data from a stage III lung adenocarcinoma biopsy and matched normal blood sample. After alignment and QC, you run GATK Mutect2 in tumor-normal paired mode. The raw VCF contains 28,432 variant calls. After FilterMutectCalls, 1,247 variants PASS all filters. ANNOVAR annotation identifies 3 clinically actionable somatic variants: EGFR L858R (VAF 38%), TP53 R175H (VAF 41%), and KRAS G12C (VAF 22%). Your clinical colleague asks: 'Which of these should drive treatment decisions?'

? How do you validate each variant? What additional information (databases, evidence tiers) do you consult? Why does KRAS G12C matter therapeutically in 2024? How do you communicate uncertainty about the TP53 variant?

Learning Objectives: After reading this chapter, you will be able to (1) explain the principles of read alignment and the BWA-MEM2 algorithm; (2) describe the GATK Best Practices pipeline for germline and somatic variant calling; (3) interpret SAM/BAM metrics from samtools flagstat; (4) explain Base Quality Score Recalibration (BQSR); (5) describe the difference between HaplotypeCaller and Mutect2; (6) annotate and filter VCF files using ANNOVAR and gnomAD; (7) apply clinical variant interpretation frameworks (AMP/ASCO tiers, OncoKB, ClinVar) to somatic variants.

KEY CONCEPT

The GATK Best Practices pipeline has two distinct modes:

- GERMLINE variant calling (HaplotypeCaller): constitutional DNA → inherited SNPs/indels
- SOMATIC variant calling (Mutect2): tumor vs. normal → acquired somatic mutations

Complete variant calling pipeline:

FASTQ → [BWA-MEM2] → BAM → [MarkDuplicates] → [BQSR] → [HaplotypeCaller/Mutect2] → raw VCF → [Filtering] → PASS VCF → [Annotation] → Interpreted Report

6.1 Read Alignment: Principles and Tools

Before variant calling, FASTQ reads must be aligned to the reference genome to determine their genomic origin. Alignment is a computationally intensive process: approximately 150 million reads in a typical 30x WGS run must each be aligned to the ~3.2 billion base pair human genome.

6.1.1 The BWA-MEM2 Algorithm

BWA-MEM2 (Burrows-Wheeler Aligner, Maximal Exact Match, version 2) is the current gold standard short-read aligner for clinical genomics. The algorithm uses three major steps:

11. FM-index based seed finding: The reference genome is indexed using a Burrows-Wheeler Transform (BWT) and a Ferragina-Manzini (FM) index, enabling rapid identification of all

exact matches (seeds) of a read subsequence in the reference. This compressed index (~8 Gb for the human genome) fits in memory for fast lookup.

12. Seed chaining: Short exact matches (seeds) are clustered into candidate alignment regions. The Smith-Waterman algorithm extends seeds into full alignments within these regions.
13. Scoring and mapping quality: For each read, BWA-MEM2 reports the best alignment with a mapping quality score (MAPQ) on a Phred scale (0–60). MAPQ reflects the confidence that the read originates from the aligned position. MAPQ 0 = multimapping (the read aligns equally well to multiple positions); MAPQ 60 = unique, high-confidence alignment. Reads with MAPQ <20 are typically excluded from variant calling to avoid false positives from repetitive region misalignments.

Alternative aligners: Bowtie2 is another popular short-read aligner optimized for very large genomes; STAR (Spliced Transcripts Alignment to a Reference) is the standard aligner for RNA-Seq, handling splice junctions by aligning reads across intron-exon boundaries. DRAGEN (Illumina) and Sentieon offer hardware-accelerated (FPGA/GPU) alignment achieving equivalent accuracy to BWA-MEM2 at dramatically reduced runtimes (hours vs. days for WGS), important in clinical settings with short turnaround time requirements.

PRACTICAL EXERCISE — Alignment with BWA-MEM2

```
# Prerequisite: reference genome indexed (see Chapter 2)

# Full alignment command with read group annotation (REQUIRED for GATK):
$ bwa-mem2 mem \
    -t 16 \
    -R '@RG\tID:sample01\tSM:patient_A\tPL:ILLUMINA\tLB:WES_lib1\tPU:H7YVKBGXF.1'
\
    GRCh38.fa \
    sample_R1_trimmed.fastq.gz \
    sample_R2_trimmed.fastq.gz \
    | samtools sort -@ 4 -o sample_aligned.bam -

# -t 16                = use 16 CPU threads
# -R '@RG\t...'        = add read group header (ESSENTIAL for GATK multi-sample)
#   ID: unique run identifier
#   SM: sample name (matches VCF sample column)
#   PL: sequencing platform
#   LB: library identifier
#   PU: platform unit (flow cell + lane)
# The pipe (|) sends BWA output directly to samtools sort (saves disk I/O)

# Index the sorted BAM:
$ samtools index sample_aligned.bam

# Check alignment statistics immediately:
$ samtools flagstat sample_aligned.bam

# Get insert size distribution (important for WES QC):
$ picard CollectInsertSizeMetrics \
    I=sample_aligned.bam \
    O=insert_size_metrics.txt \
    H=insert_size_histogram.pdf
```

✔ Expected Output / What to Look For:
flagstat: mapped >95%, properly paired >90%, duplicates <30% for WGS/WES.
Insert size: typical WES = 200-400 bp mean. Bimodal distribution = library issue.
CRITICAL: Without the -R read group flag, GATK BQSR and joint genotyping will FAIL.

6.1.2 Post-Alignment Processing

After alignment, several preprocessing steps are required before variant calling:

- **Coordinate sorting: BAM files must be sorted by genomic coordinate (not by read name) for downstream tools. Use samtools sort or Picard SortSam.**
- **Duplicate marking: PCR and optical duplicates are identified and marked using Picard MarkDuplicates or samtools markdup. For WGS/WES, duplicates are marked but not removed — they are excluded from variant calling. For UMI-based panels, Picard UmiAwareMarkDuplicateWithMateCigar performs UMI-aware duplicate marking.**
- **BAM indexing: A .bai index file must be created alongside each sorted BAM (samtools index). This enables random access to any genomic region without reading the entire BAM file.**

samtools flagstat: This command provides a summary of alignment statistics for a BAM file, reporting: total reads, reads passing QC filters, duplicates, mapped reads, paired reads, properly paired reads, reads mapped in a proper pair (both mates mapped, correct orientation and insert size), singletons (mate unmapped), and reads with mate on a different chromosome. Key metrics: mapped read rate should be >95% for good-quality human samples; properly paired rate should be >90%.

PRACTICAL EXERCISE — Marking Duplicates with Picard

```
# Mark PCR and optical duplicates (Picard — bundled with GATK):

$ gatk MarkDuplicates \
    -I sample_aligned.bam \
    -O sample_markdup.bam \
    -M sample_markdup_metrics.txt \
    --REMOVE_DUPLICATES false \
    --OPTICAL_DUPLICATE_PIXEL_DISTANCE 2500

# REMOVE_DUPLICATES false: mark (not remove) — GATK ignores marked duplicates during calling
# OPTICAL_DUPLICATE_PIXEL_DISTANCE 2500: for patterned flow cells (NovaSeq, HiSeq 4000)
#   Use 100 for older non-patterned flow cells (HiSeq 2000, MiSeq)

# Re-index the markdup BAM:
$ samtools index sample_markdup.bam

# Inspect duplication metrics:
$ cat sample_markdup_metrics.txt
# Key metric: PERCENT_DUPLICATION — the fraction of reads marked as duplicates

# For UMI-based panels (liquid biopsy), use UMI-aware duplicate marking:
$ gatk UmiAwareMarkDuplicatesWithMateCigar \
```

```
    -I sample_aligned.bam \
    -O sample_umi_markdup.bam \
    -M sample_umi_metrics.txt \
    --UMI_TAG_NAME RX
# RX = standard SAM tag for UMI sequences (set during alignment or tagging)
```

✔ Expected Output / What to Look For:

PERCENT_DUPLICATION: WGS acceptable <20%; WES acceptable <30%; panels: 50%+ is normal.

High duplication (>50% in WGS) = low input DNA or too many PCR cycles in library prep.

Duplicates are marked with FLAG 0x400 — confirmed by: samtools view -c -f 0x400 sample_markdup.bam

6.2 Variant Calling with GATK Best Practices

The Broad Institute's GATK (Genome Analysis Toolkit) provides the most widely validated and clinically adopted variant calling framework. GATK Best Practices is a modular, evidence-based pipeline for germline and somatic variant discovery.

6.2.1 Base Quality Score Recalibration (BQSR)

Illumina sequencers report per-base quality scores that are systematically biased: quality scores tend to be overestimated for some nucleotide contexts (e.g., specific dinucleotides) and underestimated for others, vary by position in the read, and are affected by the machine cycle. BQSR corrects these systematic biases by: (1) BaseRecalibrator: scanning the aligned BAM for mismatches against the reference at known variant sites (from dbSNP/gnomAD). Under the assumption that mismatches at known SNP sites are not errors, BQSR models the relationship between reported quality, read position, dinucleotide context, and actual error rate. (2) ApplyBQSR: applying the recalibration model to rewrite the base quality scores in the BAM file. BQSR improves variant calling specificity, particularly for low-allele-frequency variant detection in tumor samples.

IMPORTANT NOTE

BQSR requires a known-variant database: the genome being analyzed must not be the same as the individual from whom the known variants were collected (circular reference). For human clinical samples, dbSNP (hg38) and gnomAD serve as the known variant resource.

BQSR is not recommended for RNA-Seq variant calling (different error model applies).

PRACTICAL EXERCISE — GATK BaseRecalibrator and ApplyBQSR

```
# Download known variant resources (only needed once):
$ wget https://storage.googleapis.com/genomics-public-data/resources/broad/hg38/
      v0/Homo_sapiens_assembly38.dbsnp138.vcf
$ wget https://storage.googleapis.com/genomics-public-data/resources/broad/hg38/
      v0/Mills_and_1000G_gold_standard.indels.hg38.vcf.gz
$ wget https://storage.googleapis.com/genomics-public-data/resources/broad/hg38/
      v0/1000G_phase1.snps.high_confidence.hg38.vcf.gz

# Step 1: Build recalibration model (BaseRecalibrator):
```

```
$ gatk BaseRecalibrator \
    -I sample_markdup.bam \
    -R GRCh38.fa \
    --known-sites dbsnp138.vcf \
    --known-sites Mills_1000G.indels.hg38.vcf.gz \
    --known-sites 1000G_phase1.snps.hg38.vcf.gz \
    -O sample_recal.table
# Runtime: ~2-4 hours for 30x WGS on 8 cores

# Step 2: Apply recalibration to BAM:
$ gatk ApplyBQSR \
    -I sample_markdup.bam \
    -R GRCh38.fa \
    --bqsr-recal-file sample_recal.table \
    -O sample_bqsr.bam

# (Optional) Step 3: Evaluate recalibration quality:
$ gatk AnalyzeCovariates \
    -bqsr sample_recal.table \
    -plots sample_BQSR_report.pdf

# Final index:
$ samtools index sample_bqsr.bam
```

✔ **Expected Output / What to Look For:**

recal.table: a text file describing covariates (read group, Q-score, dinucleotide, cycle).

AnalyzeCovariates PDF: shows before/after calibration curves — quality scores should cluster closer to diagonal.

For exome/panel data: restrict BQSR to target intervals with: --intervals target_regions.bed

6.2.2 Germline Variant Calling: HaplotypeCaller

HaplotypeCaller is GATK's method for germline (constitutional) SNV and indel variant calling. Unlike position-by-position callers, HaplotypeCaller uses a haplotype-aware, assembly-based approach:

14. Active region detection: genomic regions with evidence of variation (reads supporting non-reference alleles) are identified.
15. Local de novo assembly: within each active region, all reads are assembled into candidate haplotypes using a De Bruijn graph approach.
16. Pairwise alignment: each read is realigned against each candidate haplotype.
17. Genotype likelihood calculation: using the Phred-scaled genotype likelihoods (PL field in VCF), the most likely genotype (0/0, 0/1, or 1/1) is determined.

GVCF mode and joint genotyping: For cohort analysis, HaplotypeCaller is run in '-ERC GVCF' mode, producing a gVCF for each sample. All sample gVCFs are then combined (CombineGVCFs or GenomicsDBImport) and jointly genotyped (GenotypeGVCFs). Joint genotyping improves accuracy by leveraging population-level information — a site called as homozygous reference in one sample is more convincingly confirmed when 99 other samples in the cohort also show homozygous reference. The gold standard validation dataset for HaplotypeCaller is the NIST Genome in a Bottle NA12878 (HG001) reference sample.

KEY CONCEPT

HaplotypeCaller (GATK): haplotype-aware, assembly-based germline variant caller.

Key innovation: instead of calling variants position-by-position, HaplotypeCaller:

1. Identifies 'active regions' where reads diverge from reference
2. Performs local de novo assembly using a De Bruijn graph
3. Realigns ALL reads against the assembled haplotypes
4. Calls genotypes using Bayesian likelihood: P(genotype | reads)

This assembly step enables accurate calling in complex regions (tandem repeats, micro-satellite regions) where position-by-position callers fail.

PRACTICAL EXERCISE — HaplotypeCaller — Single Sample and GVCF Mode

```
# Single sample (direct VCF output — for quick analysis, not joint genotyping):
$ gatk HaplotypeCaller \
    -R GRCh38.fa \
    -I sample_bqsr.bam \
    -O sample_raw.vcf.gz \
    --dbsnp dbsnp138.vcf \
    -L exome_targets.bed \
    --native-pair-hmm-threads 4

# GVCF mode (recommended for cohort analysis):
$ gatk HaplotypeCaller \
    -R GRCh38.fa \
    -I sample_bqsr.bam \
    -O sample.g.vcf.gz \
    -ERC GVCF \
    --dbsnp dbsnp138.vcf \
    -L exome_targets.bed \
    --native-pair-hmm-threads 4
# -ERC GVCF: output gVCF with reference blocks for ALL positions

# Joint genotyping across multiple samples (run after all GVCFs generated):
$ gatk GenomicsDBImport \
    -V sample1.g.vcf.gz -V sample2.g.vcf.gz -V sample3.g.vcf.gz \
    --genomicsdb-workspace-path genomicsdb_workspace/ \
    -L exome_targets.bed

$ gatk GenotypeGVCFs \
    -R GRCh38.fa \
    -V gendb://genomicsdb_workspace/ \
    -O cohort_raw.vcf.gz \
    --dbsnp dbsnp138.vcf

# Filter germline variants (VQSR for large cohorts, hard filter for small N):
# Hard filter for SNPs (small cohort <30 samples):
$ gatk VariantFiltration \
    -V cohort_raw.vcf.gz \
```

```
    --filter-expression 'QD < 2.0'     --filter-name 'QD2' \
    --filter-expression 'FS > 60.0'    --filter-name 'FS60' \
    --filter-expression 'MQ < 40.0'    --filter-name 'MQ40' \
    --filter-expression 'MQRankSum < -12.5' --filter-name 'MQRankSum' \
    --filter-expression 'ReadPosRankSum < -8.0' --filter-name 'ReadPos' \
    -O cohort_hardfiltered.vcf.gz
```

✔ **Expected Output / What to Look For:**

Raw HaplotypeCaller output: ~5-6 million variants for WGS, ~80,000 for WES.

After filtering: ~4-5 million SNPs + ~500,000 indels for WGS (mostly common variants).

PASS variants after VQSR: >99.9% sensitivity at Ti/Tv ratio ~2.0-2.1 (expected for human genome).

Check: bcftools stats cohort_hardfiltered.vcf.gz | grep 'Ts/Tv'

6.2.3 Somatic Variant Calling: Mutect2

Mutect2 is GATK's somatic variant caller, designed for detecting low-allele-frequency somatic mutations in tumor samples against a matched (or unmatched) normal background.

Tumor-normal paired calling: When a matched normal (germline blood or saliva) sample is available, Mutect2 compares the tumor and normal simultaneously. The normal suppresses germline variants (SNPs present in both tumor and normal) and normal sequencing artifacts, enabling detection of tumor-specific somatic mutations at low VAF. The Panel of Normals (PoN) is a collection of normal samples sequenced with the same platform and protocol; common artifactual variants seen in ≥2 normals in the PoN are filtered from tumor calls, reducing false positives from systematic sequencing errors.

Tumor-only calling: When no matched normal is available, Mutect2 uses gnomAD population allele frequencies to distinguish likely somatic variants (low gnomAD frequency) from germline polymorphisms (high gnomAD frequency). Tumor-only calling has lower specificity than paired calling and requires more aggressive filtering.

FilterMutectCalls: After variant calling, Mutect2 raw VCF outputs require filtering with FilterMutectCalls. This applies: contamination estimates (estimated from GetPileupSummaries + CalculateContamination); orientation bias filtering (OXOG artifacts from oxidative DNA damage in FFPE); strand bias filters; read position filters; and PoN-based filters. Only variants with FILTER='PASS' should be used in downstream annotation.

PRACTICAL EXERCISE — Mutect2 — Tumor-Normal Paired Mode

```
# Build Panel of Normals (PoN) from healthy samples (use 40+ normals for best results):

# Step 1: Run Mutect2 in tumor-only mode on each normal sample:
$ for normal in normal1.bam normal2.bam normal3.bam; do
    gatk Mutect2 -R GRCh38.fa -I $normal -O ${normal%.bam}.vcf.gz --max-mnp-distance 0
  done

# Step 2: Create the PoN:
$ gatk CreateSomaticPanelOfNormals \
    -V normal1.vcf.gz -V normal2.vcf.gz -V normal3.vcf.gz \
    --germline-resource gnomad.exomes.vcf.gz \
```

```
    -O panel_of_normals.vcf.gz

# Step 3: Tumor-normal paired somatic calling:
$ gatk Mutect2 \
    -R GRCh38.fa \
    -I tumor_bqsr.bam \
    -I normal_bqsr.bam \
    -normal normal_sample_name \
    --germline-resource gnomad.exomes.vcf.gz \
    --panel-of-normals panel_of_normals.vcf.gz \
    -L exome_targets.bed \
    -O tumor_raw.vcf.gz \
    -bamout tumor_realigned.bam
# -normal: must match exactly the SM field in the normal BAM read group (@RG SM:)
# -bamout: outputs the locally realigned BAM for IGV visualization

# Step 4: Estimate contamination:
$ gatk GetPileupSummaries -I tumor_bqsr.bam -V small_exac_common.vcf.gz \
    -L small_exac_common.vcf.gz -O tumor_pileups.table
$ gatk CalculateContamination -I tumor_pileups.table \
    -matched normal_pileups.table -O contamination.table

# Step 5: Filter somatic variants:
$ gatk FilterMutectCalls \
    -R GRCh38.fa \
    -V tumor_raw.vcf.gz \
    --contamination-table contamination.table \
    --orientation-bias-artifact-priors artifact_priors.tar.gz \
    -O tumor_filtered.vcf.gz

# Step 6: Extract PASS variants only:
$ bcftools view -f PASS tumor_filtered.vcf.gz -o tumor_pass.vcf.gz -O z
$ bcftools index tumor_pass.vcf.gz
$ echo 'PASS variants:' && bcftools view -H tumor_pass.vcf.gz | wc -l
```

✔ **Expected Output / What to Look For:**

Raw Mutect2 output: ~20,000-100,000 raw calls. After FilterMutectCalls PASS: ~500-2,000.

Contamination table: contamination fraction should be <5% for reliable calling.

Verify: view tumor_pass.vcf.gz at known oncogenic hotspots in IGV (EGFR, KRAS, TP53).

6.3 VCF Annotation and Clinical Variant Interpretation

6.3.1 Annotation Tools

Raw variant calls lack biological and clinical context. Annotation tools add information about gene context, predicted functional impact, population frequencies, and clinical significance.

- **ANNOVAR: A widely used Perl-based annotation tool supporting multiple databases (RefSeq, ENSEMBL, gnomAD, dbSNP, ClinVar, COSMIC, SIFT, PolyPhen-2, CADD). Produces annotated VCF or tab-delimited output.**

- **SnpEff: Open-source annotation tool predicting the functional effect of variants on transcripts and proteins (synonymous, missense, nonsense, frameshift, splice site). Generates human-readable HTML summary reports.**
- **VEP (Ensembl Variant Effect Predictor): The most comprehensive annotation tool, maintained by EMBL-EBI. Supports GRCh38 and GRCh37, annotates against Ensembl transcripts, and integrates gnomAD, ClinVar, COSMIC, and in silico prediction scores.**

PRACTICAL EXERCISE — Annotating Variants with ANNOVAR

```
# Download ANNOVAR and databases (register at annovar.openbioinformatics.org):

$ perl annotate_variation.pl -buildver hg38 -downdb -webfrom annovar refGene humandb/
$ perl annotate_variation.pl -buildver hg38 -downdb -webfrom annovar gnomad40_exome
humandb/
$ perl annotate_variation.pl -buildver hg38 -downdb -webfrom annovar clinvar_20231217
humandb/
$ perl annotate_variation.pl -buildver hg38 -downdb -webfrom annovar cosmic97_coding
humandb/
$ perl annotate_variation.pl -buildver hg38 -downdb -webfrom annovar dbnsfp42c
humandb/

# Convert VCF to ANNOVAR input format:
$ convert2annovar.pl -format vcf4old tumor_pass.vcf.gz -outfile tumor_annovar.avinput \
    -withzyg -includeinfo -allsample

# Run ANNOVAR annotation (table_annovar.pl — one-stop annotation):
$ table_annovar.pl tumor_annovar.avinput humandb/ \
    -buildver hg38 \
    -out tumor_annotated \
    -remove \
    -protocol refGene,gnomad40_exome,clinvar_20231217,cosmic97_coding,dbnsfp42c \
    -operation g,f,f,f,f \
    -nastring . \
    -vcfinput \
    -thread 4

# Operation codes: g = gene-based | f = filter-based | r = region-based

# Quick filter: extract nonsynonymous and stop-gain variants with gnomAD AF < 0.001:
$ awk 'NR==1 || ($6=="nonsynonymous SNV" || $6=="stopgain")'
tumor_annotated.hg38_multianno.txt \
    | awk 'NR==1 || $10 < 0.001 || $10=="."' > tumor_filtered_annotated.txt

# Import into R for further filtering:
# variants <- read.table('tumor_filtered_annotated.txt', header=T, sep='\t', quote='')
# clinical_vars <- subset(variants, ClinVar_SIG %in% c('Pathogenic','Likely_pathogenic'))
```

Expected Output / What to Look For:

Output file: tumor_annotated.hg38_multianno.txt — tab-separated table, one variant per row.

Key columns: Chr, Start, Ref, Alt, Func.refGene, Gene.refGene, ExonicFunc.refGene,

gnomAD_AF, ClinVar_CLNSIG, cosmic97_coding, SIFT_score, PolyPhen2_score.

Filter chain: PASS → nonsynonymous/stop-gain/splice → gnomAD AF <0.001 → ClinVar check → COSMIC.

6.3.2 Critical Annotation Databases

gnomAD (Genome Aggregation Database) v4.1 contains allele frequencies from >730,000 exomes and genomes. For somatic variant calling, any variant present in gnomAD at allele frequency >1% is almost certainly a germline polymorphism, not a somatic mutation, and should be excluded. For clinical germline variant interpretation, gnomAD frequencies inform the 'BA1' benign criterion of ACMG classification (allele frequency >5% in population databases argues for benign classification).

ClinVar aggregates variant-disease associations with clinical significance classifications (Pathogenic/Likely Pathogenic/VUS/Likely Benign/Benign) submitted by clinical laboratories worldwide. ClinVar is the primary resource for germline variant interpretation. For somatic interpretation, COSMIC (Catalogue of Somatic Mutations in Cancer) reports the cancer type, tissue origin, and frequency of each somatic variant across thousands of tumor samples.

OncoKB (Memorial Sloan Kettering) provides evidence-based therapeutic implications for somatic mutations: Level 1 (FDA-approved targeted therapy for this cancer type), Level 2 (standard care in cancer type per clinical guidelines), Level 3A/B (compelling clinical evidence or clinical trials), Level 4 (biological evidence). OncoKB is the primary resource for clinical oncology variant interpretation in precision oncology.

6.3.3 Somatic Variant Interpretation: AMP/ASCO/CAP Tiers

The joint AMP/ASCO/CAP 2017 guidelines classify somatic variants into four tiers based on clinical significance:

Tier	Definition and Examples
Tier I — Strong Clinical Significance	Variants with strong evidence: FDA-approved or professional guideline-listed biomarkers. E.g., EGFR L858R (NSCLC, erlotinib); BRAF V600E (melanoma, vemurafenib); BRCA1/2 LOF (breast/ovarian, PARP inhibitor).
Tier II — Potential Clinical Significance	Variants with published clinical evidence not yet in standard guidelines, or approved in other cancer types. E.g., KRAS G12C in colorectal cancer if sotorasib not yet guideline-listed.
Tier III — Unknown Clinical Significance	Variants with limited/conflicting evidence. VUS status. Functional prediction tools may suggest impact but clinical evidence is insufficient.
Tier IV — Benign/Likely Benign	Variants with strong evidence of no clinical significance: common germline polymorphisms (gnomAD AF >1%), synonymous variants with no splice effect.

KEY CONCEPT

Key resources for somatic interpretation:

- OncoKB (oncokb.org) — evidence-based therapeutic implications (Levels 1-4)
- COSMIC (cancer.sanger.ac.uk/cosmic) — somatic mutation frequencies in cancer types
- ClinVar — primarily germline but includes some somatic entries
- My Cancer Genome (mycancergenome.org) — curated clinical annotations

6.4 Visualization with IGV

The Integrative Genomics Viewer (IGV) is an interactive visualization tool for genomic data, essential for manual review of variant calls. IGV allows inspection of reads at any genomic locus: base-level mismatches are color-coded; soft-clipped reads are shown; paired-end insert sizes are visualized; and multiple tracks (BAM, VCF, BED, BigWig) can be displayed simultaneously. Before reporting any clinically actionable variant, visual inspection in IGV is mandatory to verify: (1) the variant is present in the BAM at the expected position and VAF; (2) the reads supporting the variant are of high quality (no systematic strand bias, no soft-clipping at the variant position, no cluster at read ends); (3) the region is not in a complex repetitive region that causes systematic alignment artifacts.

PRACTICAL EXERCISE — Manual IGV Validation of Somatic Variants

```
# Before reporting ANY clinically significant variant, validate in IGV.

# Open IGV (download from igv.org or use web version at igv.org/app):
# File → Load from File → tumor_bqsr.bam AND normal_bqsr.bam
# File → Load from File → tumor_filtered.vcf.gz

# Navigate to variant (e.g., EGFR L858R on GRCh38):
# Type in navigation bar: chr7:55,259,515

# IGV checklist for variant validation:
# 1. Count supporting reads: are there at least 10 ALT reads?
# 2. Check strand balance: are ALT reads on BOTH strands (not just + or -)?
# 3. Check read position: are ALT reads distributed across read position (not clustered at ends)?
# 4. Check base quality: are supporting reads high quality (not grey/washed-out)?
# 5. Normal sample: does the normal show 0 ALT reads at the same position?
# 6. Alignment context: is this in a repetitive region? (grey highlighted bases = low MAPQ)

# Automate IGV batch sessions for many variants (IGV batch mode):
# Create a batch script (igv_commands.txt):
# genome GRCh38
# load tumor_bqsr.bam
# load normal_bqsr.bam
# goto chr7:55259515
# snapshot EGFR_L858R.png
$ igv --batch igv_commands.txt --igvDirectory ./igv_home/
```

Expected Output / What to Look For:

PASS variant in IGV should show: balanced strand support, reads across full length, no soft-clip artifact.

Artifact patterns to reject: all reads on one strand (strand bias artifact); only at read ends (end artifact);

reads all below Q20 (low quality artifact); same as normal sample (germline, not somatic).

6.5 Liquid Biopsy: Ultra-Deep Sequencing and UMI Correction

Liquid biopsy presents the greatest analytical challenge in clinical genomics: ctDNA may be present at VAF below 1% in patient plasma, requiring detection of 1 variant-supporting read in 100 (or 1 in

10,000) reads against a background of sequencing errors. Two complementary strategies address this challenge:

- **Ultra-deep sequencing: Targeted panels at 10,000x–100,000x coverage ensure statistical power to detect rare variants. At 10,000x coverage, a variant at 0.1% VAF would be supported by ~10 reads — barely above the statistical noise floor.**
- **Unique Molecular Identifiers (UMIs): Short random DNA barcodes (6–12 bp) are incorporated into adapter sequences at the ligation step, tagging each individual library molecule with a unique code. After PCR amplification, all reads sharing the same UMI originated from the same original molecule. UMI-aware deduplication (using tools such as fgbio or Picard UmiAwareMarkDuplicateWithMateCigar) generates consensus reads from all reads with the same UMI, dramatically reducing PCR-introduced sequencing errors. This enables confident detection of variants at VAF as low as 0.01%.**

SELF-CHECK EXERCISES

1. A BAM file shows 88% properly paired reads and 12% singletons. Is this concerning? What might cause a high singleton rate?

2. Explain why BQSR uses known variant sites from dbSNP rather than simply comparing to the reference at all positions.

3. HaplotypeCaller is called on tumor DNA without a matched normal. What category of variants will it detect that Mutect2 would not? What category might it miss?

4. A somatic VCF contains a variant: KRAS G12V, VAF 5%, gnomAD AF 0.000008, COSMIC count 4,521, OncoKB Level 1 in pancreatic cancer. The patient has colorectal cancer. How would you classify this variant and communicate it to the clinician?

5. Why is visual inspection of variants in IGV mandatory before reporting, even when a bioinformatics pipeline has already applied all standard filters?

6. A tumor-only Mutect2 analysis (no matched normal) finds a TP53 R175H variant at VAF 3%. gnomAD AF = 0.000002. Explain TWO reasons why tumor-only calling is less reliable than tumor-normal paired calling, and how you would attempt to validate this specific variant.

7. BQSR requires 'known variant sites'. Why can you NOT use the variants you just called from your sample as the known sites for BQSR of the same sample?

8. A somatic variant has: COSMIC count 8,421 in lung adenocarcinoma; gnomAD AF 0.000001; OncoKB Level 2; VAF 22%. Classify this variant using AMP/ASCO/CAP tiers and recommend whether to report it in a clinical oncology setting.

9. Explain the difference between VQSR and hard filtering for germline variant QC. When is hard filtering preferred over VQSR?

CHAPTER SUMMARY

BWA-MEM2: FM-index and local assembly-based aligner. MAPQ reflects alignment confidence; MAPQ <20 reads excluded from variant calling.

Post-alignment processing: sort → mark duplicates → index → (optional) BQSR.

BQSR corrects systematic biases in base quality scores by modeling context-specific error rates.

HaplotypeCaller: haplotype-aware assembly for germline variants. GVCF mode enables joint genotyping.

Mutect2: somatic variant calling. Tumor-normal paired calling is more specific than tumor-only. FilterMutectCalls applies contamination, strand bias, and PoN filters. Annotation: ANNOVAR/VEP + gnomAD (population AF) + ClinVar (germline) + COSMIC/OncoKB (somatic).
AMP/ASCO/CAP Tier I–IV classification guides clinical reporting of somatic variants. Liquid biopsy: ultra-deep sequencing + UMI correction enables VAF detection <0.1%.

INFORMATION
DRAGEN and Sentieon: Hardware-accelerated alternatives to GATK.
DRAGEN (Illumina, FPGA-based): processes 30x WGS in <1 hour. Equivalent accuracy to GATK.
Sentieon (GPU-based): exact mathematical replication of GATK algorithms, 5-50x faster.
Both are used in clinical labs (DKFZ, BIH, major cancer centers) for short turnaround time.
Regulatory consideration: DRAGEN is FDA 510(k) cleared for some indications — important for IVDR compliance.

Glossary — Chapter 6

BWA-MEM2: Burrows-Wheeler Aligner with Maximal Exact Match seeds, version 2. The standard short-read aligner for clinical genomics using BWT/FM-index for rapid seed finding.

MAPQ: Mapping Quality score — Phred-scaled probability that the read is misaligned. MAPQ 60 = highest confidence; MAPQ 0 = multimapping.

BQSR: Base Quality Score Recalibration — a GATK step that corrects systematic biases in reported Phred quality scores using known variant sites as calibration anchors.

HaplotypeCaller: GATK's haplotype-aware, local assembly-based germline SNV and indel variant caller.

Mutect2: GATK's somatic variant caller using a Bayesian model to distinguish tumor-specific mutations from germline polymorphisms and technical artifacts.

Panel of Normals (PoN): A collection of germline normal samples sequenced with the same protocol, used by Mutect2 to filter common artifactual variants.

ANNOVAR: Annotate Variation — a Perl tool for functional annotation of genetic variants from VCF files against multiple databases.

VEP: Ensembl Variant Effect Predictor — a comprehensive variant annotation tool integrating gene models, population frequencies, and clinical databases.

OncoKB: A precision oncology knowledge base (Memorial Sloan Kettering) providing evidence-based therapeutic implications for somatic mutations (Levels 1–4).

IGV: Integrative Genomics Viewer — interactive desktop/web tool for visualization of alignments, variants, and annotation tracks at any genomic locus.

CIGAR string: Compact Idiosyncratic Gapped Alignment Report — encodes the per-base alignment operations (M: match, I: insertion, D: deletion, S: soft-clip, N: intron skip) in SAM/BAM format.

Chapter 7: Single-Cell RNA Sequencing — Resolving Cellular Heterogeneity

CLINICAL CASE — Problem-Based Learning

Scenario: A research team studying chemotherapy resistance in breast cancer analyzes a tumor sample by bulk RNA-Seq and finds overall upregulation of ABC transporter genes. They conclude the tumor is drug resistant. A collaborator suggests single-cell RNA-Seq (scRNA-Seq) instead. The scRNA-Seq data reveals 11 distinct cell clusters. Only one cluster — representing 3.2% of total cells — shows extreme ABC transporter upregulation. The remaining tumor cells are actually sensitive to therapy.

? Why did bulk RNA-Seq mislead the analysis? What does this tell you about the limitations of averaging? How would you identify the resistant subpopulation and characterize it further? What clinical implications does this finding have?

Learning Objectives: After reading this chapter, you will be able to (1) explain why scRNA-Seq provides information inaccessible to bulk RNA-Seq; (2) describe the 10x Genomics Chromium droplet-based workflow; (3) interpret and perform the Seurat analysis pipeline; (4) explain UMAP and Louvain clustering; (5) perform differential expression analysis and pathway enrichment; (6) describe trajectory analysis (pseudotime); (7) critically evaluate AI tools for automated cell type annotation.

7.1 Why Single-Cell RNA Sequencing?

Bulk RNA-Seq measures the average gene expression across thousands to millions of cells. This averaging masks cell-to-cell variability — the heterogeneity that underlies development, disease, and drug resistance. A bulk RNA-Seq experiment on a tumor biopsy cannot distinguish the expression profiles of tumor cells, immune cells, stromal fibroblasts, endothelial cells, and rare stem-like cells present within the same biopsy. This compositional and transcriptional heterogeneity is precisely what drives cancer progression, immunotherapy response, and therapeutic resistance.

Single-cell RNA sequencing resolves this heterogeneity by measuring the transcriptome of each individual cell. Instead of one data point per gene per sample, scRNA-Seq generates one data point per gene per cell per sample — typically 1,000 to 50,000 cells per experiment, each profiled for 20,000+ genes. This creates a high-dimensional data matrix (cells × genes) that is computationally analyzed to identify cell types, cell states, developmental trajectories, and rare populations.

DID YOU KNOW?

The Human Cell Atlas project aims to create a comprehensive reference map of all human cell types — estimated at ~37 trillion cells spanning ~400 distinct types. Single-cell RNA-Seq is the primary technology driving this atlas. By 2023, over 50 million cells from 33 organs had been profiled.

Question Type	Bulk RNA-Seq	Single-Cell RNA-Seq
Cell types present	Cannot identify	Identifies all types + proportions
Rare populations (<1%)	Signal drowned by majority	Detects if ≥10-20 cells present
Cell-type-specific DE	Cannot assign to cell type	DE within specific clusters
Cell heterogeneity	Hidden by averaging	Full resolution of all states
Developmental trajectories	Requires time series	Inferred from single snapshot

Question Type	Bulk RNA-Seq	Single-Cell RNA-Seq
Cost per sample	~300–800 EUR (library + sequencing)	~1,500–4,000 EUR (library + sequencing)
Reads required	20–50M reads	500M–2B reads (50,000 cells × ~20k reads/cell)
Data complexity	Low — one vector per gene	Very high — full matrix analysis required

INFORMATION

The Human Cell Atlas (HCA): an international project aiming to map every cell type

in the human body at single-cell resolution. By 2024, >100 million cells profiled

across 40+ organs. scRNA-Seq is the primary technology driving this atlas.

Clinical impact: HCA reference atlases are used to identify disease-specific cell states

in cancer, autoimmune disease, COVID-19 severity, and aging.

Website: www.humancellatlas.org

7.2 Library Preparation for scRNA-Seq: 10x Genomics Chromium

The 10x Genomics Chromium system is the dominant commercial scRNA-Seq platform, used in >80% of published scRNA-Seq studies. It uses microfluidic droplet encapsulation to capture and barcode individual cells.

Workflow: A single-cell suspension (cells must be dissociated from tissue — a critical wet-lab step) is loaded onto the 10x Chromium Controller. The controller co-encapsulates individual cells with gel beads (containing barcoded reverse transcription primers) and reverse transcriptase into oil droplets (GEMs — Gel Beads in EMulsion). Each bead carries ~750,000 copies of a primer containing: a 10x cell barcode (16 bp, unique to each bead/droplet), a UMI (12 bp, random, unique to each cDNA molecule), a poly-dT sequence (for mRNA capture), and Illumina sequencing primer sequences. Within each GEM: cell lysis releases mRNA; the poly-dT primer hybridizes to poly-A mRNA tails; reverse transcription produces cDNA tagged with the cell barcode and UMI.

After breaking the emulsion, cDNA from all cells is pooled, amplified, and processed into a standard Illumina library. The resulting FASTQ files are processed by Cell Ranger (10x Genomics' software pipeline), which performs: FASTQ-to-alignment (STAR aligner), barcode and UMI counting, and generation of a feature-barcode matrix (the primary output: a sparse matrix of UMI counts per gene per cell barcode).

IMPORTANT NOTE

Multiplets (doublets): When two cells are captured in the same GEM, they share a cell barcode

and appear as one artificial 'super-cell' with a mixture of two transcriptomes.

Doublets inflate apparent complexity and can create spurious intermediate clusters.

Doublet detection tools (DoubletFinder, Scrublet) are routinely applied before downstream analysis.

WARNING

CRITICAL WET-LAB FAILURE MODES that cannot be rescued bioinformatically:

1. Cell clumping: clumps generate multiplets (doublets) — inflate false cluster complexity.

Prevention: filter through 40 µm strainer, use anti-clumping buffer.

2. Low viability (<85%): dead cells release ambient RNA → contaminates all droplets.

Prevention: process tissue immediately; use FACS viability gating before loading.

3. Incorrect cell concentration: too low = wasted run; too many = high doublet rate.

10x guideline: 700–1,200 cells/µL. Check with hemocytometer AND automated counter.

4. Tissue-specific dissociation artifacts: harsh dissociation upregulates stress genes

(FOS, JUN, HSPA1A) in ALL cells — a technical signal, not biology.

Prevention: cold dissociation protocols, minimize dissociation time (<45 min).

7.3 Raw Data Processing — Cell Ranger

Cell Ranger (10x Genomics, free software) is the official pipeline that processes raw FASTQ files into the count matrix used for downstream analysis. It wraps STAR for alignment and implements 10x-specific barcode and UMI processing.

KEY CONCEPT

Cell Ranger count output — the three files you will use in Seurat:

```
matrix.mtx.gz  → sparse count matrix in Matrix Market format
                 (rows = features/genes; columns = cell barcodes)
barcodes.tsv.gz → list of valid cell barcodes (one per column of matrix)
features.tsv.gz → gene IDs and names (one per row of matrix)
```

These three files form a 'triangle' — they must be kept together in one directory.
Read into R with: Seurat::Read10X(data.dir = 'filtered_feature_bc_matrix/')

Cell Ranger outputs TWO versions of the matrix:
filtered_feature_bc_matrix/ → only barcodes called as cells (use this for analysis)
raw_feature_bc_matrix/ → all detected barcodes including empty droplets

PRACTICAL EXERCISE — Running Cell Ranger count

```
# Prerequisites: Cell Ranger installed, reference genome downloaded

# Download Cell Ranger from: support.10xgenomics.com/single-cell-gene-expression

# Download pre-built GRCh38 reference (10x-optimized with GENCODE annotation):
$ wget https://cf.10xgenomics.com/supp/cell-exp/refdata-gex-GRCh38-2024-A.tar.gz
$ tar -xf refdata-gex-GRCh38-2024-A.tar.gz

# Run Cell Ranger count (main command):
$ cellranger count \
    --id=sample01_cellranger \
    --transcriptome=refdata-gex-GRCh38-2024-A/ \
    --fastqs=/data/raw_fastqs/sample01/ \
    --sample=sample01 \
    --localcores=16 \
    --localmem=64 \
```

```
    --expect-cells=8000

# Parameter explanation:
# --id            : output directory name (created by cellranger)
# --transcriptome  : path to 10x reference genome + annotation
# --fastqs         : directory containing the FASTQ files
# --sample         : sample name (prefix of FASTQ files)
# --localcores 16  : number of CPU cores
# --localmem 64    : memory in GB (recommend 64 GB minimum for GRCh38)
# --expect-cells   : expected cell count (helps Cell Ranger knee-point detection)

# Check run status (Cell Ranger shows a live progress page):
# Open in browser: http://localhost:PORT (shown in terminal output)

# When complete, check the HTML summary:
$ firefox sample01_cellranger/outs/web_summary.html &

# Navigate to output directory:
$ ls sample01_cellranger/outs/
# Key files:
# web_summary.html          → interactive QC report (OPEN THIS FIRST)
# metrics_summary.csv       → machine-readable QC metrics
# filtered_feature_bc_matrix/ → count matrix (use for Seurat)
# raw_feature_bc_matrix/    → all barcodes including empty droplets
# possorted_genome_bam.bam  → aligned reads
# molecule_info.h5          → UMI-level information (for multiplet detection)
```

✔ **What to Look For / Expected Output:**

web_summary.html: check all metrics against the expected ranges (see table below).

metrics_summary.csv: parse with Python/R for automated batch QC.

Typical runtime: 4–8 hours for 5,000 cells on 16 cores with 64 GB RAM.

7.3.1 Cell Ranger QC Metrics — What to Check

Metric	Acceptable Range and Interpretation
Estimated Number of Cells	Within ±20% of expected. Too low = low viability or low input. Too high = doublet rate concern.
Mean Reads per Cell	>20,000 ideally. <10,000 = under-sequenced; increase sequencing depth.
Median Genes per Cell	WGS 500–3,000 (tissue-dependent). >6,000 may indicate doublets.
Valid Barcodes (%)	>75%. Low % = degraded library or incorrect sequencing configuration.
Sequencing Saturation (%)	>60% indicates sufficient sequencing depth. <40% = more sequencing needed.
Reads Mapped to Genome (%)	>70%. Low % = contamination (mycoplasma?) or reference mismatch.
Reads Mapped to Transcriptome (%)	50–80%. Lower in nuclei (snRNA-Seq) due to intronic reads.
Q30 Bases in Barcode (%)	>85%. Low = sequencing quality issue specifically at Read 1.

Metric	Acceptable Range and Interpretation
Q30 Bases in UMI (%)	>85%. Critical for accurate deduplication.

INFORMATION

Cell Ranger alternative: STARsolo (integrated into STAR aligner) produces equivalent count matrices and is faster for large datasets. It is free, open-source, and does not require 10x registration.

STARsolo key parameters for 10x v3 data:

--soloType CB_UMI_Simple

--soloCBwhitelist 3M-february-2018.txt (10x v3 barcode whitelist)

--soloCBstart 1 --soloCBlen 16 --soloUMIstart 17 --soloUMIlen 12

For ONT-based long-read single-cell RNA-Seq: use oarfish or bambu for UMI-aware isoform counting — enabling full-length transcript quantification per cell.

7.3 The Seurat Analysis Pipeline

Seurat (R package, Satija Lab) is the most widely used scRNA-Seq analysis toolkit. The standard Seurat workflow transforms raw count matrices into biologically interpretable cell clusters through the following steps:

7.3.1 Quality Control and Filtering

The feature-barcode matrix contains all detected cell barcodes, including empty GEMs (ambient RNA noise), dead or damaged cells, and doublets. Initial QC filters are applied on three per-cell metrics:

- **nFeature_RNA: number of distinct genes detected per cell. Typical range: 200–6,000. Very low values (<200): empty GEMs or dead cells. Very high values (>6,000): doublets.**
- **nCount_RNA: total UMI count per cell. Typically 1,000–50,000. Highly correlated with nFeature_RNA.**
- **percent.mt: percentage of reads mapping to mitochondrial genes (genes on chrMT). Normal cells: <15–25%. Damaged or dying cells release cytoplasmic mRNA (reducing total UMI count) while retaining intact mitochondria (elevating %mito). High %mito is a marker of low cell quality.**

Standard QC thresholds (must be dataset-specific): nFeature_RNA > 200 & < 6,000; percent.mt < 20. These thresholds are data-dependent and should be set by inspecting violin plots and scatter plots of QC metrics, not applied blindly.

PRACTICAL EXERCISE — Loading the Count Matrix into Seurat

```
# Install required packages (run once):
# install.packages('Seurat')
# BiocManager::install(c('glmGamPoi', 'scran'))
# install.packages(c('ggplot2','patchwork','dplyr','Matrix'))
```

```
# Load libraries:
library(Seurat)
library(ggplot2)
library(patchwork)
library(dplyr)

# ── Load count matrix from Cell Ranger output ──────────────────────
counts <- Read10X(data.dir = 'sample01_cellranger/outs/filtered_feature_bc_matrix/')
# counts: a sparse matrix (dgCMatrix class), genes × cells
# Check dimensions:
dim(counts)  # e.g. 33538 genes × 8412 cells

# ── Create Seurat object ──────────────────────────────────────────
seurat_obj <- CreateSeuratObject(
    counts    = counts,
    project   = 'TumorBiopsy_Patient01',
    min.cells = 3,    # keep genes detected in ≥3 cells
    min.features = 200  # keep cells with ≥200 genes detected
)
# min.cells = 3: removes very lowly expressed genes (noise reduction)
# min.features = 200: removes empty droplets (basic filter)

seurat_obj   # print object summary
# An object of class Seurat
# 22,000 features across 8,200 samples within 1 assay

# Add sample metadata (IMPORTANT for multi-sample experiments):
seurat_obj$sample_id    <- 'patient_01'
seurat_obj$condition    <- 'tumor'
seurat_obj$tissue_type  <- 'breast'

# For MULTIPLE samples, load separately then merge:
# counts_s1 <- Read10X('sample01/filtered_feature_bc_matrix/')
# counts_s2 <- Read10X('sample02/filtered_feature_bc_matrix/')
# s1 <- CreateSeuratObject(counts_s1, project='s1')
# s2 <- CreateSeuratObject(counts_s2, project='s2')
# seurat_merged <- merge(s1, y=s2, add.cell.ids=c('S1','S2'))
```

✔ What to Look For / Expected Output:

dim(counts): first number = genes (~20,000–33,000); second = cells (close to expected cell count).

print(seurat_obj): confirms object creation. 'features' = genes after min.cells filter.

Typical reduction: 33,538 → ~22,000 genes (low-expressed genes removed by min.cells=3).

Doublet Detection with DoubletFinder

KEY CONCEPT

A DOUBLET is a GEM that captured TWO cells instead of one. Their barcodes appear as one artificial 'super-cell' with a mixture of TWO transcriptomes.

Why doublets are dangerous: they create spurious 'intermediate' clusters between real populations.

These artifacts can lead to false biological conclusions about transitional cell states.

10x expected doublet rates:

~1,000 cells targeted → ~0.8% doublets

~5,000 cells targeted → ~4.0% doublets

~10,000 cells targeted → ~8.0% doublets

DoubletFinder algorithm: creates artificial doublets by in silico mixing of existing cells,

then identifies real cells that cluster with these artificial doublets in PCA space.

ALWAYS run DoubletFinder BEFORE QC filtering and normalization (per-sample, not merged).

PRACTICAL EXERCISE — Doublet Detection with DoubletFinder

```
# Install DoubletFinder:
# remotes::install_github('chris-mcginnis-ucsf/DoubletFinder')
library(DoubletFinder)

# DoubletFinder must run on EACH SAMPLE SEPARATELY before merging.
# Run the basic Seurat steps first (required for doublet detection):
s1 <- NormalizeData(s1)
s1 <- FindVariableFeatures(s1, nfeatures = 2000)
s1 <- ScaleData(s1)
s1 <- RunPCA(s1, npcs = 20)
s1 <- RunUMAP(s1, dims = 1:20)  # needed for DoubletFinder
s1 <- FindNeighbors(s1, dims = 1:20)
s1 <- FindClusters(s1, resolution = 0.5)

# Step 1: Optimize pK parameter (neighborhood size — most important parameter):
sweep_res <- paramSweep(s1, PCs = 1:20, sct = FALSE)
sweep_stats <- summarizeSweep(sweep_res, GT = FALSE)
bcmvn <- find.pK(sweep_stats)
# Find the pK value with the highest BCmetric:
optimal_pK <- as.numeric(as.character(bcmvn$pK[which.max(bcmvn$BCmetric)]))
cat('Optimal pK:', optimal_pK, '\n')

# Step 2: Estimate expected doublet rate based on loaded cell number:
# From 10x doublet rate table: ~8% for 10,000 cells loaded
nExp_poi <- round(0.08 * ncol(s1))  # adjust % based on your cell loading number

# Step 3: Run DoubletFinder:
s1 <- doubletFinder(s1,
   PCs     = 1:20,
   pN      = 0.25,   # proportion of artificial doublets (default 0.25)
   pK      = optimal_pK,
   nExp    = nExp_poi,
   reuse.pANN = FALSE
)

# Step 4: Extract doublet classifications (column name varies — find it):
df_col <- grep('DF.classifications', colnames(s1@meta.data), value=TRUE)
table(s1@meta.data[[df_col]])
```

```
# Singlet: cells to keep | Doublet: cells to remove

# Step 5: Visualize doublets on UMAP:
DimPlot(s1, group.by = df_col, cols = c('gray85', 'red3'),
    pt.size = 0.5) + ggtitle('DoubletFinder: Doublet Predictions')

# Step 6: Remove doublets:
s1_clean <- subset(s1, subset = !!sym(df_col) == 'Singlet')
cat('Cells before:', ncol(s1), ' | After doublet removal:', ncol(s1_clean), '\n')
```

What to Look For / Expected Output:

Doublet proportion removed: should match the expected rate (~4-8% for standard runs).

UMAP visualization: doublets often appear at borders between clusters or form small bridges.

If >15% of cells flagged: likely too aggressive — re-check nExp_poi value.

If <1% flagged: pK may be suboptimal — re-run paramSweep with broader range.

Quality Control and Filtering

KEY CONCEPT

Three per-cell QC metrics used for filtering:

nFeature_RNA: number of distinct genes detected in the cell

→ Low (<200): empty droplet or dead cell

→ High (>6,000): likely a doublet (two cells merged)

nCount_RNA: total UMI count in the cell (total transcripts captured)

→ Highly correlated with nFeature_RNA

→ Very high (>50,000): doublet signal

percent.mt: % of UMIs mapping to mitochondrial genes (MT-CO1, MT-ND1, etc.)

→ High percent.mt: damaged/dying cell

→ Cytoplasm leaks during lysis → loss of cytoplasmic mRNA

→ Mitochondria remain intact → relative increase in MT reads

→ Threshold: typically 15–25% (tissue-dependent; neurons: up to 35%)

PRACTICAL EXERCISE — QC Metrics Calculation and Filtering

```
# ── After DoubletFinder: merge clean samples ──────────────────
seurat_obj <- merge(s1_clean, y = list(s2_clean, s3_clean),
    add.cell.ids = c('P01','P02','P03'),
    project = 'MultiPatient_Tumor'
)

# ── Calculate mitochondrial gene percentage ──────────────────
```

```
# Human mitochondrial genes start with 'MT-'
seurat_obj[['percent.mt']] <- PercentageFeatureSet(
    seurat_obj,
    pattern = '^MT-'
)

# Calculate ribosomal gene percentage (optional but informative):
seurat_obj[['percent.rb']] <- PercentageFeatureSet(
    seurat_obj,
    pattern = '^RP[SL]'
)

# ── Visualize QC metrics (BEFORE filtering) ──────────────────────
# Violin plots:
VlnPlot(
    seurat_obj,
    features = c('nFeature_RNA', 'nCount_RNA', 'percent.mt'),
    ncol = 3,
    pt.size = 0.1
)
ggsave('QC_violin_before_filtering.pdf', width=14, height=5)

# Scatter plots (identify doublets as high nCount + high nFeature):
p1 <- FeatureScatter(seurat_obj, 'nCount_RNA',   'nFeature_RNA') + geom_smooth()
p2 <- FeatureScatter(seurat_obj, 'nCount_RNA',   'percent.mt')
p3 <- FeatureScatter(seurat_obj, 'nFeature_RNA', 'percent.mt')
p1 | p2 | p3
ggsave('QC_scatter_before_filtering.pdf', width=18, height=5)

# Print summary statistics before filtering:
summary(seurat_obj$nFeature_RNA)
summary(seurat_obj$percent.mt)

# ── Set thresholds (ADJUST TO YOUR DATA) ─────────────────────────
# Thresholds are NOT universal — inspect violin plots first
nFeat_low  <- 200      # remove empty droplets
nFeat_high <- 6000     # remove doublets (adjust based on your median)
nCount_high <- 50000   # remove extreme doublets
pmt_high   <- 20       # remove damaged cells (may be 25-35 for neurons)

seurat_obj <- subset(
    seurat_obj,
    subset = nFeature_RNA > nFeat_low  &
             nFeature_RNA < nFeat_high &
             nCount_RNA   < nCount_high &
             percent.mt   < pmt_high
)

# ── Report results ────────────────────────────────────────────────
cat('Cells after QC filtering:', ncol(seurat_obj), '\n')

# Re-plot after filtering to confirm improvement:
VlnPlot(seurat_obj,
    features = c('nFeature_RNA','nCount_RNA','percent.mt'),
    ncol = 3, pt.size = 0)
ggsave('QC_violin_after_filtering.pdf', width=14, height=5)
```

✔ What to Look For / Expected Output:

Before filtering: violin plots show long tails (dead cells with low nFeature, high percent.mt).

After filtering: distributions should be unimodal and clean.

Typical retention: 70–90% of cells after QC (30% loss is acceptable).

If >50% cells removed: thresholds too aggressive, or library quality was poor.

⚠ WARNING

QC thresholds are DATASET-SPECIFIC — never apply fixed numbers blindly.

percent.mt threshold depends on cell type:

Cardiomyocytes: naturally high mitochondrial content → threshold 40-50%

Neurons: up to 35% normal

PBMC (blood): typically <15%

nFeature_RNA depends on sequencing depth and cell type:

Poorly sequenced data (5,000 reads/cell): median nFeature ~500 — do not filter at >2000

Well-sequenced (50,000 reads/cell): median nFeature ~3,000 — safe to filter at >8,000

ALWAYS look at your data first. Run VlnPlot BEFORE setting thresholds.

If you use someone else's thresholds, you may remove biologically real populations.

7.3.2 Normalization

After QC filtering, raw UMI counts are normalized to correct for differences in sequencing depth between cells (a cell captured with 10,000 total UMI and a cell captured with 1,000 UMI cannot be directly compared without normalization). Log-normalization: each cell's counts are divided by the total UMI count of that cell, multiplied by a scale factor (typically 10,000), and log-transformed (log(x+1)). This is implemented in Seurat's NormalizeData() function. SCTransform: a variance-stabilizing normalization using a regularized negative binomial regression model, which simultaneously normalizes counts and regresses out technical confounders (sequencing depth, %mito). SCTransform is increasingly preferred over log-normalization.

PRACTICAL EXERCISE — SCTransform Normalization

```
# ── SCTransform normalization (recommended) ──────────────────────
# Requires glmGamPoi package for speed:
# BiocManager::install('glmGamPoi')

seurat_obj <- SCTransform(
    seurat_obj,
    vst.flavor         = 'v2',          # use improved v2 algorithm
    vars.to.regress    = 'percent.mt',  # regress out mito contamination
    variable.features.n = 3000,         # number of HVGs to identify
    ncells             = 5000,          # cells used to learn parameters
```

```
    verbose           = TRUE
)

# vars.to.regress: regress out technical covariates
#   'percent.mt'   → remove dying cell signal
#   c('percent.mt','S.Score','G2M.Score')  → also remove cell cycle effects

# ── Cell cycle scoring (optional but recommended for cycling tissues) ──
# Load cell cycle genes (built into Seurat):
s_genes   <- cc.genes.updated.2019$s.genes
g2m_genes <- cc.genes.updated.2019$g2m.genes

seurat_obj <- CellCycleScoring(
    seurat_obj,
    s.features   = s_genes,
    g2m.features = g2m_genes,
    set.ident    = TRUE
)

# Visualize cell cycle distribution:
DimPlot(seurat_obj, group.by = 'Phase') + ggtitle('Cell Cycle Phase')

# To REMOVE cell cycle effects entirely, regress them out:
seurat_obj <- SCTransform(
    seurat_obj,
    vst.flavor      = 'v2',
    vars.to.regress = c('percent.mt', 'S.Score', 'G2M.Score'),
    variable.features.n = 3000
)

# ── (LEGACY) LogNormalize approach for reference ──────────────────
# seurat_obj <- NormalizeData(seurat_obj, normalization.method = 'LogNormalize',
#     scale.factor = 10000)
# seurat_obj <- FindVariableFeatures(seurat_obj, selection.method = 'vst',
#     nfeatures = 2000)
# seurat_obj <- ScaleData(seurat_obj, vars.to.regress = 'percent.mt')
```

✔ What to Look For / Expected Output:

SCTransform adds a new 'SCT' assay to the Seurat object. DefaultAssay is set to 'SCT'.

3,000 HVGs identified: inspect with VariableFeaturePlot(seurat_obj) — top 10 labeled.

If SCTransform takes >30 min: reduce ncells=2000 or use LogNormalize as alternative.

7.3.3 Highly Variable Gene Selection

A standard scRNA-Seq dataset contains 20,000+ genes, but most genes are not informative for distinguishing cell types (housekeeping genes have uniform expression across cells; very lowly expressed genes are dominated by technical noise). FindVariableFeatures() identifies the ~2,000 genes with the highest cell-to-cell variability — these are the genes that best distinguish different cell populations and are used for downstream dimensionality reduction. Visualization using VariableFeaturePlot() shows the mean expression vs. variability relationship.

7.3.4 Scaling and PCA

Data is scaled (ScaleData()) so that each gene has zero mean and unit variance across cells — necessary so that highly expressed genes do not dominate dimensionality reduction. Principal Component Analysis (PCA, RunPCA()) then reduces the dimensionality from ~2,000 variable genes to ~30–50 principal components. The ElbowPlot() shows the amount of variance explained by each PC, helping select the number of PCs to use in subsequent steps (typically 10–30, capturing most biological variation).

PRACTICAL EXERCISE — PCA and Dimensionality Selection

```
# ── Run PCA ──────────────────────────────────────────────────────────

seurat_obj <- RunPCA(
    seurat_obj,
    npcs     = 50,          # compute first 50 PCs
    features = VariableFeatures(seurat_obj),  # use HVGs only
    verbose  = FALSE
)

# ── Assess how many PCs to use ──────────────────────────────────────
# Method 1: ElbowPlot (look for the 'elbow' where variance levels off)
ElbowPlot(seurat_obj, ndims = 50)
ggsave('PCA_elbow_plot.pdf', width=8, height=5)
# Choose the number of PCs at the elbow (typically 10-30)

# Method 2: JackStraw permutation test (statistical, but slow):
# seurat_obj <- JackStraw(seurat_obj, num.replicate = 100, dims = 30)
# seurat_obj <- ScoreJackStraw(seurat_obj, dims = 1:30)
# JackStrawPlot(seurat_obj, dims = 1:30)

# ── Visualize top genes driving each PC ─────────────────────────────
VizDimLoadings(seurat_obj, dims = 1:2, reduction = 'pca')

# Heatmap of top cells and genes for PC1-9:
DimHeatmap(seurat_obj, dims = 1:9, cells = 500, balanced = TRUE)
ggsave('PCA_heatmap_PC1-9.pdf', width=12, height=15)

# Print top genes for first 5 PCs:
print(seurat_obj[['pca']], dims = 1:5, nfeatures = 10)

# 2D PCA plot (quick cell separation check):
DimPlot(seurat_obj, reduction = 'pca', group.by = 'orig.ident')
```

What to Look For / Expected Output:

ElbowPlot: choose n PCs where the curve 'elbows' and flattens. Typical: 15-30 PCs.

Using too few PCs: miss subtle cell type distinctions. Too many: add noise.

VizDimLoadings: PC1 often driven by MT genes (if not regressed) or housekeeping genes.

The chosen n_dims is used consistently in FindNeighbors, RunUMAP, RunHarmony.

Batch Correction with Harmony

KEY CONCEPT

Batch effects in scRNA-Seq: technical variation between samples processed on different days, by different operators, with different library prep lots, or from different patients.

When to use batch correction:

✓ Multiple samples processed at different times

✓ Samples from different patients (donor effects)

✓ Multiple sequencing runs merged together

✗ Single sample — no batch correction needed

✗ If biological differences ARE the expected signal (e.g., tumor vs. normal)

Harmony (Korsunsky et al., 2019, Nature Methods): iteratively adjusts PCA embeddings to maximize mixing between batches while preserving biological variation.

Harmony runs on the PCA matrix, not on raw counts — fast and memory-efficient.

Alternative tools: Seurat CCA integration, scVI, BBKNN, scANVI.

PRACTICAL EXERCISE — Batch Correction with Harmony

```
# Install harmony:

# install.packages('harmony')  OR  remotes::install_github('immunogenomics/harmony')
library(harmony)

# Run Harmony on PCA embeddings:
seurat_obj <- RunHarmony(
    seurat_obj,
    group.by.vars = c('sample_id'),   # correct for sample identity
    reduction     = 'pca',
    dims.use      = 1:30,
    max_iter      = 10,
    verbose       = TRUE
)
# group.by.vars can include multiple variables: c('sample_id','batch','patient')

# Harmony creates a new reduction slot: 'harmony'
# Use 'harmony' reduction instead of 'pca' for all downstream steps:

# ── Before/after Harmony comparison ──────────────────────────
# Before Harmony (PCA reduction — samples separated by batch):
p_before <- DimPlot(seurat_obj, reduction='pca',
    group.by='sample_id', dims=c(1,2)) +
    ggtitle('Before Harmony: PCA')

# After Harmony (harmony reduction — samples should intermix):
```

```
seurat_obj <- RunUMAP(seurat_obj, reduction='harmony', dims=1:30,
    reduction.name='umap.harmony')
p_after <- DimPlot(seurat_obj, reduction='umap.harmony',
    group.by='sample_id') +
    ggtitle('After Harmony: UMAP')

p_before | p_after
ggsave('Harmony_batch_correction_comparison.pdf', width=14, height=6)
```

What to Look For / Expected Output:

Before Harmony: UMAP splits cells by sample_id (batch clusters visible).

After Harmony: cells mix across samples while clusters remain (biology preserved).

If clusters still separate by sample after Harmony: biological differences are real, not batch.

Over-correction warning: if known cell types merge together — reduce max_iter or use theta=1.

7.3.5 Clustering (Louvain Algorithm)

Cell clustering in Seurat uses a graph-based approach: FindNeighbors() constructs a K-nearest-neighbor graph in PCA space (each cell is connected to its K most similar cells in PC space). FindClusters() then applies the Louvain community detection algorithm (or Leiden algorithm) to partition this graph into clusters. The resolution parameter controls cluster granularity: low resolution (0.1–0.3) produces few broad clusters; high resolution (1.0–2.0) produces many fine-grained clusters. Resolution should be chosen based on biological knowledge of expected cell types.

PRACTICAL EXERCISE — FindNeighbors and FindClusters

```
# ── Build KNN graph (use harmony reduction if batch-corrected) ──
seurat_obj <- FindNeighbors(
    seurat_obj,
    reduction = 'harmony',   # or 'pca' if single sample
    dims      = 1:30,        # must match dims used in Harmony/PCA
    k.param   = 20           # number of neighbors
)

# ── Run clustering at multiple resolutions to compare ──────────
for (res in c(0.2, 0.4, 0.6, 0.8, 1.0, 1.2)) {
    seurat_obj <- FindClusters(
        seurat_obj,
        resolution  = res,
        algorithm   = 1,    # 1=Louvain | 4=Leiden (requires leidenalg)
        random.seed = 42,
        verbose     = FALSE
    )
    cat('Resolution', res, ':', length(unique(seurat_obj$seurat_clusters)), 'clusters\n')
}

# ── Select your preferred resolution ──────────────────────────
# After examining marker genes (section 7.4.9), choose the best resolution.
# Set active identity to the chosen resolution:
Idents(seurat_obj) <- 'SCT_snn_res.0.6'   # use resolution 0.6
```

```
# ── Assess cluster stability with clustree ──────────────────────
# install.packages('clustree')
library(clustree)
clustree(seurat_obj, prefix = 'SCT_snn_res.') +
    ggtitle('Cluster stability across resolutions')
ggsave('clustree_stability.pdf', width=12, height=10)
# clustree shows how clusters split/merge as resolution increases
# Choose the resolution where clusters are stable (few arrows changing)
```

✔ What to Look For / Expected Output:

The for loop prints: e.g., 'Resolution 0.6: 14 clusters'.

clustree: look for the resolution where large clusters stop splitting unpredictably.

Typical starting point: resolution 0.4–0.8 for 5,000–30,000 cells.

After this step: cluster assignments are in seurat_obj$seurat_clusters.

7.3.6 UMAP and t-SNE Visualization

UMAP (Uniform Manifold Approximation and Projection) and t-SNE (t-distributed Stochastic Neighbor Embedding) are non-linear dimensionality reduction methods that project high-dimensional PC space into 2D for visualization. RunUMAP() uses the PCA embedding as input. UMAP generally preserves both local and global structure better than t-SNE, produces more reproducible layouts, and is computationally faster. The UMAP plot, colored by cluster identity, is the canonical visualization of scRNA-Seq data.

⚠ COMMON PITFALL

UMAP is a visualization tool, NOT an analysis tool.

Cluster boundaries in UMAP do not perfectly correspond to real biological boundaries.

Distances between clusters in UMAP are not biologically interpretable.

Two clusters that appear distant on UMAP may be transcriptionally similar,

and two clusters that appear adjacent may be biologically distinct.

Always validate biological conclusions with gene expression data, not UMAP geometry.

PRACTICAL EXERCISE — Running and Plotting UMAP

```
# ── Compute UMAP (if not already done after Harmony) ──────────
seurat_obj <- RunUMAP(
    seurat_obj,
    reduction       = 'harmony',  # or 'pca'
    dims            = 1:30,
    n.neighbors     = 30,    # larger = more global structure preserved
    min.dist        = 0.3,   # smaller = tighter clusters
    spread          = 1,
    seed.use        = 42,    # for reproducibility
    reduction.name  = 'umap',
    reduction.key   = 'UMAP_'
)
```

```
# ── Publication-quality UMAP plots ──────────────────────────────
library(ggplot2)

# By cluster:
p1 <- DimPlot(seurat_obj, reduction='umap', label=TRUE, label.size=4,
    pt.size=0.5, repel=TRUE) +
    ggtitle('Clusters (resolution 0.6)') +
    theme_classic()

# Split by condition/sample (compare batch-corrected distributions):
p2 <- DimPlot(seurat_obj, reduction='umap', group.by='condition',
    split.by='sample_id', pt.size=0.3) +
    ggtitle('UMAP split by sample')

# By cell cycle phase:
p3 <- DimPlot(seurat_obj, reduction='umap', group.by='Phase', pt.size=0.4)

# Feature plots — expression of specific genes:
FeaturePlot(
    seurat_obj,
    features = c('EPCAM','CD3E','CD14','CD19','COL1A1','PECAM1'),
    ncol = 3, pt.size = 0.3,
    order = TRUE  # plot expressing cells on top
)
ggsave('UMAP_marker_gene_FeaturePlot.pdf', width=15, height=10)

# Combine plots:
(p1 | p2) / p3
ggsave('UMAP_overview.pdf', width=16, height=12)
```

✔ What to Look For / Expected Output:

DimPlot with label=TRUE: each cluster shows its number on the UMAP.

FeaturePlot: marker genes should show spatially restricted expression in specific clusters.

CD3E (T cells), CD14 (monocytes), CD19 (B cells), EPCAM (epithelial), COL1A1 (fibroblasts).

UMAP is for visualization only — cluster distances are NOT quantitatively meaningful.

7.3.7 Cell Type Annotation

Seurat clusters are numbered (0, 1, 2, ...) and must be annotated with biologically meaningful cell type labels. Manual annotation uses known marker genes: canonical marker genes whose expression is well-established for specific cell types are used to annotate clusters. Examples: CD3E/CD3D (T cells), CD19/MS4A1 (B cells), CD14/CST3 (Monocytes), EPCAM/KRT8 (epithelial cells), COL1A1/VIM (fibroblasts), PECAM1/CDH5 (endothelial cells). FindAllMarkers() identifies differentially expressed genes (marker genes) for each cluster compared to all other clusters, ordered by log2 fold change and adjusted p-value. DotPlot() and FeaturePlot() visualize marker gene expression across clusters.

KEY CONCEPT

Seurat clusters are numbered 0, 1, 2, ... and must be annotated with biologically

meaningful cell type labels. This is done by examining marker gene expression.

Two-step approach:

1. Unsupervised: FindAllMarkers() — for each cluster, find differentially expressed genes vs. all other clusters → produces a ranked list of genes per cluster
2. Manual annotation: look up top markers in literature, databases (PanglaoDB, CellMarker 2.0), and your own biological knowledge

Key human cell type markers (canonical list):

Cell Type	Positive Markers	Negative Markers (if needed)
T cells (all)	CD3E, CD3D, CD3G, TRAC	CD19, CD14
CD4+ T helper	CD4, IL7R, CCR7 (naive)	CD8A, CD8B
CD8+ T cytotoxic	CD8A, CD8B, GZMB (effector)	CD4
Treg	FOXP3, IL2RA (CD25), CTLA4	–
NK cells	NCAM1 (CD56), FCGR3A, GNLY	CD3E
B cells	CD19, MS4A1 (CD20), CD79A	CD3E, CD14
Plasma cells	MZB1, IGHG1, XBP1, PRDM1	CD19, MS4A1
Classical monocytes	CD14, LYZ, S100A8, FCGR3A	FCGR3A (high)
Non-classical monocytes	FCGR3A (high), CX3CR1	CD14
Dendritic cells	FCER1A, CD1C, CLEC9A (pDC: LILRA4)	CD14
Macrophages (M1)	CD68, CD80, TNF, IL1B	–
Macrophages (M2/TAM)	MRC1 (CD206), CD163, CSF1R	–
Epithelial cells	EPCAM, KRT8, KRT18, CDH1	CD45 (PTPRC)
Cancer stem-like	SOX2, ALDH1A1, CD44hi, PROM1	–
Fibroblasts (CAF)	COL1A1, COL1A2, FAP, VIM	EPCAM
Endothelial cells	PECAM1 (CD31), CDH5, VWF	EPCAM, CD45
Mast cells	TPSAB1, CPA3, KIT	–

PRACTICAL EXERCISE — Finding Cluster Markers and Annotating Cell Types

```
# ── FindAllMarkers: identify marker genes for every cluster ──
# This compares each cluster vs. all others (Wilcoxon rank-sum test by default)
all_markers <- FindAllMarkers(
    seurat_obj,
    only.pos    = TRUE,   # only upregulated markers (positive markers)
    min.pct     = 0.25,   # gene must be detected in ≥25% of cluster cells
    logfc.threshold = 0.25, # minimum log2 fold-change threshold
    test.use    = 'wilcox',  # or 'LR','MAST','negbinom'
    verbose     = TRUE
)

# Filter to significant markers:
sig_markers <- all_markers %>%
```

```
    filter(p_val_adj < 0.05, avg_log2FC > 0.5) %>%
    group_by(cluster) %>%
    slice_max(order_by = avg_log2FC, n = 10)   # top 10 per cluster

# Write to file for review:
write.csv(sig_markers, 'top10_markers_per_cluster.csv')

# ── Visual inspection of marker expression ──────────────────────
# DotPlot: size = % cells expressing; color = mean expression
canonical_markers <- c(
    'CD3E','CD8A','CD4','FOXP3',       # T cells
    'NCAM1','GNLY',                    # NK
    'CD19','MS4A1',                    # B cells
    'MZB1','PRDM1',                    # Plasma
    'CD14','LYZ','FCGR3A',             # Myeloid
    'EPCAM','KRT18',                   # Epithelial
    'COL1A1','FAP',                    # Fibroblast
    'PECAM1','CDH5'                    # Endothelial
)
DotPlot(seurat_obj, features=canonical_markers, dot.scale=6) +
    RotatedAxis() +
    scale_color_gradient2(low='blue', mid='white', high='red') +
    ggtitle('Canonical marker expression per cluster')
ggsave('DotPlot_canonical_markers.pdf', width=16, height=8)

# Heatmap of top 5 markers per cluster:
top5 <- all_markers %>% group_by(cluster) %>% slice_max(avg_log2FC, n=5)
DoHeatmap(seurat_obj, features = top5$gene, size=3) +
    scale_fill_gradient2(low='blue', mid='white', high='red')
ggsave('Heatmap_top5_markers.pdf', width=16, height=12)

# ── Assign cell type labels based on marker analysis ────────────
cell_type_labels <- c(
    '0' = 'CD8+ T cells',
    '1' = 'CD4+ T helper',
    '2' = 'Cancer cells',
    '3' = 'TAM (M2 macrophages)',
    '4' = 'B cells',
    '5' = 'Fibroblasts',
    '6' = 'NK cells',
    '7' = 'Plasma cells',
    '8' = 'Endothelial',
    '9' = 'Tregs',
    '10' = 'Classical monocytes',
    '11' = 'Cancer stem-like'
)
seurat_obj <- RenameIdents(seurat_obj, cell_type_labels)
seurat_obj$cell_type <- Idents(seurat_obj)

# Final annotated UMAP:
DimPlot(seurat_obj, reduction='umap', label=TRUE, label.size=4,
    repel=TRUE, pt.size=0.4) +
```

```
    ggtitle('Annotated Cell Types') +
    theme_classic()
ggsave('UMAP_annotated_cell_types.pdf', width=10, height=8)

# Cell type composition per sample:
cell_counts <- table(seurat_obj$cell_type, seurat_obj$sample_id)
cell_proportions <- prop.table(cell_counts, margin=2)  # proportions per sample
barplot(cell_proportions, legend=TRUE, col=rainbow(nrow(cell_proportions)))
```

✔ What to Look For / Expected Output:

sig_markers CSV: open in Excel. Each row = one marker gene. Key columns:

cluster, gene, avg_log2FC (higher = more specific), pct.1 (% in cluster), pct.2 (% in others).

DotPlot: canonical markers should show clear cluster-specific expression — validates annotation.

cell_proportions: bar chart showing % of each cell type per patient/sample.

7.4 Differential Expression and Pathway Enrichment

FindMarkers() performs pairwise differential expression between selected cell populations. Multiple test correction is applied (Bonferroni or Benjamini-Hochberg FDR correction). Results show: gene, p_val, avg_log2FC, pct.1 (% of cells in group 1 expressing the gene), pct.2 (% in group 2), p_val_adj. Statistical methods include Wilcoxon rank-sum test (default, non-parametric), logistic regression, MAST (a Bayesian method accounting for bimodal expression distributions), and DESeq2 (pseudobulk approach, treating multiple cells from the same sample as replicates — the recommended approach for multi-sample experimental comparisons).

Pathway enrichment: Gene Ontology (GO) enrichment analysis and KEGG pathway analysis using clusterProfiler (R package) determine which biological processes are overrepresented among differentially expressed genes. Gene Set Enrichment Analysis (GSEA) using Hallmark or MSigDB gene sets identifies pathway-level expression differences without a binary up/down call threshold.

🔬 PRACTICAL EXERCISE — Differential Expression with FindMarkers and Pseudobulk DESeq2

```
# ── Method 1: FindMarkers — pairwise cluster comparison ──────
# Compare Tregs (cluster 9) vs. CD4+ T helper (cluster 1):
treg_vs_cd4 <- FindMarkers(
    seurat_obj,
    ident.1      = 'Tregs',
    ident.2      = 'CD4+ T helper',
    test.use     = 'MAST',
    latent.vars  = 'percent.mt',  # control for mito contamination
    min.pct      = 0.1,
    logfc.threshold = 0.25
)
treg_vs_cd4 <- treg_vs_cd4[order(treg_vs_cd4$avg_log2FC, decreasing=TRUE), ]
head(treg_vs_cd4, 20)
# Top genes: FOXP3, IL2RA, CTLA4, TIGIT should appear for Tregs

# Volcano plot:
# install.packages('EnhancedVolcano')
```

```
library(EnhancedVolcano)
EnhancedVolcano(treg_vs_cd4,
    lab      = rownames(treg_vs_cd4),
    x        = 'avg_log2FC',
    y        = 'p_val_adj',
    pCutoff  = 0.05,
    FCcutoff = 0.5,
    title    = 'Tregs vs CD4+ T helper cells'
)
ggsave('Volcano_Treg_vs_CD4.pdf', width=10, height=8)

# ── Method 2: Pseudobulk DESeq2 (RECOMMENDED for condition-based DE) ──
# RATIONALE: cells from the same patient are NOT independent observations.
# Treating 1,000 T cells from one patient as 1,000 independent replicates
# massively inflates sample size → false positives.
# Pseudobulk: sum UMI counts per cell type per PATIENT → treat patients as replicates.

# Requires ≥3 patients per condition (pseudo-replication)
library(DESeq2)

# Step 1: Subset to cell type of interest:
macro_obj <- subset(seurat_obj, subset = cell_type == 'TAM (M2 macrophages)')

# Step 2: Aggregate counts per patient:
# install.packages('Matrix')
# Pseudobulk using Seurat's AggregateExpression:
pseudobulk_counts <- AggregateExpression(
    macro_obj,
    assays    = 'RNA',
    return.seurat = FALSE,
    group.by  = c('sample_id', 'condition')  # aggregate per patient per condition
)$RNA

# Step 3: Create metadata for DESeq2:
# Each column = one patient-condition combination
coldata <- data.frame(
    sample   = colnames(pseudobulk_counts),
    condition = gsub('.*_','', colnames(pseudobulk_counts)),
    row.names = colnames(pseudobulk_counts)
)

# Step 4: DESeq2 analysis:
dds <- DESeqDataSetFromMatrix(
    countData = round(pseudobulk_counts),
    colData   = coldata,
    design    = ~ condition
)
dds <- dds[rowSums(counts(dds) >= 5) >= 3, ]  # filter lowly expressed genes
dds <- DESeq(dds)
res <- results(dds,
    contrast  = c('condition', 'resistant', 'sensitive'),
    alpha     = 0.05
```

```
)
res_df <- as.data.frame(res) %>%
  filter(!is.na(padj)) %>%
  arrange(padj)
write.csv(res_df, 'DESeq2_pseudobulk_TAM_resistant_vs_sensitive.csv')

# Summary:
summary(res)  # shows number of up/downregulated genes at padj < 0.1
```

What to Look For / Expected Output:

FindMarkers MAST output: gene, p_val, avg_log2FC, pct.1, pct.2, p_val_adj.

Volcano plot: x-axis = log2FC (positive = up in Tregs); y-axis = -log10(p_val_adj).

DESeq2 pseudobulk output: log2FoldChange, lfcSE, stat, pvalue, padj.

Significant genes: padj < 0.05 AND |log2FC| > 1. Expect 50-500 DE genes for macrophages.

Pathway enrichment analyse

PRACTICAL EXERCISE — Gene Ontology and KEGG Enrichment with clusterProfiler

```
# BiocManager::install(c('clusterProfiler','org.Hs.eg.db','enrichplot','ggplot2'))

library(clusterProfiler)
library(org.Hs.eg.db)  # human gene annotation database
library(enrichplot)

# ── Prepare gene list from DESeq2 results ──────────────────────
# Upregulated genes in resistant TAMs:
up_genes <- res_df %>%
  filter(padj < 0.05, log2FoldChange > 1) %>%
  rownames()

# Convert gene symbols to Entrez IDs (required for some databases):
gene_ids <- bitr(up_genes,
  fromType = 'SYMBOL',
  toType   = 'ENTREZID',
  OrgDb    = org.Hs.eg.db
)

# ── Gene Ontology (GO) Enrichment Analysis ─────────────────────
go_bp <- enrichGO(
  gene         = gene_ids$ENTREZID,
  OrgDb        = org.Hs.eg.db,
  ont          = 'BP',         # BP=Biological Process, CC, MF
  pAdjustMethod = 'BH',        # Benjamini-Hochberg correction
  pvalueCutoff  = 0.05,
  qvalueCutoff  = 0.2,
  readable      = TRUE         # convert IDs back to symbols in output
)

# Barplot of top GO terms:
barplot(go_bp, showCategory=20, title='GO Biological Process') +
```

```
    theme(axis.text.y = element_text(size=10))
ggsave('GO_BP_upregulated_resistant_TAMs.pdf', width=10, height=8)

# Dot plot:
dotplot(go_bp, showCategory=20) +
    ggtitle('GO-BP enrichment: Resistant TAMs')

# ── KEGG Pathway Enrichment ──────────────────────────────────────
kegg_res <- enrichKEGG(
    gene         = gene_ids$ENTREZID,
    organism     = 'hsa',   # hsa = Homo sapiens
    pAdjustMethod = 'BH',
    pvalueCutoff = 0.05
)
dotplot(kegg_res, showCategory=15) + ggtitle('KEGG pathway enrichment')

# ── Gene Set Enrichment Analysis (GSEA) — uses ALL genes, not just significant ──
# GSEA uses the full ranked gene list (by log2FC or -log10padj * sign(log2FC))
gene_list_ranked <- res_df$log2FoldChange
names(gene_list_ranked) <- rownames(res_df)
gene_list_ranked <- sort(gene_list_ranked, decreasing=TRUE)  # must be sorted

# GSEA with MSigDB Hallmark gene sets:
# BiocManager::install('msigdbr')
library(msigdbr)
hallmark_sets <- msigdbr(species='Homo sapiens', category='H') %>%
    dplyr::select(gs_name, gene_symbol)

gsea_res <- GSEA(
    geneList      = gene_list_ranked,
    TERM2GENE     = hallmark_sets,
    pvalueCutoff  = 0.05,
    pAdjustMethod = 'BH',
    minGSSize     = 15,
    maxGSSize     = 500
)

# Plot top enriched pathways:
gseaplot2(gsea_res, geneSetID=1:5, title='GSEA: Top Hallmark Pathways')
ggsave('GSEA_hallmark_resistant_TAMs.pdf', width=12, height=8)
```

✔ **What to Look For / Expected Output:**

GO-BP barplot: top terms should be biologically coherent (e.g., 'immune response', 'cytokine signaling').

GSEA plot: normalized enrichment score (NES) >1 = upregulated pathway; running sum curve shows leading edge.

Common finding in resistant TAMs: HALLMARK_INFLAMMATORY_RESPONSE, HALLMARK_TNFA_SIGNALING_VIA_NFKB.

Export results: write.csv(as.data.frame(go_bp), 'GO_BP_results.csv')

7.5 Trajectory Analysis and Pseudotime

In developmental biology and cancer biology, cells exist along continuous differentiation trajectories rather than discrete states. Trajectory analysis infers the ordering of cells along a developmental timeline (pseudotime) based on their transcriptional similarity. Monocle3 constructs a 'principal graph' through the data in UMAP space, selecting a 'root cell' (the starting point of the trajectory), and assigns each cell a pseudotime value representing its developmental progress along the trajectory. Trajectory analysis is used to study haematopoietic differentiation, tumor progression, treatment response, and cell fate decisions.

KEY CONCEPT

Trajectory analysis addresses a fundamental limitation of clustering:

biological processes (differentiation, activation, response to therapy) are CONTINUOUS,

not discrete. Cells exist along a continuum of states, not in rigid clusters.

Pseudotime: a computational ordering of cells along an inferred developmental trajectory.

It represents biological progression without a real time axis.

Monocle3 approach:

1. Learn principal graph: a 'skeleton' through the UMAP/PCA embedding
2. Choose root node: the starting point of the trajectory (biologically motivated)
3. Assign pseudotime: each cell receives a value (0 = root) representing its position along the trajectory

Use cases: T cell exhaustion progression, cancer cell dedifferentiation,

CAF activation states, monocyte-to-macrophage differentiation.

PRACTICAL EXERCISE — Trajectory Analysis with Monocle3

```
# BiocManager::install('monocle3')  # or from github: cole-trapnell-lab/monocle3

library(monocle3)
library(SeuratWrappers)  # for converting Seurat to CellDataSet

# ── Convert Seurat object to Monocle3 CellDataSet ──────────────
cds <- as.cell_data_set(seurat_obj)

# Transfer cluster information from Seurat:
cds <- cluster_cells(cds, reduction_method='UMAP')
cds@clusters$UMAP$clusters <- as.factor(seurat_obj$cell_type)

# ── Learn the principal graph (trajectory) ─────────────────────
cds <- learn_graph(
   cds,
   use_partition   = FALSE,  # FALSE: allows trajectories across partitions
   learn_graph_control = list(
      minimal_branch_len = 10,   # minimum branch length (prunes small branches)
      orthogonal_proj_tip = FALSE
```

```
    )
)

# ── Visualize the trajectory ──────────────────────────────────────
plot_cells(
    cds,
    color_cells_by     = 'cell_type',
    label_groups_by_cluster = FALSE,
    label_leaves       = FALSE,
    label_branch_points = TRUE,
    graph_label_size   = 3
)

# ── Order cells in pseudotime ─────────────────────────────────────
# Choose root node: biologically, where does the trajectory start?
# For T cell exhaustion: naive T cells (CD3E+, CCR7+, TCF7+) are the root.
# For macrophage activation: monocytes are the root.

# Interactive root selection (opens a browser window):
# cds <- order_cells(cds)  # click on root in the plot

# OR: programmatically set root using a known marker gene:
# Find the node closest to cells expressing CCR7 (naive T cell marker):
root_cells <- colnames(cds)[which(
    normalized_counts(cds)['CCR7',] > quantile(
        normalized_counts(cds)['CCR7',], 0.90
    )
)]
cds <- order_cells(cds, root_cells = root_cells)

# ── Plot pseudotime ───────────────────────────────────────────────
p1 <- plot_cells(
    cds,
    color_cells_by = 'pseudotime',
    label_cell_groups = FALSE,
    label_leaves   = FALSE,
    label_branch_points = FALSE,
    graph_label_size = 3
) + ggtitle('Pseudotime ordering')

p2 <- plot_cells(
    cds,
    color_cells_by = 'cell_type',
    label_cell_groups = TRUE
) + ggtitle('Cell types on trajectory')

p1 | p2
ggsave('Monocle3_pseudotime.pdf', width=14, height=7)

# ── Find genes that change with pseudotime ────────────────────────
# graph_test: identifies genes with spatially correlated expression along trajectory
pr_test_res <- graph_test(
```

```
    cds,
    neighbor_graph = 'principal_graph',
    cores = 8
)

# Significant pseudotime-associated genes:
pt_genes <- pr_test_res %>%
    filter(q_value < 0.01, morans_I > 0.1) %>%
    arrange(desc(morans_I))
head(pt_genes, 20)

# Plot top pseudotime-variable genes:
plot_cells(
    cds,
    genes   = head(pt_genes$gene_short_name, 6),
    show_trajectory_graph = FALSE,
    label_cell_groups     = FALSE
)
```

What to Look For / Expected Output:

principal graph: lines connecting nodes visible on the UMAP, passing through cell clusters.

Pseudotime gradient: cells near root = 0 (dark); cells far = maximum value (yellow/bright).

Coherent trajectory: naive T cells should have low pseudotime; exhausted (PDCD1+, HAVCR2+) high pseudotime.

graph_test: Moran's I statistic — values close to 1 = strong spatial autocorrelation along trajectory.

7.6 AI Tools for scRNA-Seq Annotation

Manual cell type annotation is time-consuming and requires domain expertise. Several AI tools now automate annotation: scVI (single-cell Variational Inference) uses a deep generative model to embed cells in a latent space correcting for batch effects; Scimilarity (Genentech) uses contrastive learning trained on millions of cells to annotate cell types from the latent embedding; GPT-4 and Llama-based models can annotate marker gene lists with cell type predictions when prompted appropriately.

Critical perspective: AI annotation tools reflect biases in their training datasets — predominantly cells from specific tissue types, species, and disease contexts. When applied to understudied cell types or rare populations, AI annotations can be systematically wrong. The EU AI Act (coming into full effect 2025–2026) classifies certain AI applications in medical diagnostics as 'high-risk AI systems' requiring documentation, validation, and human oversight. Blind reliance on AI cell type annotations without manual review using known marker genes is scientifically and, in clinical contexts, potentially regulatorily non-compliant.

7.8 Cell-Cell Communication Analysis with CellChat

KEY CONCEPT

Cells in a tissue communicate through ligand-receptor interactions.

CellChat predicts these interactions from scRNA-Seq expression data by:

1. Computing communication probability between sender (ligand) and

receiver (receptor) cell types based on average expression

2. Using a curated database of ligand-receptor pairs (CellChatDB)

3. Applying statistical testing and network analysis

Clinical relevance: identify tumor-immune crosstalk; find therapeutic targets in ligand-receptor interactions (e.g., PD-L1/PD-1, TGF-β signaling).

PRACTICAL EXERCISE — Cell-Cell Communication with CellChat

```
# remotes::install_github('jinworks/CellChat')
library(CellChat)
library(patchwork)

# ── Create CellChat object from Seurat ──────────────────────────
# Extract normalized expression matrix and metadata:
data_input <- GetAssayData(seurat_obj, assay='SCT', layer='data')
meta_data  <- data.frame(cell_type = seurat_obj$cell_type)

cellchat <- createCellChat(
    object   = data_input,
    meta     = meta_data,
    group.by = 'cell_type'
)

# ── Set CellChat database ────────────────────────────────────────
# Human ligand-receptor database:
CellChatDB <- CellChatDB.human
cellchat@DB <- CellChatDB

# ── Compute communication probability ────────────────────────────
cellchat <- subsetData(cellchat)        # filter to expressed genes only
cellchat <- identifyOverExpressedGenes(cellchat)
cellchat <- identifyOverExpressedInteractions(cellchat)

# Core computation (can take 15-30 min for large datasets):
cellchat <- computeCommunProb(cellchat, type='triMean')
cellchat <- filterCommunication(cellchat, min.cells=10)
cellchat <- computeCommunProbPathway(cellchat)
cellchat <- aggregateNet(cellchat)

# ── Visualizations ──────────────────────────────────────────────
# Circle plot: interaction strength between all cell types:
netVisual_circle(
    cellchat@net$weight,
    vertex.weight     = as.numeric(table(cellchat@idents)),
    weight.scale      = TRUE,
    label.edge.weight = FALSE,
    title.name        = 'Interaction strength'
)
```

```
# Heatmap of pathways:
netAnalysis_signalingRole_heatmap(cellchat, pattern='outgoing')

# Chord diagram for specific pathway (e.g., MHC-II signaling):
netVisual_chord_gene(cellchat, sources.use='TAM (M2 macrophages)',
    targets.use = c('CD8+ T cells','CD4+ T helper'),
    signaling   = 'MHC-II'
)

# Bubble plot: ligand-receptor pairs between TAMs and T cells:
netVisual_bubble(
    cellchat,
    sources.use = 'TAM (M2 macrophages)',
    targets.use = c('CD8+ T cells','CD4+ T helper'),
    remove.isolate = FALSE
)
ggsave('CellChat_TAM_Tcell_interactions.pdf', width=12, height=10)
```

✔ What to Look For / Expected Output:

Circle plot: thicker/brighter arrows = stronger signaling. TAMs often signal strongly to T cells.

Outgoing heatmap: shows which cell types are the primary senders for each pathway.

Bubble plot: each bubble = one ligand-receptor pair. Size = significance; color = probability.

Key interactions in tumor microenvironment: TGF-β (immunosuppression), PD-L1/PD-1 (exhaustion).

7.9 AI-Based Cell Type Annotation — Opportunities and Critical Evaluation

Manual cell type annotation is time-consuming and requires deep biological expertise. Several AI tools now automate or accelerate this process. Understanding how they work and their limitations is critical before relying on them in clinical or translational research.

7.9.1 SingleR — Reference-Based Annotation

PRACTICAL EXERCISE — Automated Annotation with SingleR

```
# BiocManager::install(c('SingleR','celldex'))

library(SingleR)
library(celldex)

# Load a reference dataset (Human Primary Cell Atlas):
ref <- HumanPrimaryCellAtlasData()  # or MonacoImmuneData() for immune-focused

# Extract normalized counts from Seurat (SCE format):
library(SingleCellExperiment)
sce <- as.SingleCellExperiment(seurat_obj, assay='SCT')

# Run SingleR annotation:
pred <- SingleR(
```

```
    test       = sce,
    ref        = ref,
    labels     = ref$label.fine,  # or label.main for broader categories
    de.method  = 'wilcox'
)

# Check annotation results:
table(pred$pruned.labels)  # cell type counts (pruned = low-confidence removed)

# Add predictions to Seurat object:
seurat_obj$singler_labels <- pred$pruned.labels

# Compare SingleR to manual annotation:
table(seurat_obj$cell_type, seurat_obj$singler_labels)

# Visualize annotation quality (delta score = confidence):
plotScoreHeatmap(pred, max.labels=25)
plotDeltaDistribution(pred)  # low delta = ambiguous annotation
ggsave('SingleR_annotation_quality.pdf', width=12, height=8)
```

✔ What to Look For / Expected Output:

table(seurat_obj$cell_type, seurat_obj$singler_labels): diagonal = concordance between manual and SingleR.

plotDeltaDistribution: cells with low delta score (near zero) have ambiguous label — verify manually.

SingleR works best for well-characterized cell types present in the reference dataset.

7.9.2 Critical Evaluation of AI Annotation Tools

AI Tool	Description, Strengths, and Critical Limitations
SingleR	Reference-based: assigns labels by correlation with reference datasets. Good for: well-characterized immune cell types. Limitation: labels constrained to reference — cannot discover novel types.
scVI (PyTorch/scvi-tools)	Variational autoencoder for dimensionality reduction + batch correction. Excellent for large multi-dataset integration. Limitation: latent space less interpretable; hyperparameter sensitive.
scANVI	Extension of scVI with semi-supervised annotation transfer. Uses labeled + unlabeled cells jointly. Good for cross-dataset annotation. Limitation: requires labeled seed cells.
Scimilarity	Contrastive learning on 22M cells. Fast, accurate for common cell types. Limitation: trained on specific tissues — performance on rare/novel types untested.
GPT-4/LLM prompting	Input ranked marker genes → LLM returns cell type suggestion. Practical shortcut. Limitation: hallucinations (wrong answer stated confidently), no uncertainty estimate.
CellTypist	Logistic regression on scRNA-Seq with pre-trained models for 20+ tissues. Fast and memory-efficient. Limitation: models fixed at training time — new populations missed.

⚠ WARNING

EU AI Act (effective 2025-2026) and AI tools in clinical genomics:

AI-based cell type annotation tools are increasingly used in translational research.

When used to support clinical decisions (e.g., identifying treatment-resistant subclones),

they may qualify as 'high-risk AI systems' under the EU AI Act — requiring:

- Technical documentation of training data, architecture, and validation
- Human oversight at all decision points
- Conformity assessment before clinical deployment

Current best practice: ALWAYS validate AI annotations with canonical marker genes.

A tool that confidently labels a cluster as 'NK cells' must be cross-checked:

Do these cells express CD56 (NCAM1), NKG7, GNLY? Do they LACK CD3E?

In published research: report which annotation method was used, at what confidence

threshold, and which markers were used to validate the automated annotation.

7.10 Saving, Sharing, and Reproducibility

PRACTICAL EXERCISE — Saving Results and Generating the Final Report

```
# ── Save the complete Seurat object ──────────────────────────────
saveRDS(seurat_obj, file = 'results/seurat_obj_annotated_final.rds')
# Reload later: seurat_obj <- readRDS('results/seurat_obj_annotated_final.rds')

# ── Export data for collaboration / downstream tools ─────────────
# Export count matrix (raw counts) as CSV:
raw_counts <- GetAssayData(seurat_obj, assay='RNA', layer='counts')
writeMM(raw_counts, 'exports/raw_counts.mtx')
write.csv(data.frame(gene=rownames(raw_counts)), 'exports/genes.csv')
write.csv(data.frame(barcode=colnames(raw_counts)), 'exports/barcodes.csv')

# Export metadata:
write.csv(seurat_obj@meta.data, 'exports/cell_metadata.csv')

# Export UMAP coordinates:
umap_coords <- as.data.frame(Embeddings(seurat_obj, 'umap'))
umap_coords$cell_type <- seurat_obj$cell_type
umap_coords$sample_id <- seurat_obj$sample_id
write.csv(umap_coords, 'exports/umap_coordinates_annotated.csv')

# ── Generate session info for reproducibility ────────────────────
sink('exports/session_info.txt')
sessionInfo()
sink()

# ── Quick summary statistics ─────────────────────────────────────
cat('=== Final Dataset Summary ===\n')
```

```
cat('Total cells:', ncol(seurat_obj), '\n')
cat('Total genes:', nrow(seurat_obj), '\n')
cat('Samples:', length(unique(seurat_obj$sample_id)), '\n')
cat('\nCell type distribution:\n')
print(sort(table(seurat_obj$cell_type), decreasing=TRUE))
cat('\nMedian genes per cell:', median(seurat_obj$nFeature_RNA), '\n')
cat('Median UMIs per cell:', median(seurat_obj$nCount_RNA), '\n')
```

What to Look For / Expected Output:

RDS file: complete Seurat object with all analysis steps. Share with collaborators.

session_info.txt: records exact package versions — essential for reproducibility.

exports/: submit these files to GEO (Gene Expression Omnibus) when publishing.

Cell type distribution table: verify it matches expected biology for the tissue type.

TIP

Performance tips for large datasets (>100,000 cells):

• Use BPCells for memory-efficient sparse matrix storage:

```
remotes::install_github('bnprks/BPCells')
library(BPCells); counts <- open_matrix_dir(dir='/data/bpcells_matrix/')
```

• Use sketched integration for ultra-fast analysis of millions of cells:

```
seurat_obj <- SketchData(seurat_obj, ncells=50000)  # subsample for training
ProjectIntegration(seurat_obj)  # project full dataset
```

• Parallelize SCTransform: use future package

```
library(future); plan('multicore', workers=8)
options(future.globals.maxSize = 8000 * 1024^2)  # 8 GB max
```

• For >500,000 cells: consider Scanpy (Python) instead of Seurat — faster for large data

SELF-CHECK EXERCISES

1. A scRNA-Seq experiment yields 5,000 cells. After QC filtering, 4,200 cells remain. The Seurat UMAP shows 9 clusters. Cluster 7 contains 42 cells and shows high expression of FOXP3, IL2RA, and CTLA4. What cell type is this? Why is it clinically important in tumor immunology?

2. Explain the difference between log-normalization and SCTransform. In which scenario would SCTransform be preferred?

3. A colleague argues that 'UMAP clusters with wide separation represent biologically very different cell types'. How do you respond?

4. A scRNA-Seq dataset comparing tumor-infiltrating T cells from responders vs. non-responders to immunotherapy is analyzed with FindMarkers (Wilcoxon test). Why might a pseudobulk DESeq2 approach give more reliable results?

5. What is pseudotime and how does Monocle3 compute it? In which biological question would you apply trajectory analysis?

CHAPTER SUMMARY

scRNA-Seq profiles gene expression in individual cells, revealing heterogeneity invisible to bulk RNA-Seq.

10x Genomics Chromium uses GEM droplets to barcode individual cells with 16 bp cell barcodes + 12 bp UMIs.

Cell Ranger processes FASTQ → BAM → feature-barcode matrix (sparse UMI count matrix).

Seurat pipeline: QC filter (nFeature, nCount, %mito) → normalize (SCTransform) → HVG selection → PCA → neighbor graph → Louvain clustering → UMAP → annotation.

Cell type annotation uses canonical marker genes (CD3E=T cells, CD19=B cells, CD14=monocytes).

Differential expression: FindMarkers(). Pseudobulk DESeq2 preferred for multi-sample comparisons.

Trajectory analysis (Monocle3) infers developmental ordering (pseudotime) along continuous cell states.

AI annotation tools (scVI, Scimilarity, GPT) require critical validation with known marker genes.

Glossary — Chapter 7

GEM (Gel Bead in Emulsion): A water-in-oil droplet co-encapsulating a single cell, a gel bead carrying barcoded RT primers, and reverse transcriptase. The fundamental unit of 10x Genomics scRNA-Seq.

Cell barcode: A 16 bp oligonucleotide sequence unique to each 10x gel bead, identifying all transcripts originating from a single captured cell.

UMI (Unique Molecular Identifier): A 12 bp random sequence on each 10x RT primer that uniquely tags individual cDNA molecules, enabling PCR duplicate correction and accurate transcript counting.

Seurat: An R package for scRNA-Seq data analysis developed by the Satija Lab, implementing normalization, clustering, UMAP, differential expression, and cell type annotation.

UMAP: Uniform Manifold Approximation and Projection — a non-linear dimensionality reduction method for 2D visualization of high-dimensional single-cell data.

Louvain algorithm: A graph community detection algorithm used by Seurat to identify clusters of transcriptionally similar cells in the K-nearest-neighbor graph.

SCTransform: Variance-stabilizing normalization for scRNA-Seq using regularized negative binomial regression, replacing log-normalization in modern Seurat workflows.

Pseudotime: A computational ordering of cells along an inferred developmental trajectory, representing their progression through a biological process without a chronological time axis.

Monocle3: An R package implementing trajectory analysis and pseudotime inference for scRNA-Seq data using a principal graph approach.

Doublet: A GEM capturing two cells instead of one, producing a cell barcode with a mixture of two transcriptomes. Detected and removed by DoubletFinder or Scrublet.

Chapter 8: Metataxonomics — Sequencing the Human Microbiome

CLINICAL CASE — Problem-Based Learning

Scenario: A patient with Crohn's disease completes a 14-day course of broad-spectrum antibiotics (ciprofloxacin + metronidazole) for bacterial overgrowth. Stool samples are collected before and after treatment. 16S rRNA amplicon sequencing (V3-V4 region, Illumina MiSeq) is performed. Before treatment: Shannon diversity index = 4.2; 8 phyla detected; Bacteroidetes 45%, Firmicutes 38%, Proteobacteria 10%. After treatment: Shannon diversity = 1.8; 4 phyla detected; Proteobacteria 72%, Bacteroidetes 18%, Firmicutes 8%. Three weeks later, the patient develops Clostridioides difficile infection.

? Interpret the microbiome changes caused by antibiotics. What is dysbiosis? How does the post-antibiotic microbiome profile predispose to C. difficile? What metrics quantify the change? Could you have predicted C. difficile risk from the post-treatment 16S data?

Learning Objectives: After reading this chapter, you will be able to (1) explain the principle and limitations of 16S rRNA amplicon sequencing; (2) describe the DADA2 algorithm and the concept of ASVs vs. OTUs; (3) perform and interpret the QIIME2 analysis pipeline; (4) calculate and interpret alpha and beta diversity metrics; (5) critically evaluate the challenges of 16S sequencing with Oxford Nanopore; (6) describe shotgun metagenomics and its advantages over amplicon sequencing.

8.1 Introduction to the Human Microbiome

The human body harbors an estimated 38 trillion microbial cells — roughly equal to the number of human cells — comprising bacteria, archaea, fungi, protists, and viruses. The gut microbiome alone contains >1,000 bacterial species representing ~150-fold more microbial genes than the human genome. This complex community performs critical functions: digestion of dietary fibers and production of short-chain fatty acids (SCFAs) such as butyrate (gut epithelial fuel); vitamin synthesis (B12, K2, folate); education and regulation of the immune system; protection against pathogen colonization (colonization resistance).

Dysbiosis — a disruption of the normal microbiome composition — has been associated with inflammatory bowel disease (Crohn's disease, ulcerative colitis), colorectal cancer, metabolic syndrome and type 2 diabetes, cardiovascular disease, Clostridioides difficile infection, and response to cancer immunotherapy (checkpoint inhibitor efficacy correlates with microbiome composition). These associations make microbiome profiling a clinically relevant diagnostic and research tool.

8.2 16S rRNA Amplicon Sequencing: Principles

The 16S ribosomal RNA (rRNA) gene is present in all bacteria and archaea. It contains nine hypervariable regions (V1–V9) interspersed with conserved regions. The conserved regions serve as primer binding sites for PCR amplification; the hypervariable regions contain sequence diversity sufficient to distinguish bacterial taxa at the genus and sometimes species level.

8.2.1 Amplicon Sequencing Workflow

18. DNA extraction from the biological sample (stool, saliva, swab, biopsy). Bead-beating-based mechanical disruption is preferred for efficient lysis of Gram-positive bacteria.

19. PCR amplification of the target hypervariable region using universal 16S primers. Common choices: V3-V4 (primers 341F/806R, ~460 bp amplicon — widely used, good genus-level resolution); V1-V2 (higher resolution for Firmicutes/Bacteroidetes distinction); V4 (primers 515F/806R, ~290 bp — commonly used, excellent with QIIME2/DADA2).
20. Library preparation and Illumina sequencing (MiSeq 2×250 or 2×300 bp paired-end for V3-V4; NextSeq 2×150 bp for V4).
21. Bioinformatics: denoising and taxonomic classification (DADA2/QIIME2).

8.2.2 OTUs vs. ASVs: The DADA2 Advantage

Historically, 16S analysis grouped sequences into Operational Taxonomic Units (OTUs) by clustering reads at 97% sequence identity — a computational definition of 'species'. OTU clustering discards sequence diversity below the clustering threshold, conflating multiple distinct taxa into single OTUs, and is unable to resolve sequences that differ by 1–2 bases.

DADA2 (Divisive Amplicon Denoising Algorithm 2) represents a paradigm shift. Instead of clustering, DADA2 applies a statistical error model to each sequencing run to distinguish true biological sequence variants from sequencing errors. The output is Amplicon Sequence Variants (ASVs) — exact sequences representing real biological taxa, resolved to single-nucleotide precision. Advantages of ASVs: single-nucleotide resolution (can distinguish taxa differing by 1 bp); reproducible across studies (the same ASV sequence represents the same taxon in any dataset); can be compared across studies without re-clustering; better concordance with cultured reference isolates.

IMPORTANT NOTE

ASVs vs OTUs: The field has largely transitioned to ASVs (DADA2, Deblur).

If you encounter OTU-based analyses in older literature (pre-2016), be aware:

OTU clusters may contain multiple genuine taxa; cross-study comparisons require re-clustering.

DADA2 ASVs are directly comparable across studies if the same primer pair and region are used.

8.3 The QIIME2 Framework

QIIME2 (Quantitative Insights Into Microbial Ecology, version 2) is the standard bioinformatics platform for amplicon microbiome analysis. QIIME2 uses a artifact-based workflow: all inputs and outputs are QIIME2 Artifacts (.qza files) or Visualizations (.qzv files) that encode both the data and its complete provenance (every analytical step, parameter, and version number). This provenance tracking enables full reproducibility. Visualizations (.qzv) are rendered interactively in QIIME2View (view.qiime2.org).

The QIIME2 DADA2 pipeline steps:

22. Import: Raw FASTQ files are imported into QIIME2 artifact format using qiime tools import.
23. Demultiplexing: If samples were multiplexed with inline barcodes (not Illumina dual-index), qiime demux emp-paired separates reads by barcode. For pre-demultiplexed data (standard Illumina dual-index), reads are imported directly.
24. Quality visualization: qiime demux summarize generates interactive quality plots to guide DADA2 truncation parameters.
25. DADA2 denoising: qiime dada2 denoise-paired performs: paired-end read quality trimming at user-defined truncation positions; chimera detection and removal; error model learning and correction; ASV generation and UMI-like deduplication by exact sequence. Outputs: feature table (ASV × sample count matrix) and representative sequences (FASTA of all ASVs).

26. Taxonomic classification: qiime feature-classifier classify-sklearn classifies ASVs against a reference database (SILVA 138 for bacteria/archaea; Greengenes2 as an alternative) using a naive Bayes classifier trained on the specific primer pair.
27. Phylogenetic tree: qiime phylogeny align-to-tree-mafft-fasttree generates a rooted phylogenetic tree of ASVs for UniFrac diversity calculations.
28. Diversity analysis: qiime diversity core-metrics-phylogenetic computes alpha and beta diversity metrics.

8.4 Diversity Metrics: Alpha and Beta Diversity

8.4.1 Alpha Diversity: Within-Sample Diversity

Alpha diversity measures the diversity within a single sample. It answers: 'How diverse is this microbial community?' Key metrics include:

Metric	Description and Interpretation
Observed Features	Total number of distinct ASVs/OTUs in the sample. Richness metric; affected by sequencing depth.
Shannon Diversity Index (H')	Considers both richness AND evenness. H' = -Σ(pi × ln(pi)), where pi is the relative abundance of each ASV. Range: 0 (single species) to ln(S) (maximum when all S species equally abundant). Typical healthy gut: H' ≈ 3.5–5.0.
Faith's Phylogenetic Diversity (PD)	Sum of branch lengths in the phylogenetic tree spanning all ASVs in the sample. Incorporates evolutionary relationships, not just identity.
Evenness (Pielou's J)	J' = H'/ln(S). Range 0–1. High evenness (J'≈1): all species equally abundant. Low evenness: one or few species dominate (typical after antibiotics).

Alpha diversity rarefaction curves: Alpha diversity metrics are sensitive to sequencing depth — more reads almost always reveal more rare taxa. Rarefaction analysis (qiime diversity alpha-rarefaction) plots alpha diversity as a function of sampling depth (number of reads), revealing at what depth diversity metrics plateau (are 'saturated'). Samples not reaching saturation may be undersequenced. For fair cross-sample comparison, all samples are rarefied to the same sequencing depth (subsampling reads), necessarily discarding data from deeper-sequenced samples — a recognized limitation of rarefaction.

8.4.2 Beta Diversity: Between-Sample Diversity

Beta diversity measures the dissimilarity between samples. It answers: 'How different are these microbial communities from each other?' Key metrics include:

Metric	Description and Interpretation
Bray-Curtis dissimilarity	Abundance-based distance. Ranges 0 (identical communities) to 1 (completely different). Sensitive to dominant taxa.
Jaccard distance	Presence/absence-based distance. Ignores abundance. 1 - (shared species / union of species). Measures compositional overlap.
UniFrac (unweighted)	Phylogenetic beta diversity based on presence/absence. Fraction of unique branch length in the phylogenetic tree.

Metric	Description and Interpretation
UniFrac (weighted)	Phylogenetic beta diversity weighted by ASV abundances. Combines phylogenetic distance with relative abundance information.

PCoA (Principal Coordinates Analysis): Beta diversity distance matrices are visualized using PCoA, which projects high-dimensional sample-to-sample distances into 2D or 3D. In a PCoA plot, samples that cluster together are microbiomically similar; samples that are distant are dissimilar. PERMANOVA (Permutational Multivariate ANOVA) tests statistical significance of grouping in PCoA space: qiime diversity beta-group-significance with the --p-pairwise option reports p-values for pairwise group comparisons.

Environment Setup and Data Import

PRACTICAL EXERCISE — Setting Up QIIME2 and Importing Data

```
# Activate QIIME2 conda environment (or load HPC module):

$ conda activate qiime2-amplicon-2024.10
# or on HPC:
$ module load QIIME2/2024.10

# Check version:
$ qiime info

# Prepare a manifest file (for pre-demultiplexed paired-end data):
# Create manifest.tsv with columns: sample-id, forward-absolute-filepath, reverse-absolute-filepath
$ cat manifest.tsv
sample-id  forward-absolute-filepath  reverse-absolute-filepath
ctrl_01  /data/ctrl_01_R1.fastq.gz  /data/ctrl_01_R2.fastq.gz
ctrl_02  /data/ctrl_02_R1.fastq.gz  /data/ctrl_02_R2.fastq.gz
ibd_01   /data/ibd_01_R1.fastq.gz   /data/ibd_01_R2.fastq.gz

# Import paired-end FASTQ data into QIIME2 artifact format (.qza):
$ qiime tools import \
    --type 'SampleData[PairedEndSequencesWithQuality]' \
    --input-path manifest.tsv \
    --input-format PairedEndFastqManifestPhred33V2 \
    --output-path demux.qza

# Visualize import results to check read counts per sample:
$ qiime demux summarize \
    --i-data demux.qza \
    --o-visualization demux_summary.qzv

# Open in QIIME2 View:
$ qiime tools view demux_summary.qzv
# Or: drag-and-drop the .qzv file at https://view.qiime2.org
```

Expected Output / What to Look For:

demux_summary.qzv: per-sample read counts + interactive quality plots.
Use the quality plots to decide DADA2 truncation positions (where quality drops below Q25).
Aim for: minimum 5,000 reads per sample. Samples with <1,000 reads may be excluded.

DADA2 Denoising — The Core Step

KEY CONCEPT

DADA2 performs in QIIME2: quality filtering, denoising, chimera removal, and ASV generation.
Key parameters to set (based on quality visualization from demux_summary.qzv):
--p-trunc-len-f: truncate Read 1 at this position (where Q drops below 25)
--p-trunc-len-r: truncate Read 2 at this position (Read 2 usually shorter quality)
--p-trim-left-f/r: trim N bases from 5' end (primer sequences if not pre-removed)
Critical: trunc positions must leave enough overlap for paired reads to merge.
For V3-V4 amplicons (~460 bp): trunc_f + trunc_r must exceed ~460 + 20 = 480 bp.

PRACTICAL EXERCISE — DADA2 Denoising — Full Command

```
# DADA2 denoising (this is the computationally intensive core step):
$ qiime dada2 denoise-paired \
    --i-demultiplexed-seqs demux.qza \
    --p-trunc-len-f 230 \
    --p-trunc-len-r 200 \
    --p-trim-left-f 19 \
    --p-trim-left-r 20 \
    --p-max-ee-f 2.0 \
    --p-max-ee-r 2.0 \
    --p-n-threads 8 \
    --o-table feature_table.qza \
    --o-representative-sequences rep_seqs.qza \
    --o-denoising-stats dada2_stats.qza \
    --verbose

# Parameter explanation:
# --p-trunc-len-f 230  = truncate R1 at position 230 (chosen from quality plot)
# --p-trunc-len-r 200  = truncate R2 at position 200
# --p-trim-left-f 19   = remove 19 bp from R1 5' end (V3 primer 341F = 19 bp)
# --p-trim-left-r 20   = remove 20 bp from R2 5' end (V4 primer 806R = 20 bp)
# --p-max-ee-f 2.0     = maximum expected errors in R1 (per-read quality filter)
# --p-max-ee-r 2.0     = maximum expected errors in R2
# --p-n-threads 8      = parallelization

# Visualize denoising statistics:
$ qiime metadata tabulate \
```

```
    --m-input-file dada2_stats.qza \
    --o-visualization dada2_stats.qzv
$ qiime tools view dada2_stats.qzv

# Visualize feature table (ASV × sample matrix):
$ qiime feature-table summarize \
    --i-table feature_table.qza \
    --m-sample-metadata-file metadata.tsv \
    --o-visualization feature_table_summary.qzv
$ qiime tools view feature_table_summary.qzv
```

✔ Expected Output / What to Look For:

dada2_stats.qzv: track reads through each step: input → filtered → denoised → merged → non-chimeric.

Acceptable: >70% reads passing each step. If <50% merge: increase trunc lengths or check primer trimming.

feature_table_summary.qzv: total reads per sample, total ASVs, interactive rarefaction preview.

Final ASV count (typical V3-V4 human gut): 200-2,000 ASVs per dataset.

Taxonomic Classification

PRACTICAL EXERCISE — Classifying ASVs with SILVA and QIIME2 Naive Bayes

```
# Download pre-trained SILVA 138 classifier for V3-V4 (341F/806R primers):
$ wget https://data.qiime2.org/classifiers/sklearn-1.4.2/silva/
      silva-138-99-seqs-341-806.qza

# OR train your own classifier on your primer set (recommended for non-standard primers):
$ qiime feature-classifier fit-classifier-naive-bayes \
    --i-reference-reads silva-138-99-seqs.qza \
    --i-reference-taxonomy silva-138-99-tax.qza \
    --o-classifier silva_138_99_classifier.qza

# Classify ASVs using the trained classifier:
$ qiime feature-classifier classify-sklearn \
    --i-classifier silva-138-99-seqs-341-806.qza \
    --i-reads rep_seqs.qza \
    --p-n-jobs -1 \
    --o-classification taxonomy.qza

# Visualize taxonomy table:
$ qiime metadata tabulate \
    --m-input-file taxonomy.qza \
    --o-visualization taxonomy.qzv

# Generate interactive taxonomy bar plots:
$ qiime taxa barplot \
    --i-table feature_table.qza \
    --i-taxonomy taxonomy.qza \
    --m-metadata-file metadata.tsv \
    --o-visualization taxa_barplots.qzv
$ qiime tools view taxa_barplots.qzv
```

```
# Filter out chloroplast, mitochondria, and unclassified reads:
$ qiime taxa filter-table \
    --i-table feature_table.qza \
    --i-taxonomy taxonomy.qza \
    --p-exclude mitochondria,chloroplast \
    --o-filtered-table feature_table_filtered.qza
```

✔ Expected Output / What to Look For:

taxonomy.qzv: shows each ASV with its classification and confidence score.

taxa_barplots.qzv: stacked bar chart colored by taxon — compare phylum composition across samples.

Good classification rate: >90% of ASVs classified at phylum level; >70% at genus level.

Always filter chloroplast/mitochondria — they amplify with 16S primers but are eukaryotic.

Phylogenetic Tree Construction

PRACTICAL EXERCISE — Building a Phylogenetic Tree for UniFrac

```
# Build rooted phylogenetic tree (required for UniFrac diversity metrics):
$ qiime phylogeny align-to-tree-mafft-fasttree \
    --i-sequences rep_seqs.qza \
    --o-alignment aligned_rep_seqs.qza \
    --o-masked-alignment masked_aligned_rep_seqs.qza \
    --o-tree unrooted_tree.qza \
    --o-rooted-tree rooted_tree.qza \
    --p-n-threads 8

# This command pipeline in one step:
# MAFFT: multiple sequence alignment of all ASVs
# Masking: remove highly variable alignment positions
# FastTree: build maximum-likelihood phylogenetic tree
# Rooting: midpoint root the tree

# Visualize the tree (iTOL format export):
$ qiime tools export --input-path rooted_tree.qza --output-path tree_export/
# Upload tree_export/tree.nwk to https://itol.embl.de for interactive visualization
```

✔ Expected Output / What to Look For:

rooted_tree.qza is required as input for qiime diversity core-metrics-phylogenetic.

FastTree typically takes 2-15 minutes for a dataset with <2,000 ASVs.

Inspect the Newick tree file: it should contain all your ASV sequences as leaf nodes.

Diversity Analysis — Alpha and Beta

KEY CONCEPT

Alpha diversity = within-sample diversity (richness + evenness):

- Observed features: total number of distinct ASVs

- Shannon (H'): H' = -Σ(pi × ln(pi)). Range: 0 (one species) to ln(S) (maximum evenness)
- Faith's PD: phylogenetic diversity — sum of branch lengths in the phylogenetic tree
- Evenness: J' = H' / ln(S). Range 0-1.

Beta diversity = between-sample dissimilarity:

- Bray-Curtis: abundance-based. Range 0 (identical) to 1 (completely different).
- Jaccard: presence/absence only.
- Unweighted UniFrac: phylogenetic, presence/absence.
- Weighted UniFrac: phylogenetic, abundance-weighted. Most informative for clinical studies.

PRACTICAL EXERCISE — Computing Diversity Metrics with QIIME2

```
# Determine rarefaction depth (from feature_table_summary.qzv):
# Choose depth that retains most samples but excludes undersequenced ones.
# Typical human gut: rarefy to minimum 5,000-10,000 reads per sample.

# Compute all core diversity metrics at once:
$ qiime diversity core-metrics-phylogenetic \
    --i-phylogeny rooted_tree.qza \
    --i-table feature_table_filtered.qza \
    --p-sampling-depth 10000 \
    --m-metadata-file metadata.tsv \
    --output-dir core_diversity_results/

# Outputs generated:
# rarefied_table.qza        — table rarefied to --p-sampling-depth
# faith_pd_vector.qza       — Faith's PD per sample
# observed_features_vector.qza
# shannon_vector.qza        — Shannon diversity per sample
# evenness_vector.qza       — Pielou's J per sample
# bray_curtis_distance_matrix.qza
# jaccard_distance_matrix.qza
# unweighted_unifrac_distance_matrix.qza
# weighted_unifrac_distance_matrix.qza
# *_emperor.qzv             — interactive PCoA plots for each beta metric

# Test alpha diversity significance between groups:
$ qiime diversity alpha-group-significance \
    --i-alpha-diversity core_diversity_results/shannon_vector.qza \
    --m-metadata-file metadata.tsv \
    --o-visualization shannon_significance.qzv
$ qiime tools view shannon_significance.qzv

# Test beta diversity significance (PERMANOVA):
$ qiime diversity beta-group-significance \
    --i-distance-matrix core_diversity_results/weighted_unifrac_distance_matrix.qza \
    --m-metadata-file metadata.tsv \
```

```
    --m-metadata-column disease_status \
    --p-method permanova \
    --p-pairwise \
    --o-visualization weighted_unifrac_permanova.qzv
$ qiime tools view weighted_unifrac_permanova.qzv

# Alpha rarefaction curve:
$ qiime diversity alpha-rarefaction \
    --i-table feature_table_filtered.qza \
    --i-phylogeny rooted_tree.qza \
    --p-max-depth 15000 \
    --m-metadata-file metadata.tsv \
    --o-visualization alpha_rarefaction.qzv
```

✔ Expected Output / What to Look For:

shannon_significance.qzv: Kruskal-Wallis test p-value for Shannon H' between groups.

weighted_unifrac_permanova.qzv: PERMANOVA R^2 and p-value (999 permutations).

PCoA emperor plots: interactive 3D ordination. Color by metadata variables to identify patterns.

alpha_rarefaction.qzv: curves should plateau before the chosen rarefaction depth.

Differential Abundance Analysis

PRACTICAL EXERCISE — ANCOM-BC and Differential Abundance

```
# ANCOM-BC: Analysis of Compositions of Microbiomes with Bias Correction

# More statistically robust than simple Wilcoxon on relative abundances

$ qiime composition ancombc \
    --i-table feature_table_filtered.qza \
    --m-metadata-file metadata.tsv \
    --p-formula disease_status \
    --o-differentials ancombc_differentials.qza

$ qiime composition da-barplot \
    --i-data ancombc_differentials.qza \
    --p-significance-threshold 0.05 \
    --o-visualization ancombc_barplot.qzv

# Export feature table for R analysis (DESeq2, MaAsLin2):
$ qiime tools export --input-path feature_table_filtered.qza --output-path exported_table/
# This creates: exported_table/feature-table.biom

# Convert BIOM to TSV for R:
$ biom convert -i exported_table/feature-table.biom \
    -o exported_table/feature_table.tsv --to-tsv

# In R: DESeq2-based analysis (recommended for RNA-Seq-like count data):
# library(DESeq2); library(phyloseq)
```

```
# ps <- import_biom('exported_table/feature-table.biom')
# dds <- phyloseq_to_deseq2(ps, ~ disease_status)
# dds <- DESeq(dds); results(dds, contrast=c('disease_status','IBD','Control'))
```

✔ **Expected Output / What to Look For:**

ANCOM-BC output: log2 fold-change, standard error, W-statistic, and adjusted p-value per ASV.

Significant ASVs: |log2FC| > 1 AND q-value < 0.05 (after BH correction).

Verify differential ASVs taxonomically — are they biologically plausible (Faecalibacterium loss in IBD)?

8.5 Challenges of ONT 16S Sequencing

Oxford Nanopore sequencing of 16S amplicons offers potential advantages: full-length 16S amplicon sequencing (all 9 hypervariable regions simultaneously), enabling species-level resolution beyond the V3-V4 region; real-time sequencing for rapid microbiome reporting. However, ONT 16S sequencing presents significant bioinformatics challenges:

- **Higher error rates: Even with R10.4.1 chemistry, systematic errors in 16S homopolymeric regions can generate false taxonomic diversity, creating 'ghost ASVs' that are artifactual.**
- **DADA2 incompatibility: DADA2's error model is trained on Illumina-specific error profiles and does not perform well on ONT data. Tools like NanoCLUST and EVEN-tools were developed specifically for ONT 16S data.**
- **Chimera detection challenges: Chimeric sequences (PCR artifacts joining two partial amplicons) are more difficult to detect in ONT data due to the overlapping error and chimera signals.**
- **Limited validated databases: ONT 16S classification tools have fewer validated reference databases compared to Illumina-compatible tools.**

8.6 Shotgun Metagenomics: Beyond 16S

Amplicon 16S sequencing is limited to bacteria and archaea, and only resolves taxonomy — not gene function. Shotgun metagenomics sequences all DNA in a sample without amplification, providing: species-level and sometimes strain-level resolution; functional profiling (metabolic pathways, resistance genes, virulence factors); detection of fungi, protists, and viruses; resistome characterization (antimicrobial resistance genes).

Key shotgun metagenomics tools: MetaPhlAn4 (taxonomic profiling by mapping reads to marker genes); HUMAnN3 (functional profiling — metabolic pathway abundance); KrakenUniq (taxonomic classification with unique k-mer counting to reduce false positives); MEGAHIT/metaSPAdes (metagenomic assembly); CARD/ResFinder (resistome analysis). Shotgun metagenomics requires higher sequencing depth (~10 Gb per sample for human gut versus ~50 Mb for 16S), more computational resources, and more complex host DNA depletion strategies.

Clinical relevance of the resistome: The resistome is the collection of antimicrobial resistance genes in a metagenome. Shotgun metagenomics can identify horizontal gene transfer of resistance genes between taxa, monitor resistance dynamics in ICU patients or outbreak settings, and characterize resistance profiles without culture. This has direct implications for antibiotic stewardship and infection control.

Host Read Depletion

PRACTICAL EXERCISE — Removing Human Host Reads

```
# Method 1: Computational depletion (align to human genome, keep unmapped reads)


# Align shotgun reads to GRCh38, output only UNMAPPED reads:
$ bwa-mem2 mem -t 16 GRCh38.fa R1.fastq.gz R2.fastq.gz \
    | samtools view -b -f 12 -F 256 \
    | samtools sort -n \
    | samtools fastq -1 host_depleted_R1.fastq.gz -2 host_depleted_R2.fastq.gz -

# samtools view flags:
# -f 12    = keep reads where BOTH mates are unmapped (FLAG 0x4 + 0x8 = 12)
# -F 256   = exclude secondary alignments

# Alternative: Bowtie2 (faster for host depletion):
$ bowtie2 -x GRCh38_index -1 R1.fastq.gz -2 R2.fastq.gz \
    --un-conc-gz host_depleted_R%.fastq.gz \
    -p 16 --very-fast \
    -S /dev/null
# --un-conc-gz: output read pairs where BOTH mates failed to align (unmapped =
microbial)

# Check depletion efficiency:
$ echo 'Total reads:' && zcat R1.fastq.gz | wc -l | awk '{print $1/4}'
$ echo 'After depletion:' && zcat host_depleted_R1.fastq.gz | wc -l | awk '{print $1/4}'
$ echo 'Depletion rate:' # = (total - after) / total * 100
```

Expected Output / What to Look For:

For human stool samples: expect 20-80% human DNA (depends on mucosal content).

For BAL/respiratory: may be >95% human DNA — host depletion critical.

After depletion: remaining reads are predominantly microbial + uncharacterized sequences.

Taxonomic Profiling with MetaPhlAn4

KEY CONCEPT

MetaPhlAn4 (Metagenomic Phylogenetic Analysis, version 4): aligns reads to a database of

species-specific marker genes (~1.1 million markers, ~99,000 reference genomes).

Output: relative abundance of each species (expressed as % of total classified reads).

Advantages: fast, memory-efficient, no assembly required.

Limitations: can only detect species with representation in the marker gene database.

Novel organisms with <70% identity to known sequences will be missed.

PRACTICAL EXERCISE — Running MetaPhlAn4

```
# Download MetaPhlAn4 database (first time only, ~15 GB):

$ metaphlan --install --index mpa_vJan21_CHOCOPhlAnSGB_202103 --bowtie2db
metaphlan_db/

# Run MetaPhlAn4 on a single sample:
$ metaphlan \
    host_depleted_R1.fastq.gz,host_depleted_R2.fastq.gz \
    --input_type fastq \
    --bowtie2db metaphlan_db/ \
    --index mpa_vJan21_CHOCOPhlAnSGB_202103 \
    --bowtie2out sample01.bowtie2.bz2 \
    --nproc 8 \
    --output_file sample01_metaphlan_profile.txt \
    --add_viruses

# View the output:
$ head -20 sample01_metaphlan_profile.txt
# Output columns: clade_name | clade_NCBI_taxid | relative_abundance | coverage

# Merge multiple sample profiles into one table:
$ merge_metaphlan_tables.py sample01_metaphlan_profile.txt
sample02_metaphlan_profile.txt \
    > all_samples_merged_metaphlan.txt

# Filter to species level:
$ grep -E '(s__)' all_samples_merged_metaphlan.txt | grep -v 't__' > species_profiles.txt

# Generate heatmap in R:
# library(pheatmap)
# df <- read.table('species_profiles.txt', header=T, sep='\t', row.names=1)
# pheatmap(df, scale='row', clustering_method='ward.D2')
```

✔ Expected Output / What to Look For:

Output: percentage abundance for each clade (kingdom → species → strain level).

Typical healthy gut: Bacteroides, Prevotella, Ruminococcus, Faecalibacterium at species level.

Dysbiotic samples (antibiotic treated): Enterobacteriaceae, Candida expansion.

Check classified fraction: unclassified reads >50% may indicate novel species or poor sample quality.

Functional Profiling with HUMAnN3

📌 KEY CONCEPT

HUMAnN3 (HMP Unified Metabolic Analysis Network, version 3) quantifies the functional

content of a metagenome: metabolic pathways, gene families (UniRef90), and enzyme functions.

HUMAnN3 three-step approach:

1. Species detection (using MetaPhlAn): identify present species

2. Translated search: align unclassified reads to ChocoPhlAn + UniRef90 protein databases

3. Pathway quantification: compute pathway abundances and coverage from gene family abundances

Outputs (in reads-per-kilobase, RPK, or copies-per-million, CPM):

- Gene families table (genefamilies.tsv): UniRef90 gene family abundances
- Pathway abundances (pathabundance.tsv): MetaCyc pathway abundances
- Pathway coverage (pathcoverage.tsv): completeness of each pathway

PRACTICAL EXERCISE — Running HUMAnN3

```
# Install and download databases (one-time, ~50 GB total):
$ humann_databases --download chocophlan full /ref_data/
$ humann_databases --download uniref uniref90_diamond /ref_data/
$ humann_databases --download utility_mapping full /ref_data/

# Concatenate R1 and R2 into one file (HUMAnN3 input for paired-end):
$ cat host_depleted_R1.fastq.gz host_depleted_R2.fastq.gz > sample01_concat.fastq.gz

# Run HUMAnN3 (computationally intensive: allow 4-8 hours for complex samples):
$ humann \
    --input sample01_concat.fastq.gz \
    --output humann3_output/sample01/ \
    --nucleotide-database /ref_data/chocophlan/ \
    --protein-database /ref_data/uniref/ \
    --taxonomic-profile sample01_metaphlan_profile.txt \
    --threads 8 \
    --verbose

# Re-normalize to copies per million (CPM) for cross-sample comparison:
$ humann_renorm_table \
    --input humann3_output/sample01/sample01_pathabundance.tsv \
    --output sample01_pathabundance_cpm.tsv \
    --units cpm

# Join all sample pathway abundance tables:
$ humann_join_tables \
    --input ./humann3_output/ \
    --file_name pathabundance_cpm \
    --output all_samples_pathabundance.tsv

# Split stratified output (total + per-species contributions):
$ humann_split_stratified_table \
    --input all_samples_pathabundance.tsv \
    --output humann3_split/
# Creates: *_unstratified.tsv (pathway totals) and *_stratified.tsv (per-species contributions)
```

Expected Output / What to Look For:

pathabundance_cpm.tsv: ~500-1,500 MetaCyc pathways with CPM values.

Compare butyrate biosynthesis pathway (PWY-5676) between IBD vs. control samples.

Check: all samples should have similar total CPM after normalization (~1,000,000).

Resistome Analysis — AMR Genes

PRACTICAL EXERCISE — Identifying Antimicrobial Resistance Genes with CARD/RGI

```
# Install RGI (Resistance Gene Identifier) from CARD:
# conda install -c bioconda rgi

# Download CARD database (update regularly — resistance genes change):
$ rgi load --card_json card.json --local

# Option 1: Align reads directly to CARD (fast, sensitive):
$ rgi bwt \
    --read_one host_depleted_R1.fastq.gz \
    --read_two host_depleted_R2.fastq.gz \
    --output_file sample01_rgi_bwt \
    --threads 8 \
    --local --clean

# Option 2: From assembled contigs (more accurate, requires assembly):
# First assemble: (see section 6.5)
$ rgi main \
    --input_sequence assembled_contigs.fa \
    --output_file sample01_rgi_assembly \
    --input_type contig \
    --local --clean \
    --num_threads 8

# View tabular results:
$ cat sample01_rgi_bwt.gene_mapping_data.txt | head -20

# Alternative: ResFinder (web or command line) for clinical settings:
$ python run_resfinder.py \
    -ifa assembled_contigs.fa \
    -o resfinder_output/ \
    -s 'Other' \
    -l 0.6 -t 0.9 \
    --acquired \
    --point
# -l 0.6: minimum coverage 60% | -t 0.9: minimum identity 90%
# --acquired: search acquired resistance genes
# --point: include point mutations in chromosomal genes
```

Expected Output / What to Look For:

RGI output: AMR gene name, drug class, resistance mechanism, percent identity, coverage.

Key resistance genes to flag: blaKPC (carbapenem), blaNDM (carbapenem), mcr (colistin),

vanA/B (vancomycin), mecA (methicillin-MRSA).

Cross-check with clinical culture/sensitivity data if available.

Metagenomic Assembly with MEGAHIT

PRACTICAL EXERCISE — De Novo Assembly of Metagenomic Reads

```
# MEGAHIT: ultra-fast memory-efficient assembler for metagenomes:

$ megahit \
    -1 host_depleted_R1.fastq.gz \
    -2 host_depleted_R2.fastq.gz \
    -o megahit_output/ \
    --k-min 21 --k-max 141 --k-step 12 \
    -t 16 \
    --min-contig-len 1000

# --k-min/max/step: range of k-mer sizes (multi-k approach improves sensitivity)
# --min-contig-len 1000: only output contigs >=1000 bp (shorter contigs are unreliable)

# Check assembly statistics:
$ quast.py megahit_output/final.contigs.fa -o quast_report/ --threads 4
# Key metrics: N50, total assembled bp, number of contigs, largest contig

# Taxonomically classify assembled contigs (Kraken2):
$ kraken2 \
    --db /ref_data/kraken2_standard_db/ \
    --threads 8 \
    --classified-out classified_contigs.fa \
    --report kraken2_report.txt \
    --output kraken2_output.txt \
    megahit_output/final.contigs.fa

# Visualize Kraken2 report with Bracken (abundance re-estimation at species level):
$ bracken \
    -d /ref_data/kraken2_standard_db/ \
    -i kraken2_report.txt \
    -o bracken_species.txt \
    -l S \
    -t 10
# -l S: report at Species level | -t 10: minimum reads to report
```

Expected Output / What to Look For:

MEGAHIT N50: typical gut metagenome N50 = 1-10 kb; complex communities have lower N50.

QUAST report: total assembly length should be 2-10x the input sequencing depth.

Kraken2 classified fraction: >60% classified is good for gut; clinical samples may vary widely.

SELF-CHECK EXERCISES

1. A healthy volunteer gut microbiome shows Shannon diversity H' = 4.5. After 7 days of broad-spectrum antibiotics, H' = 1.9. Interpret this change in biological terms. Is this clinically significant?

2. Explain the difference between alpha and beta diversity. Which metric would you use to compare microbiome composition between 10 patients with IBD and 10 healthy controls?

3. Why are ASVs (DADA2) superior to OTUs for cross-study comparisons? What is the key biological assumption underlying ASV generation?

4. A QIIME2 DADA2 analysis retains 68% of paired-end reads after denoising. The 32% of reads that were discarded — what are the most likely reasons for their removal?

5. A researcher wants to characterize both the taxonomy AND the antibiotic resistance gene content of ICU patient gut microbiomes. Would you recommend 16S amplicon sequencing or shotgun metagenomics? Justify your answer.

CHAPTER SUMMARY

The gut microbiome contains >1,000 bacterial species performing digestion, immunity, and colonization resistance functions.

16S rRNA amplicon sequencing uses conserved primer binding sites flanking hypervariable regions (V3-V4, V4) for culture-independent microbiome profiling.

DADA2 generates ASVs (single-nucleotide resolution) by learning run-specific error models — superior to OTU clustering.

QIIME2 provides a provenance-tracking artifact-based framework for reproducible amplicon analysis.

Alpha diversity (within-sample): Shannon H', Observed features, Faith's PD, Evenness.

Beta diversity (between-sample): Bray-Curtis, Jaccard, UniFrac. Visualized by PCoA; tested by PERMANOVA.

ONT 16S faces error profile challenges — use NanoCLUST/EVEN-tools, not standard DADA2.

Shotgun metagenomics provides functional profiling and resistome characterization beyond 16S taxonomy.

Glossary — Chapter 8

16S rRNA gene: The gene encoding the small subunit ribosomal RNA in prokaryotes. Contains conserved primer binding regions and hypervariable regions (V1–V9) used for taxonomic identification.

ASV (Amplicon Sequence Variant): An exact sequence variant generated by DADA2's error-corrected denoising of amplicon reads. Provides single-nucleotide resolution, replacing OTU-based clustering.

OTU (Operational Taxonomic Unit): A cluster of sequences grouped at 97% identity, historically used as a proxy for bacterial 'species' in 16S analysis. Largely replaced by ASVs.

DADA2: Divisive Amplicon Denoising Algorithm 2 — a statistical method for learning sequencing error models and correcting amplicon reads to produce exact ASVs.

QIIME2: Quantitative Insights Into Microbial Ecology version 2 — a bioinformatics platform for amplicon and metagenomics analysis with artifact-based provenance tracking.

Shannon diversity (H'): An alpha diversity metric measuring community richness and evenness. H' = -Σ(pi × ln(pi)). Higher values indicate greater diversity.

Alpha diversity: A measure of biodiversity within a single sample (richness and evenness of the microbial community in one sample).

Beta diversity: A measure of dissimilarity in community composition between two or more samples.

UniFrac: A phylogeny-based beta diversity metric measuring the fraction of unique evolutionary history shared between microbial communities.

PCoA: Principal Coordinates Analysis — an ordination method that visualizes high-dimensional beta diversity distance matrices in 2D/3D scatter plots.

Dysbiosis: A disruption of the normal composition, diversity, and function of the microbiome, associated with disease states.

Resistome: The collection of all antimicrobial resistance genes in a metagenome or genome.

PERMANOVA: Permutational Multivariate ANOVA — a non-parametric statistical test for significance of group differences in beta diversity distance matrices.

Chapter 9: Epigenomics — ChIP-Seq, ATAC-Seq, and DNA Methylation

CLINICAL SCENARIO
A study compares chromatin accessibility in therapy-resistant vs. sensitive cancer cells.
ATAC-Seq reveals that resistant cells have open chromatin at promoters of drug efflux genes
(ABCB1, ABCG2) even BEFORE drug exposure — a pre-existing epigenetic state, not a mutation.
Traditional WES/WGS showed no genetic difference between sensitive and resistant cells.
→ What does this tell us about the limitations of DNA-only approaches?
→ How could epigenomic profiling change clinical stratification of cancer patients?

KEY CONCEPT
Epigenomics: the study of heritable gene expression changes NOT encoded in DNA sequence.
Three main NGS-based epigenomic assays:
ChIP-Seq (Chromatin ImmunoPrecipitation + Sequencing):
→ maps transcription factor binding sites or histone modifications genome-wide
→ requires: antibody (IP) + crosslinking + sonication + Illumina library prep
ATAC-Seq (Assay for Transposase-Accessible Chromatin with Sequencing):
→ maps open/accessible chromatin regions (promoters, enhancers) genome-wide
→ uses Tn5 transposase to simultaneously cut and ligate adapters to accessible DNA
WGBS (Whole Genome Bisulfite Sequencing):
→ maps DNA methylation (5mC) at single-CpG resolution genome-wide
→ bisulfite converts unmethylated C → U; methylated C remains C
→ or: ONT native sequencing directly detects 5mC without chemical treatment

9.1 ChIP-Seq Analysis Pipeline

9.1.1 Biological Background

ChIP-Seq captures protein-DNA interactions by: (1) crosslinking protein to DNA in living cells (formaldehyde); (2) shearing chromatin to ~200 bp fragments; (3) immunoprecipitating with an antibody targeting the protein of interest; (4) reversing crosslinks, purifying DNA, and sequencing. The resulting reads map to genomic regions where the protein of interest was bound.

Key applications: histone modification mapping (H3K27ac = active enhancers; H3K4me3 = active promoters; H3K27me3 = Polycomb repression); transcription factor binding site mapping; chromatin remodeler occupancy.

9.1.2 ChIP-Seq Analysis Pipeline

PRACTICAL EXERCISE — ChIP-Seq: QC, Alignment, and Peak Calling

```
# ChIP-Seq typical pipeline: FASTQ → QC → Align → Filter → Peak call → Annotate


# Step 1: FastQC (same as standard — check adapter content, quality)
$ fastqc ChIP_H3K27ac.fastq.gz --outdir fastqc_chipseq/

# Step 2: Trim adapters (Trim Galore = Cutadapt + FastQC wrapper):
$ trim_galore \
    --paired \
    --quality 20 \
    --illumina \
    --fastqc \
    ChIP_H3K27ac_R1.fastq.gz ChIP_H3K27ac_R2.fastq.gz \
    --output_dir trimmed/

# Step 3: Align to GRCh38 with Bowtie2 (standard for ChIP-Seq):
$ bowtie2 \
    -x GRCh38_bowtie2_index \
    -1 trimmed/ChIP_H3K27ac_R1_trimmed.fastq.gz \
    -2 trimmed/ChIP_H3K27ac_R2_trimmed.fastq.gz \
    -p 16 \
    --no-mixed --no-discordant \
    -X 2000 \
    | samtools sort -@ 4 -o ChIP_H3K27ac.bam -
$ samtools index ChIP_H3K27ac.bam

# Step 4: Filter: remove duplicates, low-MAPQ, blacklist regions:
$ picard MarkDuplicates I=ChIP_H3K27ac.bam O=ChIP_H3K27ac_mkdup.bam
M=mkdup_metrics.txt
$ samtools view -b -F 0x400 -q 30 ChIP_H3K27ac_mkdup.bam >
ChIP_H3K27ac_filtered.bam
$ bedtools intersect -v -abam ChIP_H3K27ac_filtered.bam \
    -b ENCFF356LFX_GRCh38_blacklist.bed > ChIP_H3K27ac_clean.bam
$ samtools index ChIP_H3K27ac_clean.bam
# Blacklist: regions of anomalous signal (centromeres, telomeres, etc.)
# ENCODE blacklists available at: encodeproject.org/annotations/ENCSR636HFF/

# Step 5: Peak calling with MACS2 (broad marks vs. narrow marks):
# Narrow peaks (transcription factors, H3K4me3, H3K27ac):
$ macs2 callpeak \
    -t ChIP_H3K27ac_clean.bam \
    -c input_control_clean.bam \
    -f BAM \
    -g hs \
```

```
    -n H3K27ac_peaks \
    --outdir macs2_peaks/ \
    -p 1e-5 \
    --nomodel --extsize 200

# Broad peaks (H3K27me3, H3K9me3 — repressive marks covering large domains):
$ macs2 callpeak -t ChIP_H3K27me3.bam -c input.bam -f BAM -g hs \
    -n H3K27me3 --broad --broad-cutoff 0.1 --outdir macs2_broad/

# Step 6: Annotate peaks (Homer):
$ annotatePeaks.pl macs2_peaks/H3K27ac_peaks_peaks.narrowPeak hg38 \
    -annStats peak_annotation_stats.txt > H3K27ac_annotated_peaks.txt
# Assigns each peak to: promoter, intron, exon, intergenic, TTS, 5'UTR, 3'UTR
```

✔ Expected Output / What to Look For:

MACS2 narrowPeak output: chr, start, end, peak name, score, strand, fold-enrichment, -log10pvalue, -log10qvalue, summit offset.

Typical H3K27ac ChIP-Seq: 20,000-100,000 peaks genome-wide.

QC: FRiP score (Fraction of Reads in Peaks) should be >0.1 (>10%) for good ChIP efficiency.

ENCODE quality standards: FRiP >0.3 for TF ChIP; >0.2 for histone ChIP.

9.1.3 ChIP-Seq QC Metrics

QC Metric	Description and Threshold
FRiP (Fraction of Reads in Peaks)	% of reads overlapping called peaks. TF ChIP: >0.3; Histone ChIP: >0.2. Low FRiP = poor immunoprecipitation.
NSC (Normalized Strand Correlation)	Signal-to-noise ratio. NSC >1.05 indicates reasonable ChIP quality.
RSC (Relative Strand Correlation)	Another strand correlation metric. RSC >0.8 = good; <1 = poor ChIP.
% mitochondrial reads	High % mtDNA reads indicates ATAC/ChIP of opened chromatin near mt. >20% = concern.
Peak number	H3K27ac: 20k-100k peaks. TF: 500-50k. Too few = antibody failure; too many = non-specific.

9.2 ATAC-Seq Analysis Pipeline

KEY CONCEPT

ATAC-Seq uses the Tn5 transposase to simultaneously fragment and tag open chromatin regions.

Tn5 preferentially inserts its adapters into accessible (nucleosome-free) DNA.

The resulting sequencing reads map to open chromatin — promoters, enhancers, insulator elements.

Key features of ATAC-Seq data:

- Nucleosome-free regions (NFR): <150 bp fragments → transcription factor binding sites

- Mononucleosome fragments: 150-300 bp
- Dinucleosome fragments: 300-500 bp
- Fragment size distribution is diagnostic of ATAC-Seq library quality

PRACTICAL EXERCISE — ATAC-Seq Pipeline with ENCODE Standards

```
# Step 1: QC and trimming (Trim Galore with paired-end):

$ trim_galore --paired --nextera \
    ATAC_R1.fastq.gz ATAC_R2.fastq.gz \
    --output_dir atac_trimmed/
# --nextera: remove Nextera transposase adapter sequences used in ATAC-Seq kits

# Step 2: Align with Bowtie2 (ATAC requires local alignment, larger insert sizes):
$ bowtie2 -x GRCh38_bowtie2_index \
    -1 atac_trimmed/ATAC_R1_trimmed.fastq.gz \
    -2 atac_trimmed/ATAC_R2_trimmed.fastq.gz \
    -X 2000 --local \
    -p 16 \
    | samtools sort -@ 4 -o ATAC_aligned.bam -
$ samtools index ATAC_aligned.bam

# Step 3: Filter duplicates, low-MAPQ, mitochondrial reads, and blacklisted regions:
$ samtools view -b -F 0x400 -q 30 ATAC_aligned.bam \
    | grep -v chrM \
    | samtools view -b > ATAC_filtered_noMT.bam
# chrM reads MUST be removed — mitochondrial DNA is entirely accessible to Tn5

# Step 4: Check fragment size distribution (CRITICAL QC for ATAC-Seq):
$ picard CollectInsertSizeMetrics \
    I=ATAC_filtered_noMT.bam \
    O=ATAC_insert_metrics.txt \
    H=ATAC_insert_histogram.pdf \
    M=0.5
# Good ATAC-Seq: nucleosomal ladder visible at ~200, 400, 600 bp
# First peak <150 bp (NFR) should be the TALLEST peak

# Step 5: Shift reads to correct for Tn5 insertion bias:
# Tn5 cuts between two nucleotides; actual cut site is offset by +4/-5 bp
$ deeptools alignmentSieve \
    --ATACshift \
    --bam ATAC_filtered_noMT.bam \
    --outFile ATAC_shifted.bam \
    --numberOfProcessors 8
$ samtools index ATAC_shifted.bam

# Step 6: Peak calling (NFR peaks = open chromatin regions):
$ macs2 callpeak \
    -t ATAC_shifted.bam \
    -f BAMPE \
```

```
    -g hs \
    -n ATAC_peaks \
    --outdir macs2_atac/ \
    --nomodel --shift -100 --extsize 200 \
    --keep-dup all \
    -p 0.01

# Step 7: Generate genome coverage tracks for IGV visualization:
$ bamCoverage \
    --bam ATAC_shifted.bam \
    --outFileName ATAC_coverage.bigwig \
    --normalizeUsing RPKM \
    --binSize 10 \
    --numberOfProcessors 8
```

✔ **Expected Output / What to Look For:**

Fragment size histogram: should show clear nucleosomal ladder (NFR <150 bp dominant).

If NFR peak is absent or very small: Tn5 digestion was incomplete — suboptimal ATAC-Seq.

FRiP score: >0.2 expected for ATAC-Seq.

Typical peak count: 50,000-200,000 ATAC-Seq peaks for a mammalian cell line.

9.3 DNA Methylation Analysis — WGBS

KEY CONCEPT

Bisulfite sequencing principle:

Unmethylated cytosine (C) → sodium bisulfite converts → Uracil (U) → sequenced as T

Methylated cytosine (5mC) → bisulfite CANNOT convert → remains C → sequenced as C

Therefore: C in reads at CpG positions = methylated; T in reads = unmethylated.

WGBS generates two types of reads:

OT (Original Top) strand: C → T conversion on top strand

OB (Original Bottom) strand: C → T conversion on bottom strand

Alignment requires a bisulfite-aware aligner (Bismark or bwa-meth).

ONT alternative: direct detection of 5mC from raw current signal (Dorado with 5mC model)

without bisulfite treatment — preserves DNA, enables simultaneous sequence + methylation calling.

PRACTICAL EXERCISE — WGBS Analysis with Bismark

```
# Step 1: Prepare bisulfite reference genome index (one-time, ~2 hours):
```

```
$ bismark_genome_preparation GRCh38/
# Creates: GRCh38/Bisulfite_Genome/ with CT and GA converted genomes

# Step 2: Trim adapters (WGBS requires specific trimming for bisulfite artifacts):
$ trim_galore \
    --paired \
    --quality 20 \
    --clip_r1 8 --clip_r2 8 \
    WGBS_R1.fastq.gz WGBS_R2.fastq.gz \
    --output_dir wgbs_trimmed/
# --clip_r1/r2 8: remove 8 bp from 5' of each read
# WGBS-specific: remove position-specific methylation bias at read ends

# Step 3: Bisulfite alignment with Bismark:
$ bismark \
    --genome GRCh38/ \
    -1 wgbs_trimmed/WGBS_R1_trimmed.fastq.gz \
    -2 wgbs_trimmed/WGBS_R2_trimmed.fastq.gz \
    --output_dir bismark_output/ \
    --parallel 4
# Parallel: uses multiple cores for faster alignment

# Step 4: Deduplicate (remove PCR duplicates):
$ deduplicate_bismark \
    --paired \
    --bam bismark_output/WGBS_R1_trimmed_bismark_bt2_pe.bam

# Step 5: Extract methylation calls (CpG contexts only):
$ bismark_methylation_extractor \
    --paired-end \
    --CpG_only \
    --comprehensive \
    --cytosine_report \
    --genome_folder GRCh38/ \
    --output bismark_meth/ \
    bismark_output/*deduplicated.bam

# Output: CpG_context_*.txt — one line per CpG with methylation count
# Columns: chr, position, strand, methylated_count, unmethylated_count, context,
trinucleotide

# Step 6: Import into R for differential methylation analysis (DSS/MethylKit):
# library(DSS)
# bs <- read.bismark('CpG_context_WGBS.txt.gz')
# dml <- DMLtest(bs, group1='treated', group2='control', smoothing=TRUE)
# dmr <- callDMR(dml, p.threshold=0.01)
```

✔ **Expected Output / What to Look For:**

Bismark alignment rate: expect 50-80% for WGBS (lower than standard WGS due to bisulfite conversion).

Bisulfite conversion rate: check non-CpG cytosine methylation — should be <2% for good conversion.

CpG methylation: CpG methylation output file shows each CpG with % methylation.
Differential methylation: DSS DMR output: chr, start, end, areaStat, p.value (DMR = differentially methylated region).

INFORMATION

ONT Native Methylation Calling — The Future Standard:

Dorado (ONT basecaller) with the --modified-bases 5mCG model simultaneously calls DNA sequence AND 5mC methylation from the same raw POD5 signal — no bisulfite needed.

```
$ dorado basecaller --modified-bases 5mCG dna_r10.4.1_e8.2_400bps_hac@v4.3.0 \
    raw_pod5_dir/ > calls_with_methylation.bam

$ modkit pileup calls_with_methylation.bam methylation.bed \
    --ref GRCh38.fa --preset traditional
```

Advantages: uses native unmodified DNA; preserves starting material; detects 5mC, 5hmC, 6mA.

Emerging use: cfDNA methylation profiling from ONT liquid biopsy for cancer detection.

9.4 Integrative Epigenomic Analysis

In modern epigenomics, a single assay is rarely sufficient. Integrating multiple epigenomic layers reveals regulatory mechanisms invisible to single-assay approaches.

Integration	Tools	Biological Insight
ChIP-Seq H3K27ac + ATAC-Seq	bedtools intersect, deeptools plotHeatmap	Active enhancers (open chromatin + active mark)
ATAC-Seq + RNA-Seq	Homer findMotifs, GREAT, limma	Open promoters drive gene expression correlation
WGBS + RNA-Seq	methylKit + DESeq2, MethylMix	Promoter hypermethylation silences tumor suppressors
ChIP-Seq (multiple marks) → chromatin states	ChromHMM, Segway	15-25 chromatin state annotation genome-wide
ATAC-Seq + scRNA-Seq	Signac (R), ArchR, SCENIC+	Single-cell gene regulatory network inference

PRACTICAL EXERCISE — Visualizing Epigenomic Data with deepTools

```
# Convert BAM to bigWig for genome browser visualization:

$ bamCoverage \
    --bam ChIP_H3K27ac_clean.bam \
    --outFileName H3K27ac.bigwig \
    --normalizeUsing RPKM \
```

```
    --binSize 10 \
    --smoothLength 60 \
    --numberOfProcessors 8

# Compute correlation matrix across multiple ChIP/ATAC samples:
$ multiBamSummary bins \
    --bamfiles H3K27ac.bam H3K4me3.bam ATAC.bam Input.bam \
    --outFileName multibam_summary.npz \
    --binSize 10000 \
    --numberOfProcessors 8

$ plotCorrelation \
    -in multibam_summary.npz \
    --corMethod spearman \
    --skipZeros \
    --plotType heatmap \
    --colorMap RdYlBu \
    -o correlation_heatmap.pdf

# Generate heatmap of signal at gene TSSs (Transcription Start Sites):
$ computeMatrix reference-point \
    --scoreFileName H3K27ac.bigwig H3K4me3.bigwig \
    --regionsFileName Gencode_v43_TSSs.bed \
    --referencePoint TSS \
    --upstream 2000 --downstream 2000 \
    --outFileName TSS_matrix.gz \
    --numberOfProcessors 8

$ plotHeatmap \
    -m TSS_matrix.gz \
    -out TSS_heatmap.pdf \
    --colorMap RdBu \
    --whatToShow 'heatmap and colorbar'
```

✔ **Expected Output / What to Look For:**

bigWig files: upload to IGV or UCSC Genome Browser for visualization.

Correlation heatmap: ChIP samples of same mark should correlate >0.9; different marks should differ.

TSS heatmap: H3K27ac and H3K4me3 should show enrichment centered on active gene TSSs.

? REVIEW QUESTIONS

1. A ChIP-Seq experiment targeting the H3K27ac mark (active enhancers) produces a FRiP score of 0.05.

What does this indicate? List three possible causes and the investigative steps.

2. ATAC-Seq fragment size distribution shows a single peak at 400 bp with no NFR (<150 bp) component.

What went wrong in the experiment? What does this mean for data interpretation?

3. In WGBS analysis, you observe 8% non-CpG methylation (in a mammalian sample).
What does this indicate about the bisulfite conversion reaction?
How does this affect interpretation of CpG methylation values?
4. A cancer sample shows ATAC-Seq peaks at the CDKN2A (p16) promoter in sensitive cells
but NO peak at the same promoter in resistant cells. WGBS shows 90% CpG methylation at
CDKN2A in resistant cells. Integrate these findings into a mechanistic explanation.

Appendix: Quick-Reference Tool Compendium

This appendix provides a consolidated reference table of all bioinformatics tools covered in this textbook, organized by analytical step. Use this during practical sessions to identify the correct tool for each step.

Step / Category	Tool(s)	Key Use Case
Library QC	Bioanalyzer, TapeStation	Fragment size distribution, DIN/RIN
Run QC	Illumina SAV / InterOp	Cluster density, %PF, Q30, PhiX error rate
Read QC	FastQC	Per-sample read quality, adapter content, duplication
Multi-sample QC	MultiQC	Aggregate FastQC + other tool reports
Read trimming (Illumina)	Trimmomatic, fastp, Cutadapt/Trim Galore!	Adapter removal, quality trimming
Read QC (ONT)	NanoPlot, NanoStat	Read length and quality distributions
Adapter removal (ONT)	Porechop, Dorado (built-in)	ONT-specific adapter trimming
Alignment (short read)	BWA-MEM2, Bowtie2	Map reads to reference genome (DNA)
Alignment (RNA-Seq)	STAR, HISAT2	Splice-aware alignment for RNA-Seq
Alignment (fast, clinical)	DRAGEN, Sentieon	Hardware-accelerated clinical pipeline
Alignment QC	samtools flagstat, Picard CollectAlignmentMetrics	Mapping rate, duplicate rate, insert size
Duplicate marking	Picard MarkDuplicates, samtools markdup	PCR/optical duplicate flagging
Variant calling (germline)	GATK HaplotypeCaller, DeepVariant	SNVs/indels in constitutional DNA
Variant calling (somatic)	GATK Mutect2, Sentieon TNscope	Somatic mutations in tumor vs. normal
Variant calling (long read)	GATK HaplotypeCaller (PacBio HiFi), Sniffles2 (ONT SVs)	SVs with ONT; accurate SNVs with HiFi
Variant filtering	GATK VQSR, FilterMutectCalls	Quality recalibration and artifact filtering
Variant annotation	ANNOVAR, SnpEff, VEP	Gene effect, population AF, clinical significance
Variant visualization	IGV (desktop/web)	Manual review of BAM reads at variant loci
scRNA-Seq preprocessing	Cell Ranger (10x)	FASTQ → BAM → feature-barcode matrix
scRNA-Seq analysis	Seurat (R)	QC, normalization, clustering, UMAP, annotation
scRNA-Seq DE	FindMarkers (Seurat), DESeq2 pseudobulk	Differential expression between clusters
Pathway enrichment	clusterProfiler (R), GSEA	GO/KEGG pathway enrichment analysis
Trajectory analysis	Monocle3 (R)	Pseudotime and cell differentiation trajectories
16S QC	QIIME2 demux summarize	Quality visualization before DADA2
16S denoising	DADA2 (via QIIME2)	Error correction, ASV generation, chimera removal

Step / Category	Tool(s)	Key Use Case
16S taxonomy	QIIME2 classify-sklearn (SILVA 138)	Taxonomic classification of ASVs
16S diversity	QIIME2 diversity core-metrics-phylogenetic	Alpha and beta diversity metrics, PCoA
Shotgun taxonomy	MetaPhlAn4, KrakenUniq	Species-level classification of shotgun reads
Shotgun function	HUMAnN3	Metabolic pathway abundance from shotgun
Resistome	CARD / ResFinder	AMR gene identification from metagenomics

Recommended Reading and Resources

The following resources complement this textbook with deeper theoretical and practical content:

- **Van der Auwera & O'Connor (2020): Genomics in the Cloud — O'Reilly Media. The definitive practical guide to GATK Best Practices.**
- **Heumos et al. (2023): Best practices for single-cell analysis across modalities — Nature Reviews Genetics. Comprehensive scRNA-Seq analytical framework review.**
- **Callahan et al. (2016): DADA2: High-resolution sample inference from Illumina amplicon data — Nature Methods. Original DADA2 paper.**
- **Andrews S: FastQC Documentation — bioinformatics.babraham.ac.uk/projects/fastqc. Official FastQC module descriptions.**
- **GATK Best Practices Documentation — gatk.broadinstitute.org. The authoritative source for GATK pipeline protocols.**
- **Seurat Documentation & Tutorials — satijalab.org/seurat. Comprehensive Seurat vignettes for all workflows.**
- **QIIME2 Documentation — docs.qiime2.org. Step-by-step tutorials including 'Moving Pictures' and 'Atacama Soil Microbiome'.**
- **nf-core Pipelines — nf-co.re. Community-validated Nextflow pipelines: sarek (variant calling), rnaseq (bulk RNA-Seq), scrnaseq (single-cell), ampliseq (16S), taxprofiler (metagenomics).**
- **Bioconductor — bioconductor.org. R/Bioconductor packages: DESeq2, edgeR, clusterProfiler, GenomicRanges, Biostrings.**
- **EMBL-EBI Training — training.ebi.ac.uk. Free online courses: Bioinformatics for Biologists, Introduction to NGS Data Analysis.**

Annex: Introduction to Bioinformatics

Learning Objectives

- Navigate a Linux filesystem confidently and write simple shell pipelines
- Use Git to document, version-control, and share computational analyses

Key Concept: What is Next-Generation Sequencing?

Next-Generation Sequencing (NGS) — also called high-throughput sequencing or massively parallel sequencing — refers to sequencing technologies that can produce millions to billions of short DNA reads in a single instrument run. Unlike Sanger sequencing (first-generation, one fragment at a time), NGS parallelises the sequencing reaction across an entire flow cell containing millions of reaction sites simultaneously. The result is a dramatic reduction in cost per base (from ~$10 per base in 1990 to less than $0.000001 today) and a dramatic increase in throughput, enabling projects that were previously unthinkable.

The term 'third-generation sequencing' (TGS) is increasingly used for platforms such as Oxford Nanopore (ONT) and PacBio that sequence individual DNA molecules without prior amplification, producing much longer reads (thousands to hundreds of thousands of bases) in real time.

1 The Linux Command Line — Your Essential Bioinformatics Tool

Virtually all bioinformatics software runs on Unix-based operating systems (Linux, macOS). Understanding the command-line interface (CLI) is therefore non-negotiable for any practising bioinformatician. This section introduces the essential commands you will use throughout this course, with detailed explanations of how each works and why it matters.

1. 1 The Linux Filesystem: Concepts and Navigation

The Linux filesystem is a single hierarchical tree of directories starting at the root, written as /. Every file and directory on the system has a unique path — a string that describes its location in this tree.

Paths can be expressed in two ways. An absolute path starts from the root and describes the complete location: /home/student/course/data/raw/sample1.fastq. A relative path describes a location relative to your current working directory: if you are already in /home/student/course/, the relative path to the same file is data/raw/sample1.fastq. Two special relative-path shortcuts are . (the current directory) and .. (the parent directory). Understanding this distinction is essential because most bioinformatics tools require you to specify file paths, and errors in paths are the most common cause of 'file not found' errors.

```
# Print the absolute path of your current working directory
$ pwd

# List the contents of the current directory
$ ls

# List with details: permissions, owner, file size (human-readable), date
$ ls -lh

# List hidden files (those starting with .) as well
```

```
$ ls -lha

# Change into a directory using an absolute path
$ cd /home/student/course_data

# Change into a subdirectory using a relative path
$ cd data/raw

# Go one level up (to the parent directory)
$ cd ..

# Go to your home directory (shortcut: ~ always expands to /home/your_username)
$ cd ~
```

Tip: Tab Completion is Your Best Friend

Instead of typing long file names in full, press the Tab key after typing the first few characters. If the path is unambiguous, the shell will complete it automatically. If there are multiple matches, press Tab twice to see all options. This both saves time and prevents typos — critical when working with long sample names like ERR4082015_R1_001.fastq.gz.

1. 2 Managing Files and Directories

Bioinformatics projects generate large numbers of files. A consistent directory structure is essential for reproducibility and for finding files weeks or months later. Adopt a structure from day one and commit it to your Git repository (see Section 1.5).

```
# Create a single new directory
$ mkdir results

# Create nested directories in a single command (-p flag = create parents as
needed)
$ mkdir -p project/data/raw project/data/processed project/results
project/scripts

# Copy a file (cp SOURCE DESTINATION)
$ cp sample1.fastq /home/student/project/data/raw/

# Copy an entire directory and its contents (-r = recursive)
$ cp -r scripts/ backup_scripts/

# Move or rename a file (mv SOURCE DESTINATION)
$ mv sample_old_name.fastq sample_correct_name.fastq

# Delete a file — WARNING: no recycle bin, deletion is permanent!
$ rm file_to_delete.txt

# Delete a directory and all its contents (-r recursive, -f force — use with
care!)
$ rm -rf directory_name/

# Check file size
$ du -sh sample1.fastq.gz

# Check available disk space on the filesystem
$ df -h .
```

> ⚠ **Warning: rm -rf is Irreversible**
>
> Linux has no recycle bin. rm deletes files immediately and permanently. rm -rf (recursive force) will delete entire directory trees without asking for confirmation. Before running any rm command, double-check the path with ls. A common safety practice is to use ls where you plan to use rm first, to visually confirm what will be deleted. Never run rm -rf / or rm -rf * in your home directory.

1. 3 Viewing and Searching File Contents

FASTQ files can contain hundreds of millions of lines. You will almost never want to open them in a text editor. Instead, use command-line tools designed to handle large files efficiently.

```
# View the first 10 lines of a file (useful for checking FASTQ format)
$ head sample1.fastq

# View the first N lines (here: first 40 lines = first 10 reads in a FASTQ)
$ head -n 40 sample1.fastq

# View the last 10 lines
$ tail sample1.fastq

# Interactively scroll through a file (space = next page, q = quit, / = search)
$ less sample1.fastq

# Count the number of lines, words, and characters in a file
$ wc -l sample1.fastq        # lines only
$ wc -c sample1.fastq        # bytes (file size)

# Search for lines matching a pattern (-c = count matching lines)
$ grep '@SRR' sample1.fastq | head -5
$ grep -c '^@' sample1.fastq    # count read headers (starts with @)

# Count reads in a FASTQ (each read = 4 lines)
$ echo $(( $(wc -l < sample1.fastq) / 4 ))

# Count reads in a GZIPPED FASTQ (zcat decompresses to stdout)
$ zcat sample1.fastq.gz | wc -l | awk '{print $1/4}'
```

> 📌 **Key Concept: FASTQ File Format**
>
> Every sequencing read in a FASTQ file is represented by exactly four lines:
>
> 1. Header line: begins with @ followed by a unique read identifier and metadata (instrument, flow cell, coordinates).
> 2. Sequence line: the actual DNA sequence (A, T, G, C, and N for ambiguous calls).
> 3. Separator line: a + character (optionally followed by the read name again).
> 4. Quality line: one ASCII character per base encoding the Phred quality score (Q = -10 × $\log_{10}$(P_error)). A Q of 30 means a 1-in-1000 chance of error; Q20 means 1-in-100.
>
> Because each read is always exactly 4 lines, dividing the total line count by 4 always gives the number of reads. This is why the command `wc -l | awk '{print $1/4}'` works reliably.

1. 4 Redirection, Pipes, and Shell Scripting

The real power of the Linux command line comes from combining commands. The pipe operator | sends the standard output (stdout) of one command directly into the standard input (stdin) of the next, allowing complex multi-step analyses to be written in a single line. Redirection operators > and >> write output to files.

```
# Pipe: count reads by searching for headers and counting matching lines
$ grep -c '^@ERR' sample1.fastq

# Pipe chain: decompress → search → count
$ zcat sample1.fastq.gz | grep '^@' | wc -l

# Redirect stdout to a new file (overwrites if file exists)
$ fastqc sample1.fastq.gz --outdir qc_results/ > fastqc_stdout.txt 2>
fastqc_errors.txt

# Append output to an existing file (does not overwrite)
$ echo 'sample1 processed' >> processing_log.txt

# Run a command on multiple files using a wildcard (*)
$ fastqc data/raw/*.fastq.gz --outdir qc_results/

# Loop over all FASTQ files in a directory
$ for f in data/raw/*.fastq.gz; do
$     echo "Processing: $f"
$     fastqc "$f" --outdir qc_results/
$ done

# Check whether a command succeeded (exit code 0 = success, non-zero = error)
$ echo $?
```

For analyses involving multiple samples, writing a shell script (a text file containing a sequence of commands, saved with the .sh extension) is far more reproducible than typing commands interactively. Scripts can be version-controlled with Git, shared with collaborators, and re-run exactly as written. A minimal shell script looks like this:

```
#!/bin/bash
# Script: 01_run_fastqc.sh
# Purpose: Run FastQC on all raw FASTQ files
# Author: Your Name | Date: 2025-03-01

# Activate the conda environment where FastQC is installed
conda activate qc_env

# Create output directory if it does not exist
mkdir -p results/qc

# Run FastQC on all gzipped FASTQ files in the raw data directory
fastqc data/raw/*.fastq.gz \
    --outdir results/qc/ \
    --threads 4

echo 'FastQC complete. Results in results/qc/'
```

Tip: Conda Environments — Software Management in Bioinformatics

Bioinformatics involves dozens of software tools, each with different — often conflicting — dependencies. Conda is a package and environment manager that allows you to create isolated software environments, each with their own versions of Python, R, and any tools you need. This means you can have QIIME2 (which requires Python 3.8) and a tool requiring Python 3.11 installed on the same system without conflict.

```
# List your conda environments
conda env list
# Activate an environment
conda activate qiime2-amplicon-2024.10
# Deactivate and return to base
conda deactivate
```

Practical Exercise 1.1: Linux Basics

Complete the following tasks on the course server. Record all commands in a text file called linux_exercise.sh.

1. Log in using the credentials provided. Print your current directory with pwd.
2. Create the directory structure: course/data/raw, course/data/processed, course/results, course/scripts.
3. List the contents of course/ in long format. How many directories does it contain? What are their permissions?
4. Locate the file /data/shared/sample_reads.fastq.gz. Count the number of reads it contains and write the count to course/results/read_count.txt.
5. Use head to display the first two reads (8 lines) of the file. What information is in the header line? What does the quality line look like?
6. Use grep to count how many lines in the file start with '@'. Compare to your read count from step 4. Are they equal? Why or why not?
7. Write a one-line command using a pipe that counts reads directly from a gzipped file without decompressing it to disk.

2 Version Control with Git

Reproducibility is the cornerstone of scientific practice. In bioinformatics, every command you run, every parameter you choose, and every script you write is part of your methods section — it must be recorded, documented, and retrievable. Git is a distributed version control system that tracks changes to files over time, allowing you to record exactly what you did and when, revert to any previous state, collaborate without overwriting each other's work, and share your analysis so others can reproduce it exactly.

Git has become the universal standard for managing computational analyses in biology. Many journals and funding agencies now require that all analysis code be deposited in a public repository (GitHub, GitLab, Zenodo) as a condition of publication.

2.1 Core Git Concepts

Git Term	Meaning	Ecological Analogy
Repository (repo)	A directory tracked by Git. Contains all files and the complete history of every change ever made to them.	The field notebook for an entire study, with every observation, correction, and revision preserved.

Git Term	Meaning	Ecological Analogy
Commit	A snapshot of the repository at a specific point in time. Identified by a unique 40-character hash (e.g., a3f8c2d).	A dated entry in the field notebook: 'On 2025-03-01, I ran FastQC with these parameters and found...'
Branch	An independent line of development. Allows you to test a new analysis approach without affecting the main version.	A separate sub-notebook for testing a new eDNA protocol, kept separate from the validated methods.
Remote	A copy of the repository hosted on a server (e.g., GitHub, GitLab). Enables backup, sharing, and collaboration.	A backup copy of the field notebook stored in a different location.
Clone	Downloading a remote repository to your local machine.	Printing a full copy of a colleague's field notebook to work from.
Push / Pull	Push: send your local commits to the remote. Pull: download new commits from the remote to your local copy.	Updating the shared backup notebook with your latest entries (push) or downloading a collaborator's latest entries (pull).
.gitignore	A text file listing files and patterns that Git should not track (e.g., raw data files, temporary outputs).	A note at the front of the notebook: 'Do not copy pages 1-50 (raw field measurements too large to duplicate).'

Table 1.4. Core Git concepts with ecological analogies to aid understanding.

2.2 Essential Git Commands

```
# First-time setup: tell Git who you are (only needed once per machine)
$ git config --global user.name 'Your Name'
$ git config --global user.email 'your.email@university.edu'

# Clone the course repository to your local machine
$ git clone https://github.com/tifi/ngs_course.git
$ cd ngs_course

# Check what has changed since your last commit
$ git status

# See exactly what changed inside a file
$ git diff results/read_count.txt

# Stage a specific file for the next commit
$ git add scripts/01_run_fastqc.sh

# Stage all changed files at once
$ git add .

# Commit staged changes with a descriptive message
$ git commit -m 'Add FastQC script and QC results for all 12 soil samples'

# Push your commits to the remote repository
$ git push origin main

# Download updates from collaborators
$ git pull

# View the commit history (one line per commit)
```

```
$ git log --oneline

# Revert a file to its state at the last commit (CAREFUL: discards uncommitted
changes)
$ git checkout -- scripts/01_run_fastqc.sh

# Show what a file looked like in a previous commit (use hash from git log)
$ git show a3f8c2d:scripts/01_run_fastqc.sh
```

Tip: What Makes a Good Commit Message?

A commit message should complete the sentence: 'This commit will...'. The first line should be a concise summary (≤72 characters). For complex changes, add a blank line then a detailed explanation.

Good commit messages:

- 'Add FastQC results for all 12 samples; flag sample S04 for low read count'
- 'Fix sample ID mismatch between metadata table and FASTQ filenames'
- 'Update trimming parameters based on QC review: reduce quality threshold from Q25 to Q20 for Nanopore data'

Poor commit messages:

- 'update', 'fixed', 'stuff', 'final', 'FINAL2', 'done'

Your commit history is your lab notebook. A reviewer examining your repository six months from now — or a journal editor verifying reproducibility — should be able to reconstruct what you did and why from the commit messages alone.

2.3 A Reproducible Project Structure

Before writing a single line of code, create a standardised directory structure and commit it to Git. The following structure is recommended for all projects in this course and closely mirrors what is expected in published computational ecology papers:

```
ngs_project/
├── README.md              # Overview: project description, authors, software
versions
├── .gitignore             # Files NOT tracked by Git (raw data, large outputs)
├── data/
│   ├── raw/               # Original, unmodified FASTQ files (often symlinked,
not copied)
│   ├── processed/         # QC-trimmed reads, adapter-removed files
│   └── metadata/          # Sample sheet, collection metadata, metadata tables
├── results/
│   ├── qc/                # FastQC and MultiQC reports
│   ├── taxonomy/          # QIIME2 artifacts, taxonomy tables
│   ├── diversity/         # Alpha and beta diversity outputs
│   └── figures/           # Publication-ready figures
├── scripts/               # Shell scripts, Python scripts, R scripts
│   ├── 01_fastqc.sh
│   ├── 02_trimming.sh
│   └── 03_qiime2.sh
└── notebooks/             # Jupyter notebooks for exploratory analysis
    └── 01_diversity_analysis.ipynb
```

Notice that raw data is tracked in Git but not stored in Git. Raw FASTQ files are typically too large (gigabytes to terabytes) for a Git repository. Instead, store raw data on the course server or an institutional data archive (e.g., EMBL-EBI European Nucleotide Archive for published data), and record the file checksums and download commands in your README.md.

Practical Exercise 1.2: Git Setup and First Commit

Complete on the course server. All commands should be recorded in your shell script from Exercise 1.1.

1. Clone the course repository: git clone [URL provided day of course].
2. Inside the repository, create a directory called students/[your_name]/ and inside it create a file called notes.md.
3. In notes.md, write three things you already know about NGS and three questions you have. Save the file.
4. Use git status to see what has changed. Use git add to stage your new file. Use git commit with a descriptive message. Use git push to upload your commit.
5. Use git log --oneline to view the last five commits in the repository. Who made the most recent commit before yours? What did they change?
6. Create a .gitignore file in your student directory that excludes any file ending in .fastq.gz and any directory called temp/. Commit this file.

3 Anatomy of a Bioinformatics Pipeline

A bioinformatics pipeline is a series of computational steps that transform raw sequencing data into biological conclusions. Understanding the structure of a pipeline helps you plan your analysis, anticipate where problems can arise, and know what quality checks to perform at each stage. Although pipelines vary depending on the research question and sequencing platform, virtually all NGS analyses in environmental science follow the same six-step skeleton.

Step	Name	Key Activities	Tools Used (Examples)	What Can Go Wrong
1	Data Receipt & Organisation	Receive raw files; verify checksums; organise directories; record metadata; commit to Git	md5sum, sha256sum, Git	Missing files; corrupted downloads; sample ID mismatches in metadata
2	Quality Control	Assess raw read quality; identify adapter contamination; flag problematic samples	FastQC, MultiQC, NanoPlot, NanoStat	Overlooking low-quality samples; misinterpreting per-base quality plots
3	Pre-processing (Trimming)	Remove adapter sequences; quality-trim low-scoring bases; discard short reads	Trimmomatic, fastp (Illumina); Porechop, Filtlong (Nanopore)	Over-trimming (removes real signal); under-trimming (adapters contaminate results)
4	Core Analysis	The central biological step: taxonomic classification, genome assembly, alignment, etc.	QIIME2, Kraken2, MEGAHIT, BWA, STAR — depends on application	Incorrect reference database; wrong parameters; ignoring error rates
5	Post-processing & Statistics	Filter, normalise, calculate diversity metrics, perform statistical tests	R (vegan, phyloseq, DESeq2), Python (scipy, pandas)	Using inappropriate statistical tests; not accounting for sequencing depth (rarefaction)

Step	Name	Key Activities	Tools Used (Examples)	What Can Go Wrong
6	Visualisation & Reporting	Generate figures; write methods; archive code and data	R (ggplot2), Python (matplotlib, seaborn), Jupyter	Methods not reproducible; parameters not reported; insufficient detail for replication

Table 1.5. The six-step anatomy of an NGS bioinformatics pipeline with common pitfalls at each stage.

3.1 Step 1: Data Receipt and Organisation — In Detail

The first action when receiving raw sequencing data is to verify its integrity. Data corruption can occur during transfer (network errors) or storage (bitrot on hard drives). Every FASTQ file should be accompanied by a checksum file — a short text file containing a unique fingerprint of each data file's content.

```
# Verify file integrity using MD5 checksums
# (The checksums_file.md5 is provided by the sequencing facility)
$ md5sum -c checksums_file.md5

# If all files are intact, you will see: sample1.fastq.gz: OK
# A FAILED message means the file is corrupted — do not proceed!

# Generate checksums for your own files (do this before and after any transfer)
$ md5sum data/raw/*.fastq.gz > data/raw/my_checksums.md5
$ cat data/raw/my_checksums.md5
```

After verifying integrity, record all relevant sample metadata in a structured table. Metadata is the information about your samples — collection site, date, environmental conditions, sample type, extraction method, library preparation kit, and any other variables that might affect interpretation. Metadata tables should be stored in plain text (TSV or CSV format) and committed to your Git repository on day one. Losing metadata is often more damaging than losing raw data: the sequences can be re-sequenced, but the ecological context of a sample cannot be recovered.